ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/277814853

# Algoritmos y Estructura de Datos, Parte 7: Árboles y Gráficas

**Book** · June 2015

CITATIONS
0

READS
19,684

**5 authors**, including:



Reiner Creutzburg
Brandenburg University of Applied Sciences
**512** PUBLICATIONS **506** CITATIONS

SEE PROFILE

**Some of the authors of this publication are also working on these related projects:**



Advanced Cybersecurity and Cyberforensics View project



14. Nachwuchswissenschaftlerkonferenz Ost- und Mitteldeutscher Fachhochschulen (NWK 14) View project

All content following this page was uploaded by Reiner Creutzburg on 28 June 2015.

The user has requested enhancement of the downloaded file.

# Algoritmos y Estructura de Datos

## Parte 7 de 7: Árboles y Gráficas

Libros de Wikipedia 2015

Por Wikipedians

**EDITADO POR:**

*Reiner Creutzburg, Montserrat Rodríguez, Dulce García, María Martínez*
*Fachhochschule Brandenburg*
*Fachbereich Informatik und Medien*
*PF 2132*
*D-14737 Brandenburg , Germany*
*Email: creutzburg@fh-brandenburg.de*

# Índice general


**1 Árboles** **1**

1.1 Árbol (teoría de grafos) . . . 1
1.1.1 Definiciones . . . 1
1.1.2 Clasificación . . . 2
1.1.3 Propiedades . . . 2
1.1.4 Véase también . . . 2
1.1.5 Enlaces externos . . . 2
1.2 Árbol binario . . . 2
1.2.1 Definición de teoría de grafos . . . 3
1.2.2 Tipos de árboles binarios . . . 3
1.2.3 Implementación en C . . . 3
1.2.4 Recorridos sobre árboles binarios . . . 4
1.2.5 Métodos para almacenar Árboles Binarios . . . 6
1.2.6 Codificación de árboles n-arios como árboles binarios . . . 6
1.2.7 Véase también . . . 7
1.2.8 Enlaces externos . . . 7
1.3 Árbol binario de búsqueda . . . 7
1.3.1 Descripción . . . 7
1.3.2 Implementación en Python . . . 9
1.3.3 Operaciones . . . 9
1.3.4 Tipos de árboles binarios de búsqueda . . . 14
1.3.5 Comparación de rendimiento . . . 14
1.3.6 Buscando el Árbol binario de búsqueda óptimo . . . 14
1.3.7 Véase también . . . 14
1.3.8 Referencias . . . 15
1.3.9 Enlaces externos . . . 15
1.4 Notación de infijo . . . 15
1.4.1 Véase también . . . 15
1.4.2 Enlaces externos . . . 15
1.5 Grafo completo . . . 16
1.5.1 Ejemplos . . . 16
1.5.2 Véase también . . . 16

1.5.3 Referencias . . . 16
1.6 Notación polaca . . . 16
1.6.1 Aritmética . . . 17
1.6.2 Programación de computadora . . . 17
1.6.3 Orden de las operaciones . . . 18
1.6.4 Notación polaca en lógica . . . 18
1.6.5 Autómata de pila . . . 18
1.6.6 Referencias . . . 18
1.6.7 Lecturas relacionadas . . . 19
1.6.8 Véase también . . . 19
1.7 Notación polaca inversa . . . 19
1.7.1 Funcionamiento . . . 20
1.7.2 Ventajas . . . 20
1.7.3 Desventajas . . . 20
1.7.4 El algoritmo RPN . . . 21
1.7.5 Convirtiendo desde la notación de infijo a la notación de postfijo . . . 21
1.7.6 Implementaciones . . . 21
1.7.7 Otros datos . . . 22
1.7.8 Véase también . . . 22
1.7.9 Referencias . . . 23
1.7.10 Enlaces externos . . . 23
1.8 Árbol binario de búsqueda auto-balanceable . . . 23
1.8.1 Véase también . . . 23
1.9 Rotación de árboles . . . 24
1.9.1 Ilustracion . . . 25
1.9.2 Ilustración detallada . . . 25
1.9.3 Invarianza en orden . . . 26
1.9.4 Rotaciones de reequilibrio . . . 28
1.9.5 Distancia de rotacion . . . 28
1.9.6 Ver también . . . 28
1.9.7 Referencias . . . 29
1.9.8 Links Externos . . . 29
1.10 Árbol-B . . . 29
1.10.1 Definición . . . 29
1.10.2 Definición técnica . . . 29
1.10.3 Altura: El mejor y el peor caso . . . 30
1.10.4 Estructura de los nodos . . . 30
1.10.5 Algoritmos . . . 31
1.10.6 Notas . . . 33
1.10.7 Véase también . . . 33
1.10.8 Referencias . . . 34

1.10.9 Enlaces externos . . . 34
1.11 Montículo (informática) . . . 34
1.11.1 Operaciones . . . 34
1.11.2 Véase también . . . 35
1.12 Montículo de Fibonacci . . . 36
1.13 Estructura de un Montículo de Fibonacci . . . 36
1.14 Implementación de operaciones . . . 37
1.15 Peor Caso . . . 37
1.16 Resumen de Tiempos de Ejecución . . . 37
1.17 Referencias . . . 38
1.18 Enlaces externos . . . 38
1.19 Véase también . . . 38

**2 Gráficas** **44**
2.1 Grafo . . . 44
2.1.1 Historia y problema de los puentes de Königsberg . . . 45
2.1.2 Definiciones . . . 45
2.1.3 Propiedades . . . 47
2.1.4 Ejemplos . . . 47
2.1.5 Grafos particulares . . . 47
2.1.6 Véase también . . . 48
2.1.7 Enlaces externos . . . 49
2.2 Teoría de grafos . . . 49
2.2.1 Historia . . . 50
2.2.2 Aplicaciones . . . 51
2.2.3 Tipos de grafos . . . 51
2.2.4 Representación de grafos . . . 52
2.2.5 Problemas de teoría de grafos . . . 52
2.2.6 Caracterización de grafos . . . 55
2.2.7 Algoritmos importantes . . . 58
2.2.8 Investigadores relevantes en Teoría de grafos . . . 58
2.2.9 Véase también . . . 59
2.2.10 Referencias . . . 60
2.2.11 Enlaces externos . . . 60
2.3 Grafo dirigido . . . 60
2.3.1 Terminología básica . . . 60
2.3.2 Referencias . . . 61
2.4 Matriz de adyacencia . . . 62
2.4.1 Construcción de la matriz a partir de un grafo . . . 62
2.4.2 Propiedades de la matriz de adyacencia . . . 62
2.4.3 Comparación con otras representaciones . . . 62
2.4.4 Aplicaciones . . . 62

2.4.5 Véase también . . . 62
2.5 Algoritmo de Floyd-Warshall . . . 63
2.5.1 Algoritmo . . . 63
2.5.2 Pseudocodigo . . . 63
2.5.3 Código en C++ . . . 63
2.5.4 Comportamiento con ciclos negativos . . . 64
2.5.5 Ejemplo . . . 64
2.5.6 Análisis . . . 67
2.5.7 Aplicaciones y generalizaciones . . . 67
2.5.8 Implementación del algoritmo de Floyd-Warshall . . . 67
2.5.9 Referencias . . . 67
2.5.10 Véase también . . . 68
2.5.11 Enlaces externos . . . 68
2.6 Problema del camino más corto . . . 68
2.6.1 Definición . . . 68
2.6.2 Algoritmos . . . 69
2.6.3 Algoritmos para caminos más cortos desde un origen . . . 69
2.6.4 Algoritmos para caminos más cortos entre todos los pares de vértices . . . 70
2.6.5 Aplicaciones . . . 70
2.6.6 Problemas relacionados . . . 70
2.6.7 Véase también . . . 70
2.6.8 Referencias . . . 71
2.6.9 Enlaces externos . . . 71
2.7 Búsqueda en anchura . . . 72
2.7.1 Procedimiento . . . 72
2.7.2 Pseudocódigo . . . 72
2.7.3 Referencias . . . 73
2.7.4 Véase también . . . 73
2.8 Búsqueda en profundidad . . . 73
2.8.1 Pseudocódigo . . . 73
2.8.2 Arcos DF . . . 73
2.8.3 Véase también . . . 74
2.8.4 Referencias . . . 74
2.9 Vuelta atrás . . . 74
2.9.1 Concepto . . . 74
2.9.2 Enfoques . . . 75
2.9.3 Diseño e implementación . . . 76
2.9.4 Heurísticas . . . 76
2.9.5 Ejemplos de aplicación de backtracking . . . 76
2.9.6 Ejemplos de problemas comunes resueltos usando Vuelta Atrás . . . 77
2.9.7 Aplicaciones . . . 77

2.9.8 Branch & Bound (Ramificación y poda) . . . 77
2.10 Ordenación topológica . . . 78
2.10.1 Algoritmos . . . 78
2.10.2 Ejemplos . . . 78
2.10.3 Véase también . . . 79
2.10.4 Referencias . . . 79
2.11 Algoritmo de Dijkstra . . . 79
2.11.1 Algoritmo . . . 80
2.11.2 Complejidad . . . 80
2.11.3 Pseudocódigo . . . 80
2.11.4 Otra versión en pseudocódigo sin cola de prioridad . . . 81
2.11.5 Véase también . . . 81
2.11.6 Enlaces externos . . . 81
2.12 Algoritmo voraz . . . 81
2.12.1 Esquema . . . 81
2.12.2 Ejemplos de algoritmos voraces . . . 82
2.12.3 Temas relacionados . . . 83
2.12.4 Referencias . . . 83
2.12.5 Enlaces externos . . . 83
2.13 Problema del viajante . . . 83
2.13.1 Historia . . . 83
2.13.2 Descripción . . . 85
2.13.3 Formulación de la programación lineal en enteros . . . 87
2.13.4 Calculando una solución . . . 88
2.13.5 Rendimiento humano en el TSP . . . 94
2.13.6 Longitud del camino en el TSP para conjuntos de puntos aleatorios en un cuadrado . . . 94
2.13.7 TSP de Analyst . . . 95
2.13.8 Software Libres para resolver el TSP . . . 95
2.13.9 Cultura Popular . . . 95
2.13.10 Vea también . . . 95
2.13.11 Notas . . . 95
2.13.12 Referencias . . . 96
2.13.13 También leer . . . 97
2.13.14 Enlaces externos . . . 98

**3 Text and image sources, contributors, and licenses 99**
3.1 Text . . . 99
3.2 Images . . . 101
3.3 Content license . . . 104

# Capítulo 1

# Árboles

## 1.1 Árbol (teoría de grafos)

En teoría de grafos, un **árbol** es un grafo en el que cualesquiera dos vértices están conectados por *exactamente un* camino. Un **bosque** es una unión disjunta de árboles. Un árbol a veces recibe el nombre de *árbol libre*.

### 1.1.1 Definiciones

Un **árbol** es un grafo simple no dirigido *G* que satisface:

1. *G* es conexo y no tiene ciclos .
2. *G* no tiene ciclos y, si se añade alguna arista se forma un ciclo.
3. *G* es conexo y si se le quita alguna arista deja de ser conexo.
4. *G* es conexo y el grafo completo de 3 vértices $K_3$ no es un menor de *G*.
5. Dos vértices cualquiera de *G* están conectados por un único camino simple.

Las condiciones anteriores son todas equivalentes, es decir, si se cumple una de ellas otras también se cumplen. Para árboles finitos además se cumple que: Si un árbol *G* tiene un número finito de vertices, *n*, entonces tiene $n - 1$ aristas.

Algunas definiciones relacionadas con los árboles son:

- Un grafo unidireccional simple *G* es un *bosque* si no tiene ciclos simples.
- Un *árbol dirigido* es un grafo dirigido que sería un árbol si no se consideraran las direcciones de las aristas. Algunos autores restringen la frase al caso en el que todos las aristas se dirigen a un vértice particular, o todas sus direcciones parten de un vértice particular.
- Un *árbol* recibe el nombre de *árbol con raíz* si un vértice ha sido designado *raíz*. En este caso las aristas tienen una orientación natural *hacia* o *desde* la raíz. Los árboles con raíz, a menudo con estructuras adicionales como orden de los vecinos de cada vértice, son una estructura clave en informática; véase árbol (programación).
- Un *árbol etiquetado* es un árbol en el que cada vértice tiene una única etiqueta. Los vértices de un árbol etiquetado de *n* vértices reciben normalmente las etiquetas {1,2, ..., n}.
- Un *árbol regular* u *homogéneo* es un árbol en el que cada vértice tiene el mismo grado.
- Todo *árbol* posee una altura. Recorriendo el mismo en forma de grafo dirigido y considerando que las áristas parten desde los vértices hacia algún otro vértice o hacia alguna hoja, de forma tal que todo camino inicia en la raíz y termina en una hoja, puede afirmarse que el árbol posee una altura *h*. Dicha altura será igual a la longitud del camino con más aristas.

### 1.1.2 Clasificación

Un árbol es llamado k-ario si cada nodo tiene, como máximo, k hijos. Un caso particularmente importante es el de un árbol 2-ario, al cual se denomina árbol binario.

Si todos los nodos del árbol exceptuando las hojas, poseen exactamente k hijos, ese árbol a demás de ser k-ario es completo.

Otro caso particular es el del árbol estrella, el cual consiste de un único nodo, que es la raiz. El resto de sus vértices son hojas. Todo arbol estrella de k vertices tiene un único nodo con k-1 hijos, por lo tanto, todo árbol estrella de k vertices es (k-1)-ario.

### 1.1.3 Propiedades

Todo árbol es a su vez un grafo con sólo un conjunto numerable de vértices es además un grafo plano.

Todo grafo conexo *G* admite un árbol de expansión, que es un árbol que contiene cada vértice de *G* y cuyas aristas son aristas de *G*.

Todo árbol k-ario completo de altura *h* tiene $k^h$ hojas.

Dado *n* vértices etiquetados, hay $n^{n-2}$ maneras diferentes de conectarlos para construir un grafo. El resultado se llama fórmula de Cayley. El número de árboles con *n* vértices de grado $d_1$,$d_2$,...,$d_n$ es:

$$\binom{n-2}{d_1-1,d_2-1,\ldots,d_n-1},$$

que es un coeficiente multinomial.

Contar el número de árboles no etiquetados es un problema complicado. De hecho, no se conoce ninguna fórmula para el número de árboles *t*(*n*) con *n* vértices (debe entederse aquí el número de árboles diferentes salvo isomorfismo de grafos). Los primeros valores de *t*(*n*) son 1, 1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159, ... (sucesión A000055 en OEIS). Otter (1948) probó que

$$t(n) \sim C\alpha^n n^{-5/2} \quad \text{as} n \to \infty,$$

Una fórmula más exacta para el comportamiento asintótico de *t*(*n*) implica que hay dos números α y β (α ≈ 3 y β ≈ 0.5) tales que:

$$\lim_{n\to\infty} \frac{t(n)}{\beta\alpha^n n^{-5/2}} = 1.$$

### 1.1.4 Véase también

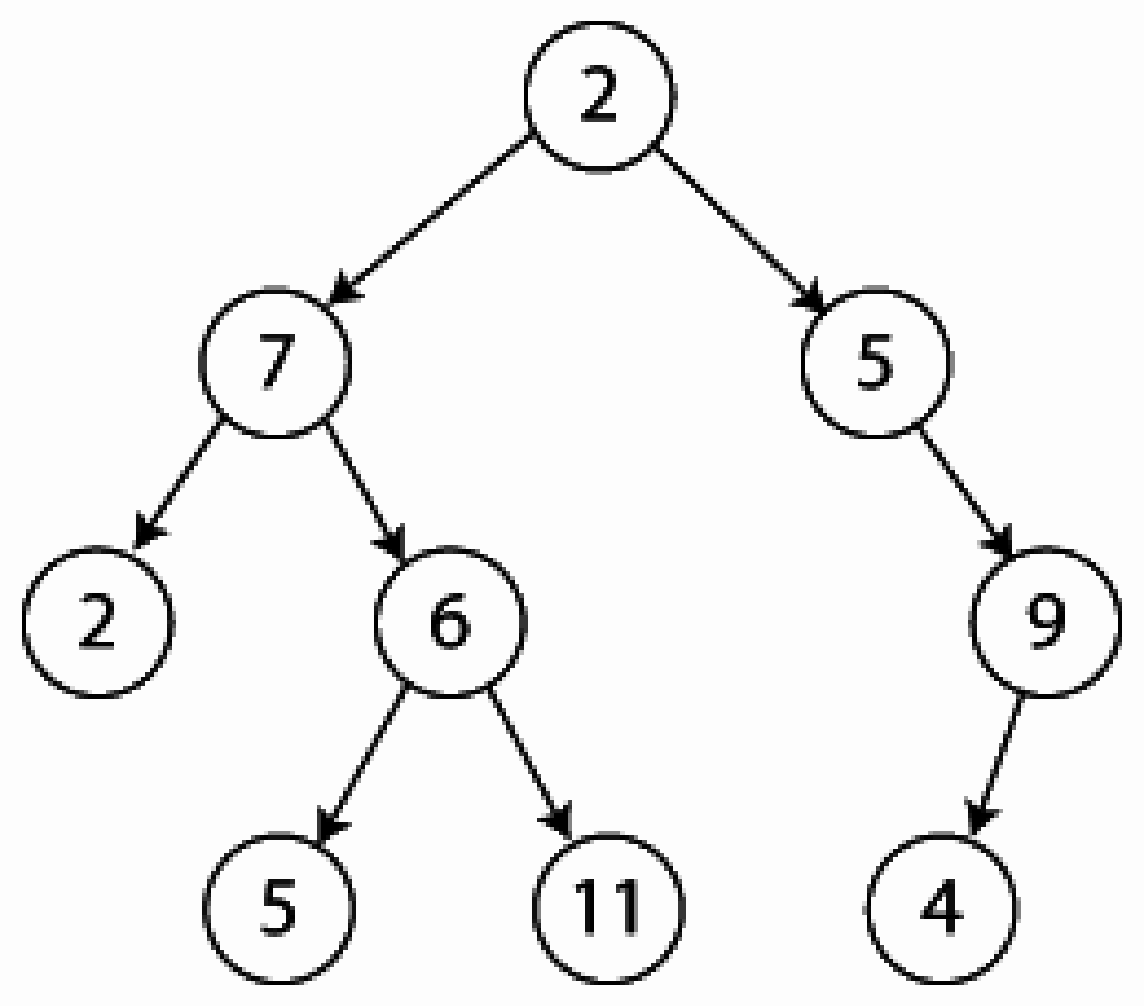
- Árbol (programación)

### 1.1.5 Enlaces externos

- Sobre los grafos VPT y los grafos EPT. Mazzoleni, María Pía. 30 de mayo de 2014.

## 1.2 Árbol binario

En ciencias de la computación, un **árbol binario** es una estructura de datos en la cual cada nodo puede tener un hijo izquierdo y un hijo derecho. No pueden tener más de dos hijos (de ahí el nombre "binario"). Si algún hijo tiene como referencia a **null**, es decir que no almacena ningún dato, entonces este es llamado un nodo externo. En el caso contrario el hijo es llamado un nodo interno. Usos comunes de los árboles binarios son los árboles binarios de búsqueda, los montículos binarios y Codificación de Huffman.




*Un árbol binario sencillo de tamaño 9, 4 niveles y altura 3 (altura = máximo nivel - 1), con un nodo raíz cuyo valor es 2.*

### 1.2.1 Definición de teoría de grafos

En teoría de grafos, se usa la siguiente definición: «Un árbol binario es un grafo conexo, acíclico y no dirigido tal que el grado de cada vértice no es mayor a 3». De esta forma solo existe un camino entre un par de nodos.

Un árbol binario con enraizado es como un grafo que tiene uno de sus vértices, llamado *raíz*, de grado no mayor a 2. Con la raíz escogida, cada vértice tendrá un único padre, y nunca más de dos hijos. Si rehusamos el requerimiento de la conectividad, permitiendo múltiples componentes conectados en el grafo, llamaremos a esta última estructura un *bosque'*.

### 1.2.2 Tipos de árboles binarios

Un **árbol binario** es un árbol en el que ningún nodo puede tener más de dos subárboles. En un árbol binario cada nodo puede tener cero, uno o dos hijos (subárboles). Se conoce el nodo de la izquierda como hijo izquierdo y el nodo de la derecha como hijo derecho.

### 1.2.3 Implementación en C

Un árbol binario puede declararse de varias maneras. Algunas de ellas son:

Estructura con manejo de memoria dinámica, siendo el puntero que apunta al árbol de tipo tArbol:

typedef struct nodo { int clave; struct nodo *izdo, *dcho; }Nodo;

Estructura con arreglo indexado:

typedef struct tArbol { int clave; tArbol hIzquierdo, hDerecho; } tArbol; tArbol árbol[NUMERO_DE_NODOS];

En el caso de un árbol binario casi-completo (o un árbol completo), puede utilizarse un sencillo arreglo de enteros con tantas posiciones como nodos deba tener el árbol. La información de la ubicación del nodo en el árbol es implícita a cada posición del arreglo. Así, si un nodo está en la posición $i$, sus hijos se encuentran en las posiciones $2i$+1 y $2i$+2, mientras que su padre (si tiene), se encuentra en la posición truncamiento($(i-1)/2$) (suponiendo que la raíz está en la posición cero). Este método se beneficia de un almacenamiento más compacto y una mejor localidad de referencia, particularmente durante un recorrido en preorden. La estructura para este caso sería por tanto:

int árbol[NUMERO_DE_NODOS];

### 1.2.4 Recorridos sobre árboles binarios

#### Recorridos en profundidad

El método de este recorrido es tratar de encontrar de la cabecera a la raíz en nodo de unidad binaria. Ahora pasamos a ver la implementación de los distintos recorridos:

**Recorrido en preorden** En este tipo de recorrido se realiza cierta acción (quizás simplemente imprimir por pantalla el valor de la clave de ese nodo) sobre el nodo actual y posteriormente se trata el subárbol izquierdo y cuando se haya concluido, el subárbol derecho. Otra forma para entender el recorrido con este método seria seguir el orden: nodo raíz, nodo izquierda, nodo derecha.

En el árbol de la figura el recorrido en preorden sería: 2, 7, 2, 6, 5, 11, 5, 9 y 4.

void preorden(tArbol *a) { if (a != NULL) { tratar(a); //Realiza una operación en nodo preorden(a->hIzquierdo); preorden(a->hDerecho); } }

Implementación en pseudocódigo de forma iterativa:

push(s,NULL); //insertamos en una pila (stack) el valor NULL, para asegurarnos de que esté vacía push(s,raíz); //insertamos el nodo raíz MIENTRAS (s <> NULL) HACER p = pop(s); //sacamos un elemento de la pila tratar(p); //realizamos operaciones sobre el nodo p SI (D(p) <> NULL) //preguntamos si p tiene árbol derecho ENTONCES push(s,D(p)); FIN-SI SI (I(p) <> NULL) //preguntamos si p tiene árbol izquierdo ENTONCES push(s,I(p)); FIN-SI FIN-MIENTRAS

**Recorrido en postorden** En este caso se trata primero el subárbol izquierdo, después el derecho y por último el nodo actual. Otra forma para entender el recorrido con este método seria seguir el orden: nodo izquierda, nodo derecha, nodo raíz. En el árbol de la figura el recorrido en postorden sería: 2, 5, 11, 6, 7, 4, 9, 5 y 2.

void postorden(tArbol *a) { if (a != NULL) { postorden(a->hIzquiedo); postorden(a->hDerecho); tratar(a); //Realiza una operación en nodo } }

**Recorrido en inorden** En este caso se trata primero el subárbol izquierdo, después el nodo actual y por último el subárbol derecho. En un ABB este recorrido daría los valores de clave ordenados de menor a mayor. Otra forma para entender el recorrido con este método seria seguir el orden: nodo izquierda, nodo raíz, nodo derecha. En el árbol de la figura el recorrido en inorden sería: 2, 7, 5, 6, 11, 2, 5, 4, 9.

Esquema de implementación:

void inorden(tArbol *a) { if (a != NULL) { inorden(a->hIzquierdo); tratar(a); //Realiza una operación en nodo inorden(a->hDerecho); } }

#### Recorridos en amplitud (o por niveles)

En este caso el recorrido se realiza en orden por los distintos niveles del árbol. Así, se comenzaría tratando el nivel 1, que solo contiene el nodo raíz, seguidamente el nivel 2, el 3 y así sucesivamente. En el árbol de la figura el recorrido en amplitud sería: 2, 7, 5, 2, 6, 9, 5, 11 y 4.

Al contrario que en los métodos de recorrido en profundidad, el recorrido por niveles no es de naturaleza recursiva. Por ello, se debe utilizar una cola para recordar los subárboles izquierdos y derecho de cada nodo.

El esquema algoritmo para implementar un recorrido por niveles es exactamente el mismo que el utilizado en la versión iterativa del recorrido en preorden pero cambiando la estructura de datos que almacena los nodos por una cola.

Implementación en C:

void arbol_recorrido_anch (tipo_Arbol* A) { tipo_Cola cola_nodos; // esta cola esta implementada previamente, almacena punteros (posiciones de nodos de árbol) tipo_Pos nodo_actual; // este es un puntero llevara el recorrido if (vacio(A)) // si el árbol esta vacio, salimos return; cola_inicializa(&cola_nodos); // obvio, y necesario cola_enqueue(A, &cola_nodos); // se encola la raíz while (!vacia(&cola_nodos)) { // mientras la cola no se vacie se realizara el recorrido nodo_actual = cola_dequeue(&cola_nodos) // de la cola saldran los nodos ordenados por nivel printf("%c,", nodo_actual->info); // se “procesa” el nodo donde va el recorrido, en este caso se imprime if (nodo_actual->izq != null) // si existe, ponemos el hijo izquierdo en la cola cola_enqueue(nodo_actual->izq, &cola_nodos); if (nodo_actual->der != null) // si existe, ponemos el hijo derecho en la cola cola_enqueue(nodo_actual->der, &cola_nodos); } // al vaciarse la cola se han visitado todos los nodos del árbol }

**Creación de árboles a partir de los recorridos**

Para poder dibujar un árbol binario en base a los recorridos, se necesitan por lo menos dos de los recorridos de profundidad (en caso de que no se repitan los nodos, ya que si se repiten los nodos es recomendable tener los tres recorridos), ya sean inorden y preorden o inorden y postorden, la única diferencia entre usar el recorrido en preorden o postorden es que en preorden se usa el primer nodo para encontrar la raíz y en postorden se usa el último nodo.

El método consiste en ir dividiendo los recorridos del árbol en pequeños subárboles, se va encontrando la raíz con el preorden o postorden y se divide en dos subárboles basándonos en el recorrido en inorden. En el caso de que los nodos se repitan es conveniente tener los 3 recorridos para identificar más fácilmente cuál de los nodos es la raíz actual.

Para el árbol de la figura corresponden los siguientes recorridos:

Preorden $2, 7, 2, 6, 5, 11, 5, 9, 4$

Inorden $2, 7, 5, 6, 11, 2, 5, 4, 9$

Postorden $2, 5, 11, 6, 7, 4, 9, 5, 2$

Para encontrar la raíz es necesario tener el recorrido preorden o postorden, ya que la raíz es el primer nodo o el último nodo respectivamente. En este caso la raíz es el $2$ .

Una vez encontrada la raíz, es necesario saber su posición en el recorrido inorden, del paso anterior se tiene el nodo $2$ , pero existen $2$ nodos con ese valor, el primero y el de en medio. Si el primer dos es la raíz, entonces no existe ninguna rama del lado izquierdo, en ese caso la siguiente raíz de acuerdo con el recorrido en postorden es $5$ y de acuerdo con preorden es $7$ , lo cual es una incongruencia, de esa forma sabemos que el otro $2$ es la raíz.

Entonces marcamos la raíz en el recorrido inorden:

Preorden $\underline{2}, 7, 2, 6, 5, 11, 5, 9, 4$

Inorden $2, 7, 5, 6, 11, \underline{2}, 5, 4, 9$

Postorden $2, 5, 11, 6, 7, 4, 9, 5, \underline{2}$

El recorrido inorden, es un recorrido de los árboles binarios en los que se empieza desde el nodo que se encuentra más a la izquierda de todos, sigue con la raíz y termina con los nodos del lado derecho, entonces, como en el recorrido inorden ya encontramos la raíz, la parte izquierda representa el subárbol izquierdo y la parte derecha representa el subárbol derecho.

En los recorridos tenemos 5 nodos a la izquierda del $2$ y a la derecha se encuentran 3 valores, entonces podemos crear los recorridos para el subárbol izquierdo y el subárbol derecho

Se sigue repitiendo el proceso hasta encontrar todos los nodos del árbol, en este punto la siguiente raíz izquierda es el $7$ y la raíz derecha el $5$ .

Cuando se llegan a nodos en los que únicamente cuentan con una rama es necesario saber que rama es la derecha y cuál es la izquierda (para algunos árboles con balanceo como los AVL), por ejemplo siguiendo la rama de la derecha

partiendo de que el $5$ es la raíz el recorrido inorden es $5, 4, 9$ entonces el siguiente nodo va a la derecha, no hay nodo a la izquierda, después, los recorridos para el subárbol son:

Preorden $9, 4$

Inorden $4, 9$

Postorden $4, 9$

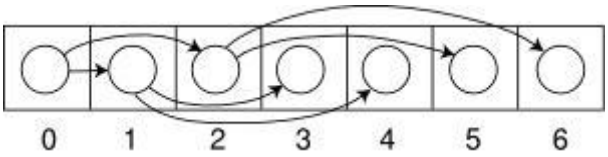
Finalmente el siguiente nodo se coloca a la izquierda del $9$ .

Este método es $100\%$ efectivo cuando no existen nodos repetidos, cuando los nodos se repiten la complejidad aumenta para poder descubrir cuál es el nodo raíz en el recorrido inorden.

### 1.2.5 Métodos para almacenar Árboles Binarios

Los árboles binarios pueden ser construidos a partir de lenguajes de programación de varias formas. En un lenguaje con registros y referencias, los árboles binarios son construidos típicamente con una estructura de nodos y punteros en la cual se almacenan datos, cada uno de estos nodos tiene una referencia o puntero a un nodo izquierdo y a un nodo derecho denominados hijos. En ocasiones, también contiene un puntero a un único nodo. Si un nodo tiene menos de dos hijos, algunos de los punteros de los hijos pueden ser definidos como nulos para indicar que no dispone de dicho nodo. En la figura adjunta se puede observar la estructura de dicha implementación.

Los árboles binarios también pueden ser almacenados como una estructura de datos implícita en vectores, y si el árbol es un árbol binario completo, este método no desaprovecha el espacio en memoria. Tomaremos como notación la siguiente: si un nodo tiene un índice i, sus hijos se encuentran en índices 2i + 1 y 2i + 2, mientras que sus padres (si los tiene) se encuentra en el índice $\left\lfloor \frac{i-1}{2} \right\rfloor$ (partiendo de que la raíz tenga índice cero). Este método tiene como ventajas el tener almacenados los datos de forma más compacta y por tener una forma más rápida y eficiente de localizar los datos en particular durante un preoden transversal. Sin embargo, desperdicia mucho espacio en memoria.




### 1.2.6 Codificación de árboles n-arios como árboles binarios

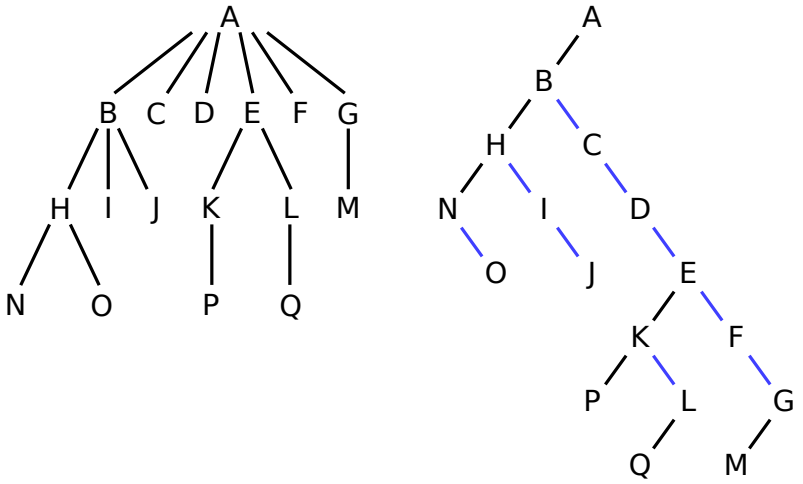
Hay un mapeo uno a uno entre los árboles generales y árboles binarios, el cual en particular es usado en Lisp para representar árboles generales como árboles binarios. Cada nodo N ordenado en el árbol corresponde a un nodo N' en el árbol binario; el hijo de la izquierda de N' es el nodo correspondiente al primer hijo de N, y el hijo derecho de N' es el nodo correspondiente al siguiente hermano de N, es decir, el próximo nodo en orden entre los hijos de los padres de N.

Esta representación como árbol binario de un árbol general, se conoce a veces como un árbol binario primer hijo hermano, o un árbol doblemente encadenado.

Una manera de pensar acerca de esto es que los hijos de cada nodo estén en una lista enlazada, encadenados junto con el campo derecho, y el nodo solo tiene un puntero al comienzo o la cabeza de esta lista, a través de su campo izquierdo.

Por ejemplo, en el árbol de la izquierda, la A tiene 6 hijos (B, C, D, E, F, G). Puede ser convertido en el árbol binario de la derecha.

Un ejemplo de transformar el árbol n-ario a un árbol binario cómo pasar de árboles n-arios a árboles FLOFO.




El árbol binario puede ser pensado como el árbol original inclinado hacia los lados, con los bordes negros izquierdos representando el primer hijo y los azules representado los siguientes hermanos.

Las hojas del árbol de la izquierda serían escritas en Lisp como:

(((N O) I J) C D ((P) (Q)) F (M))

Que se ejecutará en la memoria como el árbol binario de la derecha, sin ningún tipo de letras en aquellos nodos que tienen un hijo izquierdo.

### 1.2.7 Véase también

- Árbol (estructura de datos)
- Árbol multirrama
- Árbol binario de búsqueda
- Árbol de Fibonacci
- Partición de espacio binario

### 1.2.8 Enlaces externos

- Árbol binario de búsqueda en PHP

## 1.3 Árbol binario de búsqueda

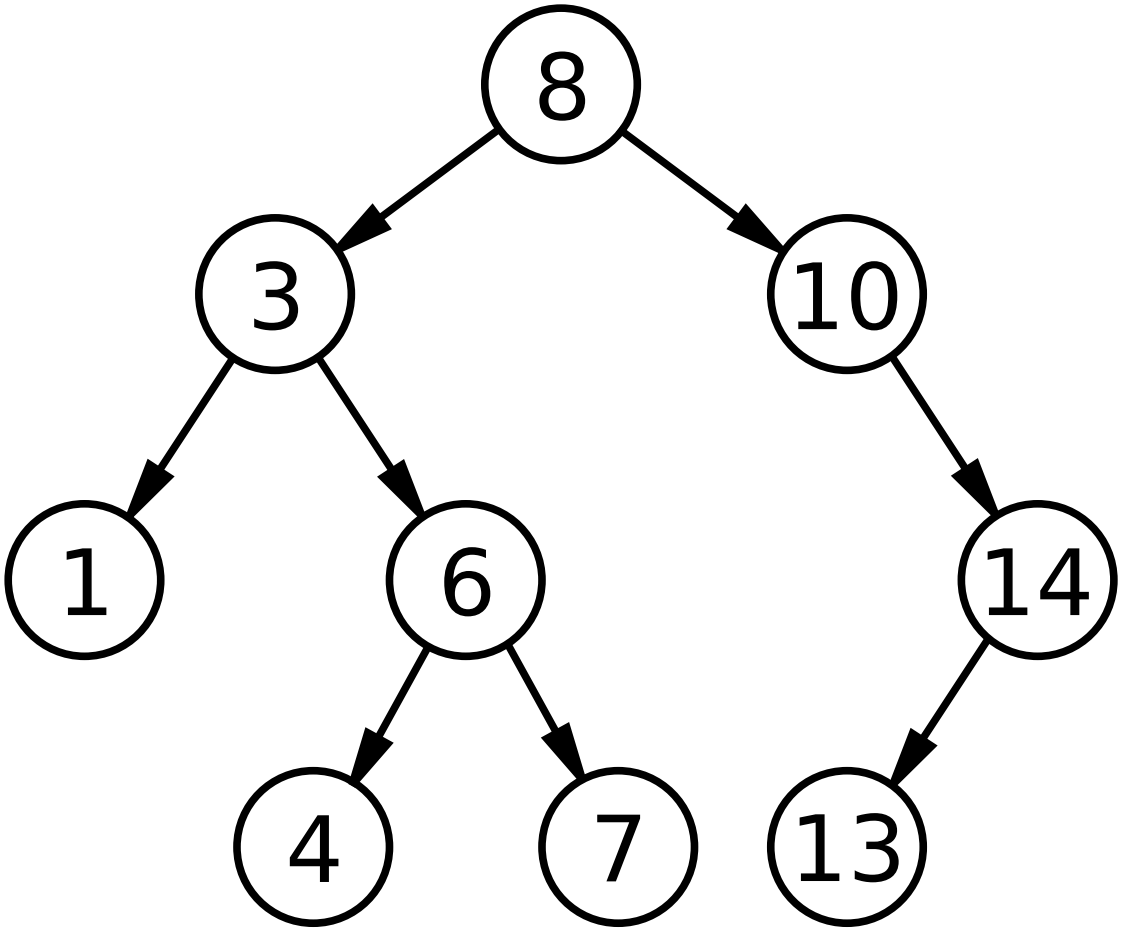
Un **árbol binario de búsqueda** también llamados BST (acrónimo del inglés **B**inary **S**earch **T**ree) es un tipo particular de árbol binario que presenta una estructura de datos en forma de árbol usada en informática.

### 1.3.1 Descripción

Un árbol binario de búsqueda (ABB) es un árbol binario definido de la siguiente forma:

Para una fácil comprensión queda resumido en que es un árbol binario que cumple que el subárbol izquierdo de cualquier nodo (si no está vacío) contiene valores menores que el que contiene dicho nodo, y el subárbol derecho (si no está vacío) contiene valores mayores.

Para estas definiciones se considera que hay una relación de orden establecida entre los elementos de los nodos. Que cierta relación esté definida, o no, depende de cada lenguaje de programación. De aquí se deduce que puede haber distintos árboles binarios de búsqueda para un mismo conjunto de elementos.




*Un árbol binario de búsqueda de tamaño 9 y profundidad 3, con raíz 8 y hojas 1, 4, 7 y 13*

La altura h en el peor de los casos es siempre el mismo tamaño que el número de elementos disponibles. Y en el mejor de los casos viene dada por la expresión $h = ceil(log_2(c+1))$ , donde ceil indica redondeo por exceso.

El interés de los árboles binarios de búsqueda (ABB) radica en que su recorrido en inorden proporciona los elementos ordenados de forma ascendente y en que la búsqueda de algún elemento suele ser muy eficiente.

Dependiendo de las necesidades del usuario que trate con una estructura de este tipo, se podrá permitir la igualdad estricta en alguno, en ninguno o en ambos de los subárboles que penden de la raíz. Permitir el uso de la igualdad provoca la aparición de valores dobles y hace la búsqueda más compleja.

Un árbol binario de búsqueda no deja de ser un caso particular de árbol binario, así usando la siguiente especificación de árbol binario en maude:

**fmod** ARBOL-BINARIO {X :: TRIV}**is sorts** ArbolBinNV{X} ArbolBin{X} . **subsort** ArbolBinNV{X} < ArbolBin{X} . ***** generadores op** crear : -> ArbolBin{X} [**ctor**] . **op** arbolBin : X$Elt ArbolBin{X} ArbolBin{X} -> ArbolBinNV{X} [**ctor**] . **endfm**

podemos hacer la siguiente definición para un árbol binario de búsqueda (también en maude):

**fmod** ARBOL-BINARIO-BUSQUEDA {X :: ORDEN} **is protecting** ARBOL-BINARIO{VOrden}{X} . **sorts** ABB{X} ABBNV{X} . **subsort** ABBNV{X} < ABB{X} . **subsort** ABB{X} < ArbolBin{VOrden}{X} . **subsort** ABBNV{X} < ArbolBinNV{VOrden}{X} . ***** generadores op** crear : -> ArbolBin{X} [**ctor**] . **op** arbolBin : X$Elt ArbolBin{X} ArbolBin{X} -> ArbolBinNV{X} [**ctor**] . **endfm**

con la siguiente teoría de orden:

**fth** ORDEN **is protecting** BOOL . **sort** Elt . ***** operaciones op** _<_ : Elt Elt -> Bool . **endfth**

para que un árbol binario pertenezca al tipo árbol binario de búsqueda debe cumplir la condición de ordenación siguiente que iría junto al módulo ARBOL-BINARIO-BUSQUEDA:

**var** R : X$Elt . **vars** INV DNV : ABBNV{X} . **vars** I D : ABB{X} . **mb** crear : ABB{X} . **mb** arbolBin(R, crear, crear) : ABBNV{X} . **cmb** arbolBin(R, INV, crear) : ABBNV{X} **if** R > max(INV) . **cmb** arbolBin(R, crear, DNV) : ABBNV{X} **if** R < min(DNV) . **cmb** arbolBin(R, INV, DNV) : ABBNV{X} **if** (R > max(INV)) and (R < min(DNV)) . **ops** min max : ABBNV{X} -> X$Elt . **eq** min(arbolBin(R, crear, D)) = R . **eq** min(arbolBin(R, INV, D)) = min(INV) . **eq** max(arbolBin(R, I, crear)) = R . **eq** max(arbolBin(R, I, DNV)) = max(DNV) .

### 1.3.2 Implementación en Python

class nodo: izq , der, dato = None, None, 0 def __init__(self, dato): # crea un nodo self.izq = None self.der = None self.dato = dato
class arbolBinario: def __init__(self): # inicializa la raiz self.raiz = None def agregarNodo(self, dato): # crea un nuevo nodo y lo devuelve return nodo(dato) def insertar(self, raiz, dato): # inserta un dato nuevo en el árbol if raiz == None: # si no hay nodos en el árbol lo agrega return self.agregarNodo(dato) else: # si hay nodos en el árbol lo recorre if dato <= raiz.dato: # si el dato ingresado es menor que el dato guardado va al subárbol izquierdo raiz.izq = self.insertar(raiz.izq, dato) else: # si no, procesa el subárbol derecho raiz.der = self.insertar(raiz.der, dato) return raiz

### 1.3.3 Operaciones

Todas las operaciones realizadas sobre árboles binarios de búsqueda están basadas en la comparación de los elementos o clave de los mismos, por lo que es necesaria una subrutina, que puede estar predefinida en el lenguaje de programación, que los compare y pueda establecer una relación de orden entre ellos, es decir, que dados dos elementos sea capaz de reconocer cual es mayor y cual menor. Se habla de clave de un elemento porque en la mayoría de los casos el contenido de los nodos será otro tipo de estructura y es necesario que la comparación se haga sobre algún campo al que se denomina clave.

#### Búsqueda

La búsqueda Silaina consiste en acceder a la raíz del árbol, si el elemento a localizar coincide con éste la búsqueda ha concluido con éxito, si el elemento es menor se busca en el subárbol izquierdo y si es mayor en el derecho. Si se alcanza un nodo hoja y el elemento no ha sido encontrado se supone que no existe en el árbol. Cabe destacar que la búsqueda en este tipo de árboles es muy eficiente, representa una función logarítmica. El máximo número de comparaciones que necesitaríamos para saber si un elemento se encuentra en un árbol binario de búsqueda estaría entre $[\log_2(N+1)]$ y N, siendo N el número de nodos. La búsqueda de un elemento en un ABB (Árbol Binario de Búsqueda) se puede realizar de dos formas, iterativa o recursiva.

Ejemplo de versión iterativa en el lenguaje de programación C, suponiendo que estamos buscando una clave alojada en un nodo donde está el correspondiente “dato” que precisamos encontrar:

data Buscar_ABB(abb t,clave k) { abb p; dato e; e=NULL; p=t; if (!estaVacio(p)) { while (!estaVacio(p) && (p->k!=k) ) { if (k < p->k) { p=p->l; } if (p->k < k) { p=p->r; } } if (!estaVacio(p) &&(p->d!=NULL) ) { e=copiaDato(p->d); } } return e; }

Véase ahora la versión recursiva en ese mismo lenguaje:

int buscar(tArbol *a, int elem) { if (a == NULL) { return 0; } else if (a->clave < elem) { return buscar(a->hDerecho, elem); } else if (a->clave > elem) { return buscar(a->hIzquierdo, elem); } else { return 1; } }

Otro ejemplo en Python:

def buscar(raiz, clave): # busca el valor clave dentro del arbol if raiz == None: print 'No se encuentra' else: # if clave == raiz.dato: print 'El valor ',clave,' se encuentra en el ABB' elif clave < raiz.dato: # lado izquierdo return buscar(raiz.izq, clave) else: # lado derecho return buscar(raiz.der, clave)

En Pascal:

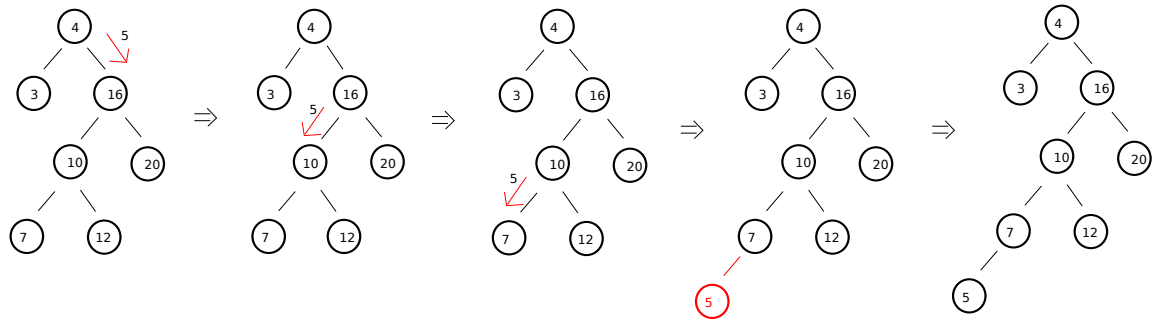
Function busqueda(T:ABR, y: integer):ABR begin if (T=nil) or (^T.raiz=y) then busqueda:=T; else if (^T.raiz<y) then busqueda:=busqueda(^T.dch,y); else busqueda:=busqueda(^T.izq,y); end;

Una especificación en maude para la operación de búsqueda quedaría de la siguiente forma:

**op** esta? : X$Elt ABB{X} -> Bool . **var** R R1 R2 : X$Elt . **vars** I D : ABB{X} . **eq** esta?(R, crear) = **false** . **eq** esta?(R1, arbolBin(R2, I, D)) = **if** R1 == R2 **then** **true else** **if** R1 < R2 **then** esta?(R1, I) **else** esta?(R1, D) **fi fi** .

### Inserción

La inserción es similar a la búsqueda y se puede dar una solución tanto iterativa como recursiva. Si tenemos inicialmente como parámetro un árbol vacío se crea un nuevo nodo como único contenido el elemento a insertar. Si no lo está, se comprueba si el elemento dado es menor que la raíz del árbol inicial con lo que se inserta en el subárbol izquierdo y si es mayor se inserta en el subárbol derecho.




*Evolución de la inserción del elemento "5" en un ABB.*

Como en el caso de la búsqueda puede haber varias variantes a la hora de implementar la inserción en el TAD (Tipo Abstracto de Datos), y es la decisión a tomar cuando el elemento (o clave del elemento) a insertar ya se encuentra en el árbol, puede que éste sea modificado o que sea ignorada la inserción. Es obvio que esta operación modifica el ABB perdiendo la versión anterior del mismo.

A continuación se muestran las dos versiones del algoritmo en pseudolenguaje, iterativa y recursiva, respectivamente.

PROC InsertarABB(árbol:TABB; dato:TElemento) VARIABLES nuevonodo,pav,pret:TABB clavenueva:Tclave ele: TElemento INICIO nuevonodo <- NUEVO(TNodoABB) nuevonodo^.izq <- NULO nuevonodo^.der <- NULO nuevonodo^.elem <- dato SI ABBVacío (árbol) ENTONCES árbol <- nuevonodo ENOTROCASO clavenueva <- dato.clave pav <- árbol // Puntero Avanzado pret <- NULO // Puntero Retrasado MIENTRAS (pav <- NULO) HACER pret <- pav ele = pav^.elem SI (clavenueva < ele.clave ) ENTONCES pav <- pav^.izq EN OTRO CASO pav <- pav^.dch FINSI FINMIENTRAS ele = pret^.elem SI (clavenueva < ele.clave ) ENTONCES pret^.izq <- nuevonodo EN OTRO CASO pret^.dch <- nuevonodo FINSI FINSI FIN PROC InsertarABB(árbol:TABB; dato:TElemento) VARIABLES ele:TElemento INICIO SI (ABBVacío(árbol)) ENTONCES árbol <- NUEVO(TNodoABB) árbol^.izq <- NULO árbol^.der <- NULO árbol^.elem <- dato EN OTRO CASO ele = InfoABB(árbol) SI (dato.clave < ele.clave) ENTONCES InsertarABB(árbol^.izq, dato) EN OTRO CASO InsertarABB(árbol^.dch, dato) FINSI FINSI FIN

Se ha podido apreciar la simplicidad que ofrece la versión recursiva, este algoritmo es la traducción en C. El árbol es pasado por referencia para que los nuevos enlaces a los subárboles mantengan la coherencia.

void insertar(tArbol **a, int elem) { if (*a == NULL) { *a = (tArbol *) malloc(sizeof(tArbol)); (*a)->clave = elem; (*a)->hIzquierdo = NULL; (*a)->hDerecho = NULL; } else if ((*a)->clave < elem) insertar(&(*a)->hDerecho, elem); else if ((*a)->clave > elem) insertar(&(*a)->hIzquierdo, elem); }

En Python el mecanismo de inserción se define, por ejemplo, dentro de la clase que defina el ABB (ver más arriba).

Otro ejemplo en Pascal:

Procedure Insercion(var T:ABR, y:integer) var ultimo:ABR; actual:ABR; nuevo:ABR; begin ultimo:=nil; actual:=T; while (actual<>nil) do begin ultimo:=actual; if (actual^.raiz<y) then actual:=actual^.dch else actual:=actual^.izq; end; new(nuevo); ^nuevo.raiz:=y; ^nuevo.izq:=nil; ^nuevo.dch:=nil; if ultimo=nil then T:=nuevo else if ultimo^.raiz<y then ultimo^.dch:=nuevo else ultimo^.izq:=nuevo; end;

Véase también un ejemplo de algoritmo recursivo de inserción en un ABB en el lenguaje de programación Maude:

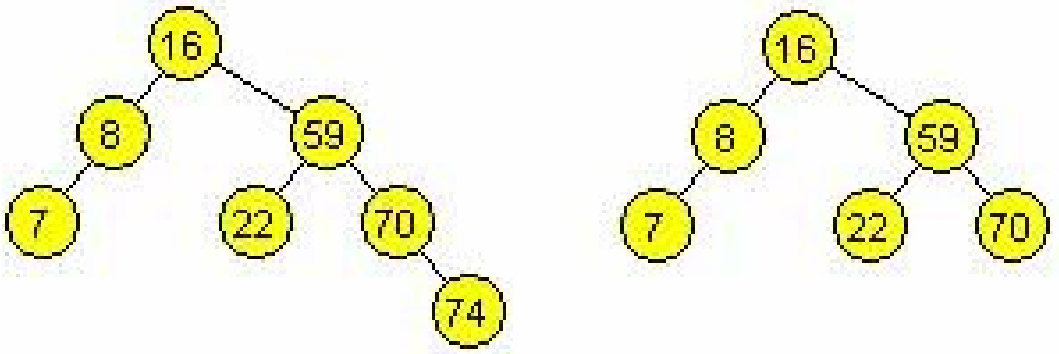
**op** insertar : X$Elt ABB{X} -> ABBNV{X} . **var** R R1 R2 : X$Elt . **vars** I D : ABB{X} . **eq** insertar(R, crear) = arbolBin(R, crear, crear) . **eq** insertar(R1, arbolBin(R2, I, D)) = **if** R1 < R2 **then** arbolBin(R2, insertar(R1, I), D) **else** arbolBin(R2, I, insertar(R1, D)) **fi** .

La operación de inserción requiere, en el peor de los casos, un tiempo proporcional a la altura del árbol.

### Borrado

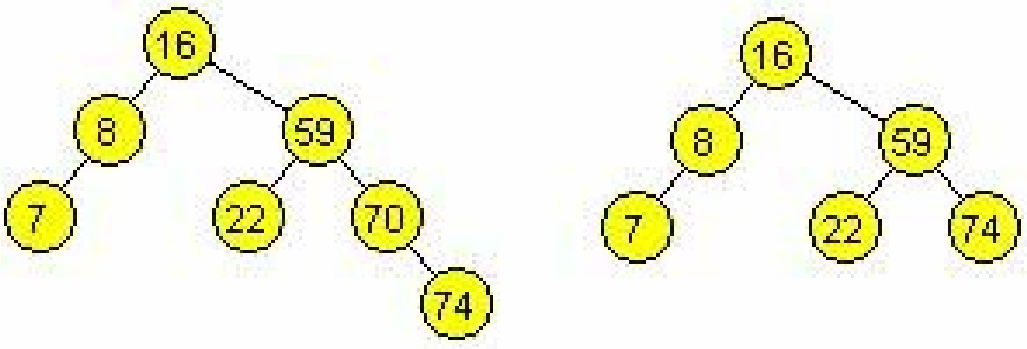
La operación de borrado no es tan sencilla como las de búsqueda e inserción. Existen varios casos a tener en consideración:

- **Borrar un nodo sin hijos o nodo hoja**: simplemente se borra y se establece a nulo el apuntador de su padre.




*Nodo a eliminar 74*

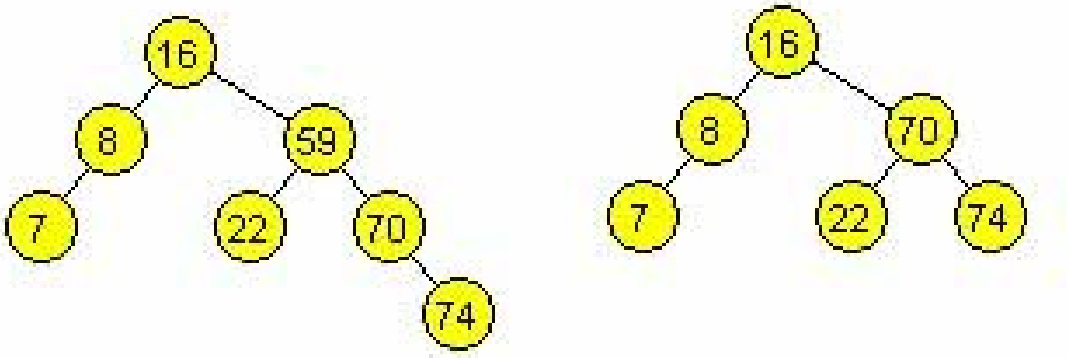
- **Borrar un nodo con un subárbol hijo**: se borra el nodo y se asigna su subárbol hijo como subárbol de su padre.




*Nodo a eliminar 70*

- **Borrar un nodo con dos subárboles hijo**: la solución está en reemplazar el valor del nodo por el de su predecesor o por el de su sucesor en inorden y posteriormente borrar este nodo. Su predecesor en inorden será

el nodo más a la derecha de su subárbol izquierdo (mayor nodo del subarbol izquierdo), y su sucesor el nodo más a la izquierda de su subárbol derecho (menor nodo del subarbol derecho). En la siguiente figura se muestra cómo existe la posibilidad de realizar cualquiera de ambos reemplazos:




*Nodo a eliminar 59*

El siguiente algoritmo en C realiza el borrado en un ABB. El procedimiento *reemplazar* busca la mayor clave del subárbol izquierdo y la asigna al nodo a eliminar.

void reemplazar(tArbol **a, tArbol **aux); /*Prototipo de la funcion "reemplazar"*/ void borrar(tArbol **a, int elem) { tArbol *aux; if (*a == NULL) return; if ((*a)->clave < elem) borrar(&(*a)->hDerecho, elem); else if ((*a)->clave > elem) borrar(&(*a)->hIzquierdo, elem); else if ((*a)->clave == elem) { aux = *a; if ((*a)->hIzquierdo == NULL) *a = (*a)->hDerecho; else if ((*a)->hDerecho == NULL) *a = (*a)->hIzquierdo; else reemplazar(&(*a)->hIzquierdo, &aux); free(aux); } } void reemplazar(tArbol **a, tArbol **aux) { if ((*a)->hDerecho == NULL) { (*aux)->clave = (*a)->clave; *aux = *a; *a = (*a)->hIzquierdo; } else reemplazar(&(*a)->hDerecho, & aux); }

Otro ejemplo en Pascal.

Procedure Borrar(var T:ABR, x:ABR) var aBorrar:ABR; anterior:ABR; actual:ABR; hijo:ABR; begin if (^x.izq=nil) or (^x.dch=nil) then aBorrar:=x; else aBorrar:=sucesor(T,x); actual:=T; anterior:=nil; while (actual<>aBorrar) do begin anterior:=actual; if (^actual.raiz<^aBorrar.raiz) then actual:=^actual.dch; else actual:=^actual.izq; end; if (^actual.izq=nil) then hijo:=^actual.dch; else hijo:=^actual.izq; if (anterior=nil) then T:=hijo; else if (^anterior.raiz<^actual.raiz) then ^anterior.dch:=hijo; else ^anterior.izq:=hijo; if (aBorrar<>x) then ^x.raiz:=^aBorrar.raiz; free(aBorrar); end;

Véase también un ejemplo de algoritmo recursivo de borrado en un ABB en el lenguaje de programación Maude, considerando los generadores crear y arbolBin. Esta especificación hace uso de la componente clave a partir de la cual se ordena el árbol.

**op** eliminar : X$Elt ABB{X} -> ABB{X} . **varS** R M : X$Elt . **vars** I D : ABB{X} . **vars** INV DNV : ABBNV{X} . **ops** max min : ArbolBin{X} -> X$Elt . **eq** min(arbolBin(R, crear, D)) = R . **eq** max(arbolBin(R, I, crear)) = R . **eq** min(arbolBin(R, INV, D)) = min(INV) . **eq** max(arbolBin(R, I, DNV )) = max(DNV) . **eq** eliminar(M, crear) = crear . **ceq** eliminar(M, arbolBin(R, crear, D)) = D **if** M == clave(R) . **ceq** eliminar(M, arbolBin(R, I, crear)) = I **if** M == clave(R) . **ceq** eliminar(M, arbolBin(R, INV, DNV)) = arbolBin(max(INV), eliminar(clave(max(INV)), INV), DNV) **if** M == clave(R) . **ceq** eliminar(M, arbolBin(R, I, D)) = arbolBin(R, eliminar(M, I), D) **if** M < clave(R) . **ceq** eliminar(M, arbolBin(R, I, D)) = arbolBin(R, I, eliminar(M, D)) **if** clave(R) < M .

### Otras Operaciones

Otra operación sería por ejemplo comprobar que un árbol binario es un árbol binario de búsqueda. Su implementación en maude es la siguiente:

**op** esABB? : ABB{X} -> Bool . **var** R : X$Elt . **vars** I D : ABB{X} . **eq** esABB?(crear) = **true** . **eq** esABB?(arbolbBin(R, I, D)) = (Max(I) < R) and (Min(D) > R) and (esABB?(I)) and (esABB?(D)) .

**Recorridos**

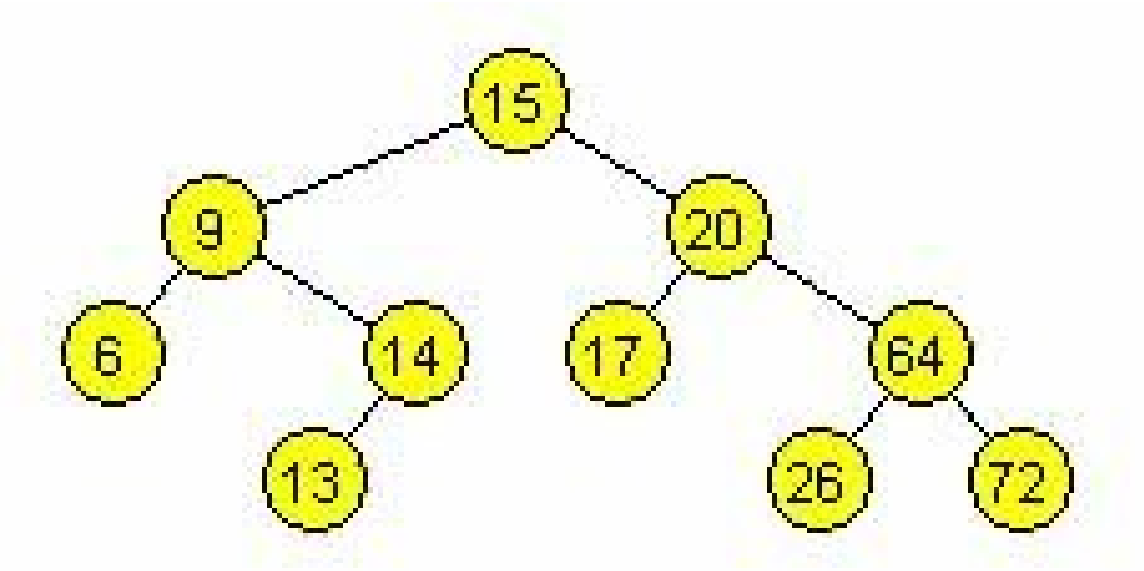
Se puede hacer un recorrido de un árbol en profundidad o en anchura.

Los recorridos en anchura son por niveles, se realiza horizontalmente desde la raíz a todos los hijos antes de pasar a la descendencia de alguno de los hijos.

El coste de recorrer el ABB es O(n), ya que se necesitan visitar todos los vértices.

El recorrido en profundidad lleva al camino desde la raíz hacia el descendiente más lejano del primer hijo y luego continúa con el siguiente hijo. Como recorridos en profundidad tenemos inorden, preorden y postorden.

Una propiedad de los ABB es que al hacer un recorrido en profundidad inorden obtenemos los elementos ordenados de forma ascendente.




*Ejemplo árbol binario de búsqueda*

Resultado de hacer el recorrido en:

**Inorden** = [6, 9, 13, 14, 15, 17, 20, 26, 64, 72].

**Preorden** = [15, 9, 6, 14, 13, 20, 17, 64, 26, 72].

**Postorden** =[6, 13, 14, 9, 17, 26, 72, 64, 20, 15].

Recorridos en Visual Basic .Net

'función de recorrido en PREORDEN Public Function preorden() As String cadenasalida = "" rePreorden(raíz) Return cadenasalida End Function Private Sub rePreorden(ByVal padre As Nodo) If IsNothing(padre) Then Return End If cadenasalida = cadenasalida & "-" & padre.dato rePreorden(padre.ant) rePreorden(padre.sig) End Sub 'función de recorrido en POSTORDEN Public Function postorden() As String cadenasalida = "" reposorden(raíz) Return cadenasalida End Function Private Sub repostorden(ByVal padre As Nodo) If IsNothing(padre) Then Return End If repostorden(padre.ant) repostorden(padre.sig) cadenasalida = cadenasalida & "-" & padre.dato End Sub 'función de recorrido en ENORDEN Public Function inorden() As String cadenasalida = "" reinorden(raíz) Return cadenasalida End Function Private Sub reinorden(ByVal padre As Nodo) If IsNothing(padre) Then Return End If reinorden(padre.ant) cadenasalida = cadenasalida & "-" & padre.dato reinorden(padre.sig) End Sub

Recorridos en C con funciones recursivas

struct Nodo{ char nombre[30]; struct Nodo *izq; struct Nodo *der; }; typedef struct Nodo Nodo; typedef Nodo *Arbol; void preOrden(Arbol abb){ if(abb) { printf("%s\n", abb->nombre); preOrden(abb->izq); preOrden(abb->der); } } void postOrden(Arbol abb){ if(abb) { postOrden(abb->izq); postOrden(abb->der); printf("%s\n", abb->nombre); } } void inOrden(Arbol abb){ if(abb) { inOrden(abb->izq); printf("%s\n", abb->nombre); inOrden(abb->der); } }

### 1.3.4 Tipos de árboles binarios de búsqueda

Hay varios tipos de árboles binarios de búsqueda. Los árboles AVL, árbol rojo-negro, son árboles autobalanceables . Los árbol biselado son árboles también autobalanceables con la propiedad de que los elementos accedidos recientemente se accederá más rápido en posteriores accesos. En el montículo como en todos los árboles binarios de búsqueda cada nodo padre tiene un valor mayor que sus hijos y además es completo, esto es cuando todos los niveles están llenos con excepción del último que puede no estarlo, por último, en lo montículos, cada nodo mantiene una prioridad y siempre, un nodo padre tendrá una prioridad mayor a la de su hijo.

Otras dos maneras de configurar un árbol binario de búsqueda podría ser como un árbol completo o degenerado.

Un árbol completo es un árbol con "n" niveles, donde cada nivel d <= n-1; el número de nodos existentes en el nivel "d" es igual que $2^d$. Esto significa que todos los posibles nodos existen en esos niveles, no hay ningún hueco. Un requirimiento adicional para un árbol binario completo es que para el nivel "n", los nodos deben estar ocupados de izquierda a derecha, no pudiendo haber un hueco a la izquierda de un nodo ocupado.

Un árbol degenerativo es un árbol que, para cada nodo padre, sólo hay asociado un nodo hijo. Por lo que se comporta como una lista enlazada.

### 1.3.5 Comparación de rendimiento

D. A. Heger(2004)[1] realiza una comparación entre los diferentes tipos de árboles binarios de búsqueda para encontrar que tipo nos daría el mejor rendimiento para cada caso. Los montículos se encuentran como el tipo de árbol binario de búsqueda que mejor resultado promedio da, mientras que los árboles rojo-negro los que menor rendimiento medio nos aporta.

### 1.3.6 Buscando el Árbol binario de búsqueda óptimo

Si nosotros no tenemos en mente planificar un árbol binario de búsqueda, y sabemos exactamente como de frecuente serán visitados cada elemento podemos construir un árbol binario de búsqueda óptimo con lo que conseguiremos que la media de gasto generado a la hora de buscar un elemento sea minimizado.

Asumiendo que conocemos los elementos y en qué nivel está cada uno, también conocemos la proporción de futuras búsquedas que se harán para encontrar dicho elemento. Si es así, podemos usar una solución basada en la programación dinámica.

En cambio, a veces sólo tenemos la estimación de los costes de búsqueda, como pasa con los sistemas que nos muestra el tiempo que ha necesitado para realizar una búsqueda. Un ejemplo, si tenemos un ABB de palabras usado en un corrector ortográfico, deberíamos balancear el árbol basado en la frecuencia que tiene una palabra en el Corpus lingüístico, desplazando palabras como "de" cerca de la raíz y palabras como "vesánico" cerca de las hojas. Un árbol como tal podría ser comparado con los árboles Huffman que tratan de encontrar elementos que son accedidos frecuentemente cerca de la raíz para producir una densa información; de todas maneras, los árboles Huffman sólo puede guardar elementos que contienen datos en las hojas y estos elementos no necesitan ser ordenados.

En cambio, si no sabemos la secuencia en la que los elementos del árbol van a ser accedidos, podemos usar árboles biselados que son tan buenos como cualquier árbol de búsqueda que podemos construir para cualquier secuencia en particular de operaciones de búsqueda.

Árboles alfabéticos son árboles Huffman con una restricción de orden adicional, o lo que es lo mismo, árboles de búsqueda con modificación tal que todos los elementos son almacenados en las hojas.

### 1.3.7 Véase también

- Árbol (programación)
- Árbol Binario
- Árbol AVL
- Árbol 2-3
- Árbol B

- Árbol Rojo-Negro
- Árbol Splay
- Árbol Multirrama

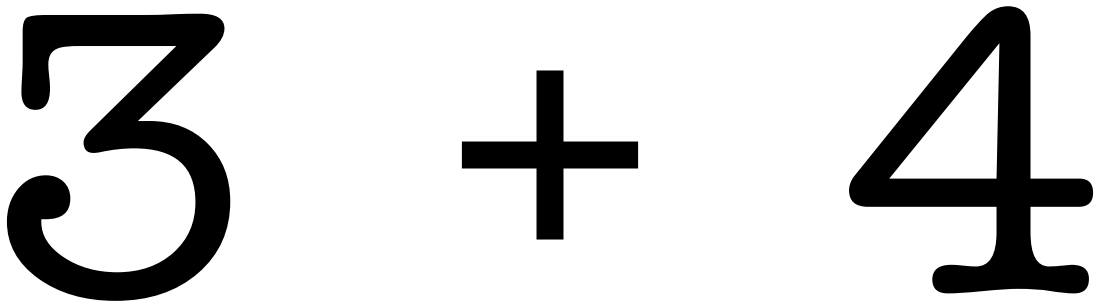
### 1.3.8 Referencias

[1] Heger, Dominique A. (2004), «A Disquisition on The Performance Behavior of Binary Search Tree Data Structures», *European Journal for the Informatics Professional* **5** (5), http://www.cepis.org/upgrade/files/full-2004-V.pdf

### 1.3.9 Enlaces externos

- Árboles binarios de búsqueda en Google
- Implementación de árboles binarios de búsqueda en distintos lenguajes
- Aplicación JAVA de árboles

## 1.4 Notación de infijo



*Notación de infijo.*

La **notación de infijo** es la notación común de fórmulas aritméticas y lógicas, en la cual se escriben los operadores entre los operandos en que están actuando (ej. 2 + 2) usando un estilo de infijo. No es tan simple de analizar (parser) por las computadoras, como la notación de prefijo (ej. + 2 2) o la notación de postfijo (ej. 2 2 +), aunque muchos lenguajes de programación la utilizan debido a su familiaridad.

En la notación de infijo, a diferencia de las notaciones de prefijo o posfijo, es necesario rodear entre paréntesis a los grupos de operandos y operadores, para indicar el orden en el cual deben ser realizadas las operaciones. En la ausencia de paréntesis, ciertas reglas de prioridad determinan el orden de las operaciones.

### 1.4.1 Véase también

- Notación de postfijo, también llamada notación polaca inversa
- Notación de prefijo, también llamada notación polaca
- Algoritmo shunting yard, usado para convertir la notación infija en la notación posfija o en un árbol
- Métodos de entrada en calculadoras, una comparación de notaciones usadas en calculadoras de bolsillo

### 1.4.2 Enlaces externos

- *RPN or DAL? A brief analysis of Reverse Polish Notation against Direct Algebraic Logic*

## 1.5 Grafo completo

En teoría de grafos, un **grafo completo** es un grafo simple donde cada par de vértices está conectado por una arista.

Un grafo completo de *n* vértices tiene $n(n-1)/2$ aristas, y se nota $K_n$ . Es un grafo regular con todos sus vértices de grado $n-1$ . La única forma de hacer que un grafo completo se torne disconexo a través de la eliminación de vértices, sería eliminándolos todos.

El teorema de Kuratowski dice que un grafo planar no puede contener $K_5$ (ó el grafo bipartito completo $K_{3,3}$ ) y todo $K_n$ incluye a $K_{n-1}$ , entonces ningún grafo completo $K_n$ con $n \geq 5$ es planar

### 1.5.1 Ejemplos

Los grafos completos de 1 a 12 nodos son los siguientes:

### 1.5.2 Véase también

- Grafo
- Teoría de grafos

### 1.5.3 Referencias

## 1.6 Notación polaca



*Notación polaca.*

La **notación polaca**, también conocida como **notación de prefijo** o **notación prefija**, es una forma de notación para la lógica, la aritmética, y el álgebra. Su característica distintiva es que coloca los operadores a la izquierda de sus operandos. Si la aridad de los operadores es fija, el resultado es una sintaxis que carece de paréntesis u otros signos de agrupación, y todavía puede ser analizada sin ambigüedad. El lógico polaco Jan Łukasiewicz inventó esta notación alrededor de 1920 para simplificar la lógica proposicional.

Aquí hay una cita de *Axiom and Generalizing Deduction* de Nicod , página 180.

> Vine sobre la idea de una notación libre de paréntesis en 1924. Utilicé esa notación por primera vez en mi artículo Lukasiewicz(1), P. 610, nota al pie de la página.

La referencia de arriba, citada por Jan Lukasiewicz es al parecer un informe litografiado en polaco.

Alonzo Church menciona esta notación en su libro clásico sobre lógica matemática como digna de observación en los sistemas notacionales incluso contrastados con la *Exposición notacional lógica y el trabajo Principia Mathematica* de Whitehead y Russell.[1]

Mientras que no se ha usado más en lógica, la notación polaca ha encontrado un espacio en las ciencias de la computación.

### 1.6.1 Aritmética

La expresión para sumar los números uno y dos, en la notación de prefijo, se escribe "+ 1 2" en vez de "1 + 2". En expresiones más complejas, los operadores todavía preceden sus operandos, pero los operandos pueden ser ellos mismos expresiones no triviales incluyendo sus propios operadores. Por ejemplo, la expresión que sería escrita en la notación de infijo convencional como

(5 - 6) * 7

puede ser escrito en prefijo como

* (- 5 6) 7

o simplemente

* - 5 6 7

Puesto que los simples operadores aritméticos son todos binarios (por lo menos, en contextos aritméticos), cualquier representación prefijo de ellos es inequívoca, y poner signos de agrupamiento a la expresión de prefijo es innecesario. En el ejemplo anterior, los paréntesis en la versión de infijo eran requeridos. Si los movemos:

5 - (6 * 7)

o simplemente los quitamos:

5 - 6 * 7

cambiaría el significado y el resultado de toda la expresión. Sin embargo, la versión correspondiente de prefijo de este segundo cálculo sería escrita como:

- 5 * 6 7

El proceso de la substracción es diferido hasta que ambos operandos de la substracción se hayan leído (es decir, 5 y el producto de 6 y 7). Como con *cualquier* notación, las expresiones más interiores son evaluadas primero, pero en la notación de prefijo este "interioridad" se puede transportar por el orden en vez del agrupamiento.

La notación de prefijo de la aritmética simple es en gran parte de interés académico. Como la similar notación de posfijo o notación polaca inversa, la notación de prefijo ha sido usada en algunas calculadoras comerciales (HP-11C).April 2009[*cita requerida*] Sin embargo, la aritmética de notación de prefijo es usada con frecuencia como primer paso conceptual en la enseñanza de la construcción de un compilador.

### 1.6.2 Programación de computadora

La notación de prefijo ha visto una amplia aplicación con las S-expressions de Lisp, donde son requeridos los paréntesis debido a los operadores aritméticos que tienen aridad variable. El lenguaje de programación Ambi usa la notación polaca para operaciones aritméticas y la construcción del programa. La posfija notación polaca inversa es usada en muchos lenguajes de programación basados en pila como PostScript, y es el principio de operación de ciertas calculadoras, notablemente las de Hewlett-Packard.

Aunque sea obvio, es importante observar que el número de operandos en una expresión debe igualar al número de operadores más uno, de lo contrario la sentencia no tiene ningún sentido (asumiendo que solamente son usados operadores binarios en la expresión). Esto puede ser fácil de pasarlo por alto cuando se trata con expresiones más largas y más complicadas con varios operadores, así que se debe tener cuidado de comprobar con minuciosidad que una expresión tiene sentido al usar la notación de prefijo.

### 1.6.3 Orden de las operaciones

El orden de operaciones es definido dentro de la estructura de la notación de prefijo y puede ser fácilmente determinada. Una cosa a tener presente es que al ejecutar una operación, la operación es aplicada AL primer operando POR el segundo operando. Esto no es un problema con las operaciones que conmutan, pero para las operaciones no conmutativas como la división o la substracción, este hecho es crucial para análisis de una sentencia. Por ejemplo, la sentencia siguiente:

/ 10 5 = 2 (prefijo)

Se lee como "Divide 10 POR 5". Así la solución es 2, no ½ como sería el resultado de un análisis incorrecto de dividir 5 entre 10.

La notación de prefijo es especialmente popular entre las operaciones basadas en pila debido a su capacidad natural de distinguir fácilmente el orden de las operaciones sin la necesidad de paréntesis. Para evaluar el orden de las operaciones bajo la notación de prefijo, incluso no se necesita memorizar una jerarquía operacional, como con la notación de infijo. En lugar de eso, se mira directamente a la notación para descubrir qué operador evaluar primero. Leyendo una expresión de izquierda a derecha, primero se busca un operador y se procede a buscar dos operandos. Si se encuentra otro operador antes de que se encuentren los dos operandos, entonces el operador viejo es colocado a un lado hasta que este nuevo operador sea resuelto. Este proceso se itera hasta que un operador sea resuelto, lo cual debería suceder siempre, puesto que en una sentencia completa debe haber un operando más que la cantidad de operadores. Una vez que esté resuelto, el operador y los dos operandos son reemplazados por un nuevo operando. Puesto que un operador y dos operandos son eliminados y un operando es añadido, hay una pérdida neta de un operador y un operando, lo cual todavía deja una expresión con N operadores y N+1 operandos, permitiendo así que el proceso iterativo continúe. Ésta es la teoría general tras el uso de stacks en lenguajes de programación para evaluar una sentencia en la notación de prefijo, aunque hay varios algoritmos que manipulan el proceso. Una vez que es analizada una sentencia en la notación de prefijo, llega a ser menos intimidante mientras que permite una cierta separación desde la convención con una añadida conveniencia. Un ejemplo muestra la facilidad con la cual una sentencia compleja en la notación de prefijo se puede descifrar a través del orden de las operaciones:

La expresión en notación de infijo ((15 / (7 - (1 + 1))) * 3) - (2 + (1 + 1)) se resuelve de la siguiente manera en notación polaca o de prefijo:

- * / 15 - 7 **+ 1 1** 3 + 2 + 1 1 = - * / 15 - 7 **2** 3 + 2 + 1 1 = - * / 15 **5** 3 + 2 + 1 1 = - * **3** 3 + 2 + 1 1 = - **9** + 2 **+ 1 1** = - 9 + 2 **2** = **- 9 4** = **5**

### 1.6.4 Notación polaca en lógica

La siguiente tabla muestra el núcleo de la notación para la lógica de sentencias de Jan Łukasiewicz. La notación convencional recién fue establecida entre las décadas de 1970 y 1980. Algunas de las letras usadas corresponden a ciertos vocablos polacos:

### 1.6.5 Autómata de pila

La notación polaca es la originada por un Autómata con pila, en la que los operadores siempre preceden a los operandos sobre los que actúan, y que tiene la ventaja de no necesitar paréntesis:

**Estándar**

Ejemplo 1: 2 * ( 3 + 5 ) Ejemplo 2: 2 * 3 + 5

**Polaca**

Ejemplo 1: * 2 + 3 5 Ejemplo 2: + * 2 3 5

### 1.6.6 Referencias

[1] Church, Alonzo (1944). *Introduction to Mathematical Logic*. Princeton, New Jersey: Princeton University Press. - p.38: "Worthy of remark is the parenthesis-free notation of Jan Lukasiewicz. In this the letters N, A, C, E, K are used in the roles of negation, disjunction, implication, equivalence, conjunction respectively. ..."

- Weisstein, Eric W. «ReversePolishNotation». En Weisstein, Eric W. *MathWorld* (en inglés). Wolfram Research.

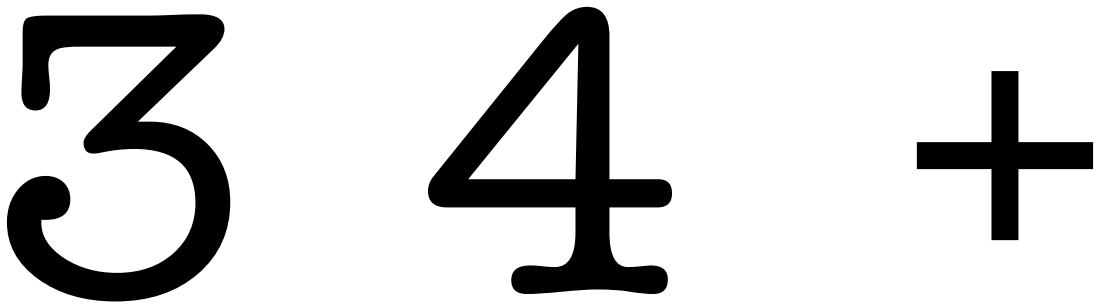
### 1.6.7 Lecturas relacionadas

- Łukasiewicz, Jan (1957). *Aristotle's Syllogistic from the Standpoint of Modern Formal Logic*. Oxford University Press.
- Łukasiewicz, Jan, "Philosophische Bemerkungen zu mehrwertigen Systemen des Aussagenkalküls", *Comptes rendus des séances de la Société des Sciences et des Lettres de Varsovie*, 23:51-77 (1930). Translated by H. Weber as "Philosophical Remarks on Many-Valued Systems of Propositional Logics", in Storrs McCall, *Polish Logic 1920-1939*, Clarendon Press: Oxford (1967).

### 1.6.8 Véase también

- Notación de infijo
- Notación de postfijo, también llamada notación polaca inversa
- Lenguaje de programación LISP
  - Expresión S
- Notación húngara

## 1.7 Notación polaca inversa




*Notación Polaca Inversa (RPN).*

La **notación polaca inversa**, **notación de postfijo**, o **notación posfija**, (en inglés, **Reverse Polish notation**, o **RPN**), es un método algebraico alternativo de introducción de datos. Su nombre viene por analogía con la relacionada notación polaca, una notación de prefijo introducida en 1920 por el matemático polaco Jan Łukasiewicz, en donde cada operador está antes de sus operandos. En la notación polaca inversa es al revés, primero están los operandos y después viene el operador que va a realizar los cálculos sobre ellos. Tanto la notación polaca como la notación polaca inversa no necesitan usar paréntesis para indicar el orden de las operaciones mientras la aridad del operador sea fija.

El esquema polaco inverso fue propuesto en 1954 por Burks, Warren, y Wright[1] y reinventado independientemente por Friedrich L. Bauer y Edsger Dijkstra a principios de los años 1960, para reducir el acceso de la memoria de computadora y para usar el stack para evaluar expresiones. La notación y los algoritmos para este esquema fueron extendidos por el filósofo y científico de la computación australiano Charles Leonard Hamblin a mediados de los años 1960.[2][3] Posteriormente, Hewlett-Packard lo aplicó por primera vez en la calculadora de sobremesa HP-9100A en 1968 y luego en la primera calculadora científica de bolsillo, la HP-35. Durante los años 1970 y los años 1980, el RPN tenía cierto valor incluso entre el público general, pues fue ampliamente usado en las calculadoras de escritorio del tiempo - por ejemplo, las calculadoras de la serie HP-10C.

En ciencias de la computación, la notación de postfijo es frecuentemente usada en lenguajes de programación concatenativos y basados en pila. También es común en sistemas basados en flujo de datos y tuberías, incluyendo las tuberías de Unix.

### 1.7.1 Funcionamiento

Su principio es el de evaluar los datos directamente cuando se introducen y manejarlos dentro de una estructura LIFO *(Last In First Out)*, lo que optimiza los procesos a la hora de programar.

Básicamente la diferencias con el método algebraico o notación de infijo es que, al evaluar los datos directamente al introducirlos, no es necesario ordenar la evaluación de los mismos, y que para ejecutar un comando, primero se deben introducir todos sus argumentos, así, para hacer una suma 'a+b=c' el RPN lo manejaría 'a b +', dejando el resultado 'c' directamente.

Nótese que la notación polaca inversa no es literalmente la imagen especular de la notación polaca: el orden de los operandos es igual en la tres notaciones (infijo, prefijo o polaca, y postfijo o polaca inversa), lo que cambia es que el lugar donde va el operador. En la notación infija, el operador va en el medio de los operandos, mientras que en la notación polaca va antes y en la notación polaca inversa va después. Así pues, “640 / 16” (en notación de infijo), se escribe como "/ 640 16” (en notación polaca) y como “640 16 /" en notación polaca inversa. El orden de los operandos es importante cuando se manejan operadores no conmutativos (como la resta o la división), así, si dividimos 10 entre 2, por ejemplo, en las tres notaciones se debe escribir de la siguiente manera: “10 / 2”, "/ 10 2”, “10 2 /".

### 1.7.2 Ventajas

- Los cálculos se realizan secuencialmente según se van introduciendo operadores, en vez de tener que esperar a escribir la expresión al completo. Debido a esto, se cometen menos errores al procesar cálculos complejos.

- El proceso de apilación permite guardar resultados intermedios para un uso posterior. Esta característica permite que las calculadoras RPN computen expresiones de complejidad muy superior a la que alcanzan las calculadoras algebraicas.

- No requiere paréntesis ni reglas de preferencia, al contrario que la notación algebraica, ya que el proceso de apilamiento permite calcular la expresión por etapas.

- En las calculadoras RPN, el cálculo se realiza sin tener que apretar la tecla "=" (aunque se requiere pulsar la tecla “Enter” para añadir cifras a la pila).

- El estado interno de la calculadora siempre consiste en una pila de cifras sobre las que se puede operar. Dado que no se pueden introducir operadores en la pila, la notación polaca inversa es conceptualmente más sencilla y menos dada a errores que otras notaciones.

- En términos educativos, la notación polaca inversa requiera que el estudiante comprenda la expresión que se está calculando. Copiar una expresión algebraica directamente a una calculadora sin comprender la aritmética dará un resultado erróneo.

### 1.7.3 Desventajas

- La adopción casi universal de la notación algebraica en los sistemas educativos hace que no haya muchas razones prácticas inmediatas para que los alumnos aprendan la notación polaca inversa. No obstante, muchos estudiantes[*¿quién?*] afirman que, una vez aprendida, la notación polaca inversa simplifica en gran manera el cálculo de expresiones complejas.

- Es difícil usar la notación polaca inversa al escribir a mano, dada la importancia de los espacios para separar operandos. Se requiere un caligrafía muy clara para evitar confundir, por ejemplo, 12 34+ (=46) con 123 4+ (=127) o con 1 234+ (=235).

- Las calculadoras RPN son relativamente raras. Forzado a usar una calculadora algebraica, el usuario de una calculadora RPN típicamente comete errores más frecuentemente debido a sus hábitos de uso normales. No obstante, esto no es un problema tan grave en la actualidad, debido a que las calculadoras RPN son fáciles de programar.

### 1.7.4 El algoritmo RPN

El algoritmo que utilizan las calculadoras RPN es relativamente simple:

- Si hay elementos en la bandeja de entrada
  - Leer el primer elemento de la bandeja de entrada.
  - Si el elemento es un operando.
    - Poner el operando en la pila.
  - Si no, el elemento es una función (los operadores, como "+", no son más que funciones que toman dos argumentos).
    - Se sabe que la función x toma n argumentos.
    - Si hay menos de n argumentos en la pila
      - **(Error)** El usuario no ha introducido suficientes argumentos en la expresión.
    - Si no, tomar los últimos n operandos de la pila.
    - Evaluar la función con respecto a los operandos.
    - Introducir el resultado (si lo hubiere) en la pila.
- Si hay un sólo elemento en la pila
  - El valor de ese elemento es el resultado del cálculo.
- Si hay más de un elemento en la pila
  - **(Error)** El usuario ha introducido demasiados elementos.

#### Ejemplo

La expresión algebraica 5+((1+2)*4)−3 se traduce a la notación polaca inversa como 5 1 2 + 4 * + 3 - y se evalúa de izquierda a derecha según se muestra en la siguiente tabla. La “Pila” es la lista de los valores que el algortimo mantiene en su memoria después de realizar la operación dada en la segunda columna.

Al finalizar el cálculo, el resultado (en este caso, 14) aparece como el único elemento en la pila.

### 1.7.5 Convirtiendo desde la notación de infijo a la notación de postfijo

Edsger Dijkstra inventó el algoritmo shunting yard (patio de clasificación) para convertir expresiones de infijo al postfijo (RPN), nombrado así porque su operación se asemeja al de un patio de clasificación de ferrocarril.

Hay otras maneras de producir expresiones de posfijo desde la notación de infijo. La mayoría de los parsers de precedencia de operador pueden ser modificados para producir expresiones de posfijo; en particular, una vez que que haya sido construido un árbol de sintaxis abstracta, la expresión correspondiente de posfijo es dada por un recorrido postorden de ese árbol.

### 1.7.6 Implementaciones

#### Historia de las implementaciones

La primera computadora en implementar la arquitectura que permitía el RPN fue la máquina KDF9 de la compañía English Electric, que fue anunciada en 1960 y entregada (es decir, hecha disponible comercialmente) en 1963, y la Burroughs B5000 de Estados Unidos, anunciada en 1961 y también entregada en 1963. Uno de los diseñadores del B5000, Robert S. Barton, escribió más tarde que él desarrolló el RPN independientemente de Hamblin, en algún momento de 1958 mientras que leía un libro de texto sobre lógica simbólica, y antes de conocer el trabajo de Hamblin.

#### El RPN en Hewlett-Packard

Friden, Inc. introdujo la notación polaca inversa (RPN) al mercado de las calculadoras de escritorio con el EC-130 en junio de 1963. Los ingenieros de Hewlett-Packard (HP) diseñaron la calculadora de escritorio 9100A con RPN en 1968. Esta calculadora popularizó el RPN entre las comunidades científicas y de ingeniería, aunque los primeros anuncios para el 9100A fallaron en mencionar el RPN. El HP-35, la primera calculadora científica de mano del mundo, usaba RPN en 1972. HP usó el RPN en cada calculadora de mano que vendió, ya fuera científica, financiera, o programable, hasta que introdujo una calculadora al estilo de máquina de adición, el HP-10A. HP introdujo una línea de calculadoras basada en LCD a principios de los años 1980 que usaba RPN, tales como las HP-10C, HP-11C, HP-15C, HP-16C, y la famosa calculadora financiera, la HP-12C. Cuando Hewlett-Packard introdujo una posterior calculadora de negocios, el HP-19B, sin RPN, la retroalimentación de los financieros y otros acostumbrados a la 12-C le obligó a que lanzara el HP-19BII, que dio a los usuarios la opción de usar la notación algebraica o el RPN. Desde 1990 a 2003, HP manufacturó la serie HP48 de calculadoras gráficas RPN y en 2006 introdujo el HP-50g con un LCD de 131x80 píxels y un CPU ARM de 75 MHz que emulaba el CPU HP Saturn de la serie HP-48.

#### El RPN en Unión Soviética

Las calculadoras programables soviéticas (los modelos MK-52, MK-61, B3-34 y el temprano B3-21)[4] usaron el RPN para el modo automático y la programación. Las calculadoras rusas modernas MK-161[5] y MK-152,[6] diseñadas y manufacturadas en Novosibirsk desde 2007, son compatibles hacia atrás con ellos. Su arquitectura extendida también se basa en la notación polaca reversa.

### 1.7.7 Otros datos

- HP-15C ha sido posiblemente la calculadora RPN más popular, hasta los puntos de que existe una petición online para que HP vuelva a fabricarla ya que adquirir un ejemplar de segunda mano en buen estado de HP-15C cuesta varios cientos de dólares.

### 1.7.8 Véase también

- Notación de infijo
- Notación de prefijo
- Notación polaca
- Algoritmo shunting yard
- LIFO
- Máquina de pila
- Grandes sistemas de Burroughs
- Lenguaje de programación orientado a pila
- Lenguajes de programación
  - Forth
  - Joy
  - Factor
- Calculadoras HP, HP-9100, HP-35
- PostScript
- Métodos de entrada de las calculadoras
- Calculadora Orpie

### 1.7.9 Referencias

[1] "An Analysis of a Logical Machine Using Parenthesis-Free Notation," by Arthur W. Burks, Don W. Warren and Jesse B. Wright, 1954

[2] "Charles L. Hamblin and his work" by Peter McBurney

[3] "Charles L. Hamblin: Computer Pioneer" by Peter McBurney, July 27, 2008. "*Hamblin soon became aware of the problems of (a) computing mathematical formulae containing brackets, and (b) the memory overhead in having dealing with memory stores each of which had its own name. One solution to the first problem was Jan Lukasiewicz's Polish notation, which enables a writer of mathematical notation to instruct a reader the order in which to execute the operations (e.g. addition, multiplication, etc) without using brackets. Polish notation achieves this by having an operator (+, *, etc) precede the operands to which it applies, e.g., +ab, instead of the usual, a+b. Hamblin, with his training in formal logic, knew of Lukasiewicz's work.*"

[4] Elektronika B3-21 page on RSkey.org

[5] Elektronika MK-161 page on RSkey.org

[6] MK-152: Old Russian Motive in a New Space Age.

### 1.7.10 Enlaces externos

- *RPN or DAL? A brief analysis of Reverse Polish Notation against Direct Algebraic Logic* – By James Redin
- Postfix Notation Mini-Lecture – By Bob Brown
- Fith: An Alien Conlang With A LIFO Grammar – By Jeffrey Henning
- Good Ideas, Through the Looking Glass - by Niklaus Wirth
- Ambi browser-based extensible RPN calculator - By David Pratten
- RPN at HP Museum
- RPN Android Muestra la Cadena RPN para android
- Algoritmo RPN programado en VB.NET

## 1.8 Árbol binario de búsqueda auto-balanceable

En ciencias de la computación, un **árbol binario de búsqueda auto-balanceable** o **equilibrado** es un árbol binario de búsqueda que intenta mantener su *altura*, o el número de niveles de nodos bajo la raíz, tan pequeños como sea posible en todo momento, automáticamente. Esto es importante, ya que muchas operaciones en un árbol de búsqueda binaria tardan un tiempo proporcional a la altura del árbol, y los árboles binarios de búsqueda ordinarios pueden tomar alturas muy grandes en situaciones normales, como cuando las claves son insertadas en orden. Mantener baja la altura se consigue habitualmente realizando transformaciones en el árbol, como la rotación de árboles, en momentos clave.

Tiempos para varias operaciones en términos del número de nodos en el árbol n:

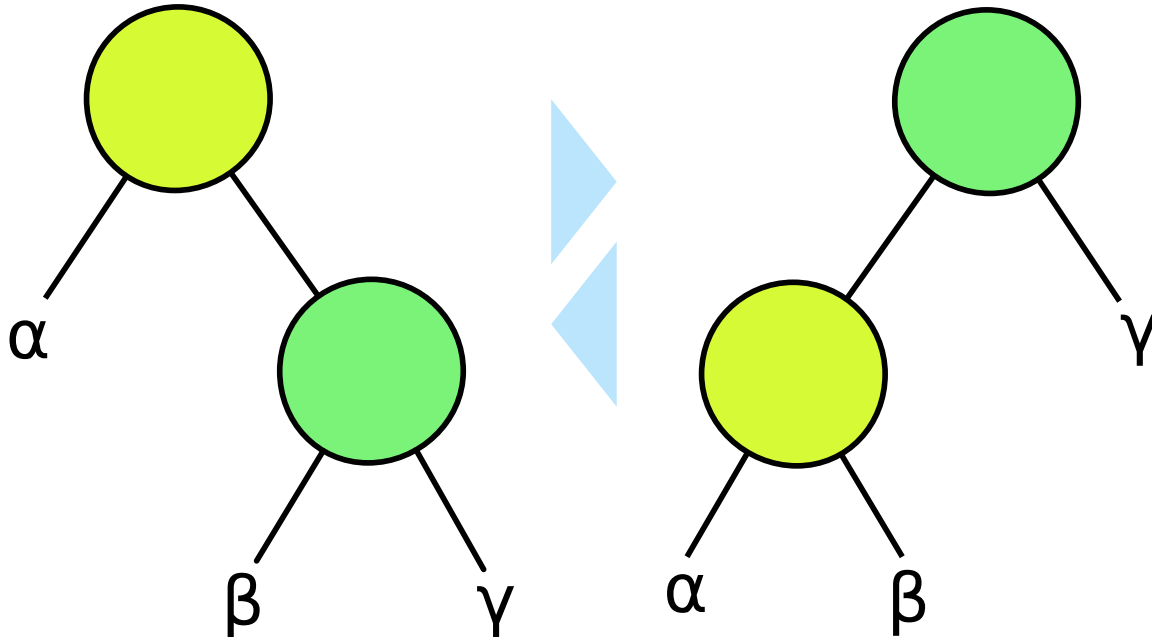
Para algunas implementaciones estos tiempos son el peor caso, mientras que para otras están amortizados.

Estructuras de datos populares que implementan este tipo de árbol:

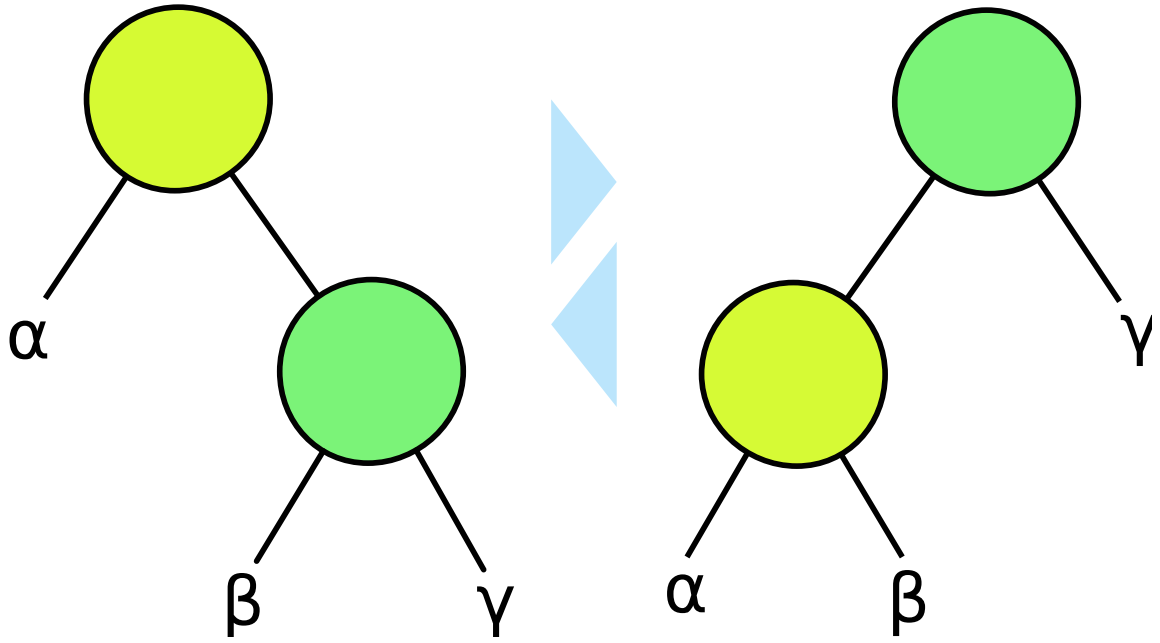
- Árbol AVL
- Árbol rojo-negro

### 1.8.1 Véase también

- Árbol-B




*Las rotaciones internas en árboles binarios son operaciones internas comunes utilizadas para mantener el balance perfecto o casi perfecto del árbol binario. Un árbol balanceado permite operaciones en tiempo logarítmico*




*Generic tree rotations.*

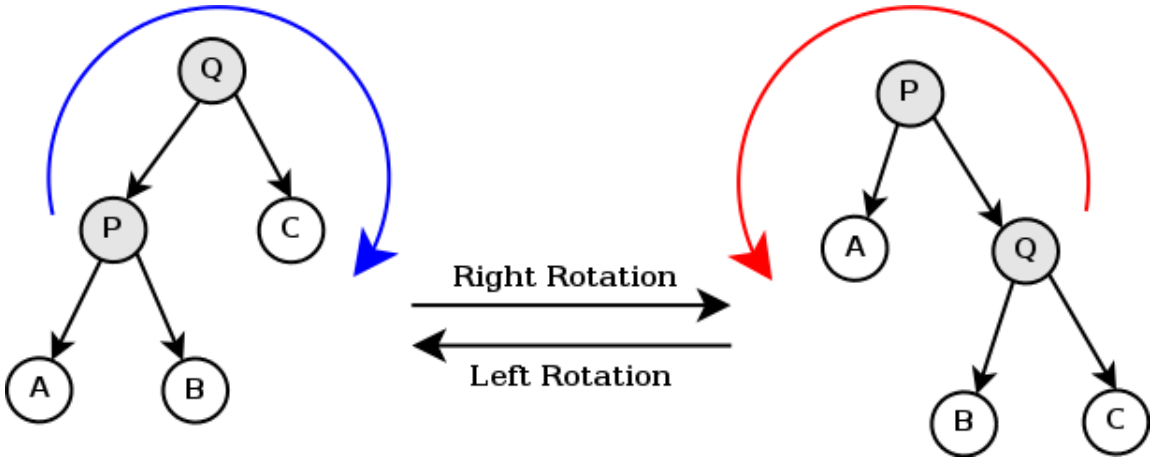
## 1.9 Rotación de árboles

En matemáticas discretas, **Rotacion de arboles** es una operación en un árbol binario que cambia la estructura sin interferir con el orden de los elementos. Un árbol de rotación se mueve hasta un nodo en el árbol y un nodo hacia abajo. Se utiliza para cambiar la forma del árbol, y en particular para disminuir su altura moviendo subárboles más pequeños hacia abajo y subárboles más grande, lo que resulta en un mejor rendimiento de muchas operaciones de los árboles.

Existe una inconsistencia en diferentes descripciones en cuanto a la definición de la **dirección de las rotaciones**. Algunos dicen que la dirección de la rotación depende del lado que los nodos del árbol se desplazan al mismo tiempo que otros dicen que depende de qué niño toma el lugar de la raíz (enfrente de la primera). En este artículo se adopta

el enfoque del lado donde los nodos quedan cambiaron a.

### 1.9.1 Ilustracion




La operación de rotación a la derecha, como se muestra en la imagen de la derecha se realiza con Q como la raíz y por lo tanto es un giro a la derecha en, o enraizado en, resultados Q. Esta operación en una rotación del árbol en el sentido horario. La operación inversa es la rotación a la izquierda, lo que resulta en un movimiento en sentido contrario a las agujas del reloj (la rotación izquierda se muestra más arriba tiene sus raíces en P). La clave para entender cómo funciona una rotación es entender sus limitaciones. En particular, el orden de las hojas del árbol (cuando se lee de izquierda a derecha, por ejemplo) no se puede cambiar (otra forma de pensar de él es que el orden en que las hojas se pueden visitar en un recorrido en el orden debe ser el mismo después de la operación como antes). Otra limitación es la característica principal de un árbol de búsqueda binaria, es decir, que el hijo derecho es mayor que el padre y el niño queda es menor que la de los padres. Observe que el hijo derecho de un hijo izquierdo de la raíz de un sub-árbol (por ejemplo nodo B en el diagrama de árbol con raíz en Q) puede convertirse en el hijo izquierdo de la raíz, que en sí mismo se convierte en el hijo derecho de la “nueva “root en el sub-árbol girado, sin violar ninguna de esas limitaciones. Como se puede ver en el diagrama, el orden de las hojas no cambia. La operación opuesta también conserva el orden y es el segundo tipo de rotación.

Asumiendo que esto es un árbol de búsqueda binaria, como se ha dicho, los elementos deben ser interpretados como variables que se pueden comparar entre sí. Los caracteres alfabéticos de la izquierda se utilizan como marcadores de posición para estas variables. En la animación a la derecha, los caracteres alfabéticos de capital se utilizan como marcadores de posición variables, mientras que las letras griegas minúsculas son marcadores de posición para todo un conjunto de variables. Los círculos representan los nodos individuales y los triángulos representan subárboles. Cada subárbol podría estar vacío, consistir en un único nodo, o consistir en cualquier número de nodos.

### 1.9.2 Ilustración detallada

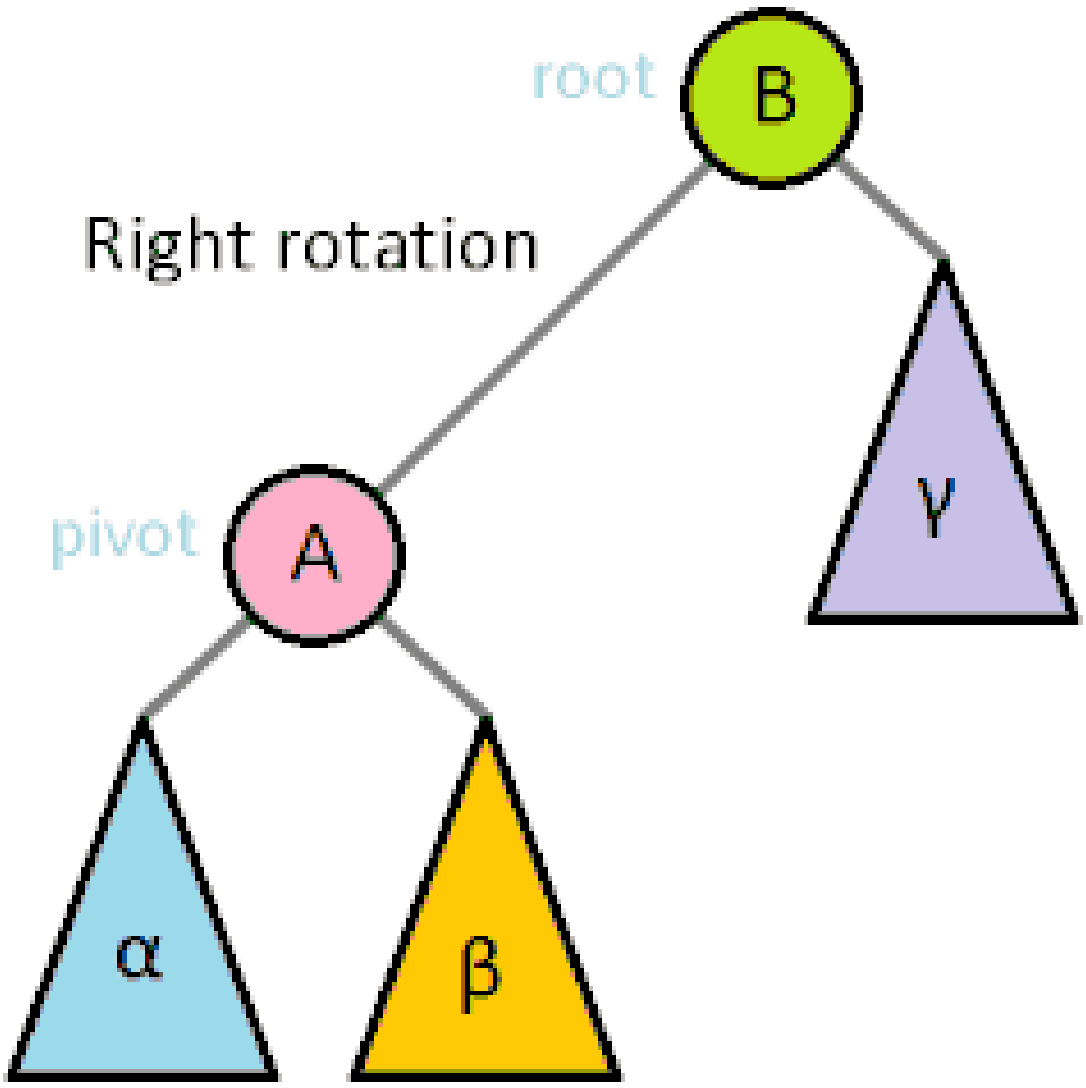
Cuando se hace girar un subárbol, el lado subárbol sobre la cual se hace girar aumenta su altura por un nodo mientras que el otro subárbol disminuye su altura. Esto hace que las rotaciones de árboles útiles para el reequilibrio de un árbol.

Utilizando la terminología de raíz para el nodo padre de los subárboles para girar, Pivot para el nodo que se convertirá en el nuevo nodo padre, RS lado rotación a girar y OS para el lado opuesto de la rotación. En el diagrama anterior para la raíz Q, el RS es C y el sistema operativo es P. El pseudo código para la rotación es:

Pivote = Root.OS Root.OS = Pivot.RS Pivot.RS = Raíz Root = Pivot

Esta es una operación constante de tiempo.

El programador también debe asegurarse de que los puntos de matrices de la raíz al pivote después de la rotación. Además, el programador debe tener en cuenta que esta operación puede dar lugar a una nueva raíz para todo el árbol y tener cuidado para actualizar los punteros en consecuencia.




*Animation of tree rotations taking place.*

### 1.9.3 Invarianza en orden

La rotación del árbol hace que el recorrido en orden de la invariante árbol binario. Esto implica el orden de los elementos no se ve afectada cuando se realiza una rotación en cualquier parte del árbol. Aquí están los recorridos finde de los árboles que se muestran arriba:

Left tree: ((A, P, B), Q, C) Right tree: (A, P, (B, Q, C))

Computar uno a partir del otro es muy simple. Lo siguiente es ejemplo de código Python que realiza ese cálculo:

def right_rotation(treenode): left, Q, C = treenode A, P, B = left return (A, P, (B, Q, C))

Otra forma de verlo es:

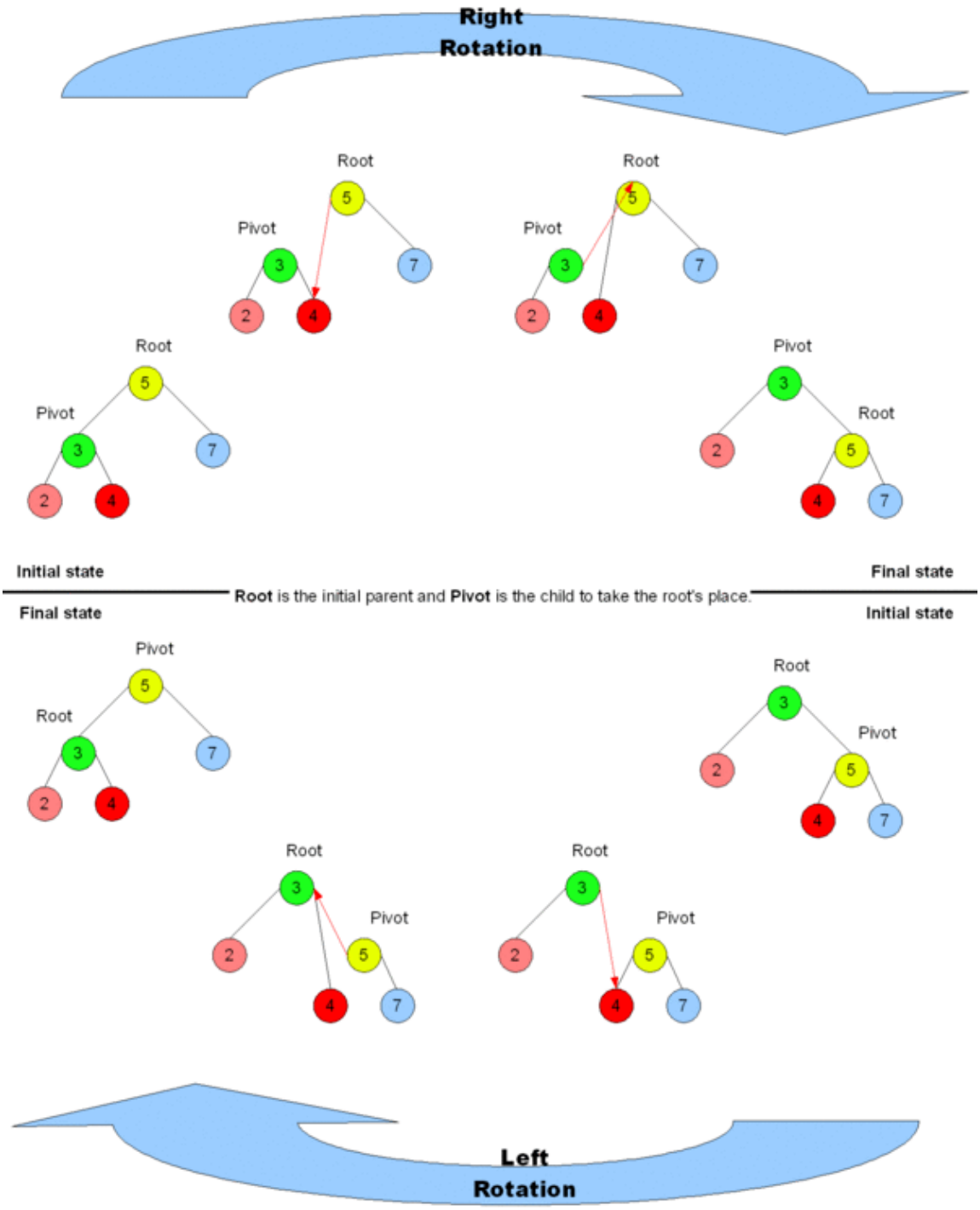
Giro a derechas del nodo Q:

Let P be Q's left child. Set Q's left child to be P's right child. Set P's right child to be Q.

Rotación izquierda del nodo P:

Let Q be P's right child. Set P's right child to be Q's left child. Set Q's left child to be P.

Todas las demás conexiones se dejan tal cual.




*Pictorial description of how rotations are made.*

También hay rotaciones dobles, que son combinaciones de rotaciones de izquierda y derecha. Un doble rotación a la izquierda en X puede ser definido como una rotación a la derecha en el hijo derecho de X seguida de una rotación a la izquierda en X; Del mismo modo, una doble rotación a la derecha en X puede ser definido como una rotación a la izquierda en el hijo izquierdo de X seguida de una rotación a la derecha en X.




*Pictorial description of how rotations cause rebalancing in an AVL tree.*

### 1.9.4 Rotaciones de reequilibrio

Un árbol puede reequilibrarse mediante rotaciones. Después de una rotación, el lado de la rotación aumenta su altura por 1, mientras que el lado opuesto a la rotación disminuye su altura de manera similar. Por lo tanto, se puede aplicar estratégicamente rotaciones para nodos cuyos izquierda infantil y derecho difieren en altura por más de 1. auto-equilibrio árboles binarios de búsqueda aplicar esta operación automáticamente. Un tipo de árbol que utiliza esta técnica reequilibrio es el árbol AVL.

### 1.9.5 Distancia de rotacion

La **distancia de rotación** entre dos árboles binarios con el mismo número de nodos es el número mínimo de rotaciones necesarias para transformar una en la otra. Con esta distancia, el conjunto de los árboles binarios de n-nodo se convierte en un espacio métrico: la distancia es simétrica y positiva cuando se les da dos árboles diferentes, y satisface la desigualdad triángular.

Es un problema abierto si existe un algoritmo de tiempo polinómico para el cálculo de la distancia de rotación.

Daniel Sleator, Robert Tarjan y William Thurston mostraron que la distancia de rotación entre dos árboles n-nodo (para n = 11) es a lo sumo 2n - 6, y que un número infinito de pares de árboles son tan separados. [1]

### 1.9.6 Ver también

- Portal:Ciencia de la Computacion. Contenido relacionado con **Ciencia de la Computacion**.
- árbol AVL, árbol rojo-negro, and árbol ensanchamiento, tipos de estructuras de datos árbol binario de búsqueda que utilizan rotaciones para mantener el equilibrio.
- Asociatividad de una operación binaria significa que la realización de una rotación de árbol en él no cambia el resultado final.

- El algoritmo Day-Stout-Warren equilibra un BST desequilibrada.
- Tamari lattice, un conjunto parcialmente ordenado en el que los elementos pueden ser definidos como árboles binarios y el orden entre los elementos se definen por la rotación del árbol.

### 1.9.7 Referencias

[1] Sleator, Daniel D.; Tarjan, Robert E.; Thurston, William P. (1988), «Rotation distance, triangulations, and hyperbolic geometry», *Journal of the American Mathematical Society* (American Mathematical Society) **1** (3): 647–681, doi:10.2307/1990951.

### 1.9.8 Links Externos

- Java applets demonstrating tree rotations
- The AVL Tree Rotations Tutorial (RTF) by John Hargrove

## 1.10 Árbol-B

### 1.10.1 Definición

La idea tras los árboles-B es que los nodos internos deben tener un número variable de nodos hijo dentro de un rango predefinido. Cuando se inserta o se elimina un dato de la estructura, la cantidad de nodos hijo varía dentro de un nodo. Para que siga manteniéndose el número de nodos dentro del rango predefinido, los nodos internos se juntan o se parten. Dado que se permite un rango variable de nodos hijo, los árboles-B no necesitan rebalancearse tan frecuentemente como los árboles binarios de búsqueda auto-balanceables. Pero, por otro lado, pueden desperdiciar memoria, porque los nodos no permanecen totalmente ocupados. Los límites (uno superior y otro inferior) en el número de nodos hijo son definidos para cada implementación en particular. Por ejemplo, en un **árbol-B 2-3** (A menudo simplemente llamado árbol 2-3 ), nodo sólo puede tener 2 ó 3 nodos hijo.

Un árbol-B se mantiene balanceado porque requiere que todos los nodos hoja se encuentren a la misma altura.

Los árboles B tienen ventajas sustanciales sobre otras implementaciones cuando el tiempo de acceso a los nodos excede al tiempo de acceso entre nodos. Este caso se da usualmente cuando los nodos se encuentran en dispositivos de almacenamiento secundario como los discos rígidos. Al maximizar el número de nodos hijo de cada nodo interno, la altura del árbol decrece, las operaciones para balancearlo se reducen, y aumenta la eficiencia. Usualmente este valor se coloca de forma tal que cada nodo ocupe un bloque de disco, o un tamaño análogo en el dispositivo. Mientras que los árboles B 2-3 pueden ser útiles en la memoria principal, y además más fáciles de explicar, si el tamaño de los nodos se ajustan para caber en un bloque de disco, el resultado puede ser un árbol B 129-513.

Los creadores del árbol B, Rudolf Bayer y Ed McCreight, no han explicado el significado de la letra *B* de su nombre. Se cree que la *B* es de *balanceado*, dado que todos los nodos hoja se mantienen al mismo nivel en el árbol. La *B* también puede referirse a *Bayer*, o a Boeing, porque sus creadores trabajaban en los *Boeing Scientific Research Labs* por ese entonces.

### 1.10.2 Definición técnica

B-árbol es un árbol de búsqueda que puede estar vacío o aquel cuyos nodos pueden tener varios hijos, existiendo una relación de orden entre ellos, tal como muestra el dibujo.

Un árbol-B de orden M (el máximo número de hijos que puede tener cada nodo) es un árbol que satisface las siguientes propiedades:

1. Cada nodo tiene como máximo M hijos.
2. Cada nodo (excepto raíz) tiene como mínimo (M-1)/2 claves.
3. La raíz tiene al menos 2 hijos si no es un nodo hoja. (según M)
4. Todos los nodos hoja aparecen al mismo nivel.

5. Un nodo no hoja con k hijos contiene k-1 elementos almacenados.
6. Los hijos que cuelgan de la raíz (r1, ···, rm) tienen que cumplir ciertas condiciones:
   (a) El primero tiene valor menor que r1.
   (b) El segundo tiene valor mayor que r1 y menor que r2, etc.
   (c) El último hijo tiene valor mayor que rm.

### 1.10.3 Altura: El mejor y el peor caso

En el mejor de los casos,la altura de un árbol-B es:

$log_M n$

En el peor de los casos,la altura de un árbol-B es:

$log_{(\frac{M}{2})} n$

Donde M es el número máximo de hijos que puede tener un nodo.

### 1.10.4 Estructura de los nodos

Cada elemento de un nodo interno actúa como un valor separador, que lo divide en subárboles. Por ejemplo, si un nodo interno tiene tres nodos hijo, debe tener dos valores separadores o elementos $a_1$ y $a_2$. Todos los valores del subárbol izquierdo deben ser menores a $a_1$, todos los valores del subárbol del centro deben estar entre $a_1$ y $a_2$, y todos los valores del subárbol derecho deben ser mayores a $a_2$.

Los nodos internos de un árbol B, es decir los nodos que no son hoja, usualmente se representan como un conjunto ordenado de elementos y punteros a los hijos. Cada nodo interno contiene un máximo de *U* hijos y, con excepción del nodo raíz, un mínimo de *L* hijos. Para todos los nodos internos exceptuando la raíz, el número de elementos es uno menos que el número de punteros a nodos. El número de elementos se encuentra entre *L-1* y *U-1*. El número *U* debe ser $2L$ o $2L-1$, es decir, cada nodo interno está por lo menos a medio llenar. Esta relación entre *U* y *L* implica que dos nodos que están a medio llenar pueden juntarse para formar un nodo legal, y un nodo lleno puede dividirse en dos nodos legales (si es que hay lugar para subir un elemento al nodo padre). Estas propiedades hacen posible que el árbol B se ajuste para preservar sus propiedades ante la inserción y eliminación de elementos.

Los nodos hoja tienen la misma restricción sobre el número de elementos, pero no tienen hijos, y por tanto carecen de punteros.

El nodo raíz tiene límite superior de número de hijos, pero no tiene límite inferior. Por ejemplo, si hubiera menos de *L-1* elementos en todo el árbol, la raíz sería el único nodo del árbol, y no tendría hijos.

Un árbol B de altura *n+1* puede contener *U* veces por elementos más que un árbol B de profundidad *n*, pero el costo en la búsqueda, inserción y eliminación crece con la altura del árbol. Como todo árbol balanceado, el crecimiento del costo es más lento que el del número de elementos.

Algunos árboles balanceados guardan valores sólo en los nodos hoja, y por lo tanto sus nodos internos y nodos hoja son de diferente tipo. Los árboles B guardan valores en cada nodo, y pueden utilizar la misma estructura para todos los nodos. Sin embargo, como los nodos hoja no tienen hijos, una estructura especial para éstos mejora el funcionamiento.

**Class nodo árbol B en c++**

#define TAMANO 1000 struct stclave { int valor; long registro; }; class bnodo { public: bnodo (int nClaves); // Constructor ~bnodo (); // Destructor private: int clavesUsadas; stclave *clave; bnodo **puntero; bnodo *padre; friend class btree; };

### 1.10.5 Algoritmos

#### Búsqueda

La búsqueda es similar a la de los árboles binarios. Se empieza en la raíz, y se recorre el árbol hacia abajo, escogiendo el sub-nodo de acuerdo a la posición relativa del valor buscado respecto a los valores de cada nodo. Típicamente se utiliza la búsqueda binaria para determinar esta posición relativa.

#### Procedimiento

1. Situarse en el nodo raíz.
2. (*) Comprobar si contiene la clave a buscar.
   (a) Encontrada fin de procedimiento.
   (b) No encontrada:
      i. Si es hoja no existe la clave.
      ii. En otro caso el nodo actual es el hijo que corresponde:
         A. La clave a buscar k < k1: hijo izquierdo.
         B. La clave a buscar k > ki y k < ki+1 hijo iésimo.
         C. Volver a paso 2(*).

#### Inserción

Las inserciones se hacen en los nodos hoja.

1. Realizando una búsqueda en el árbol, se halla el nodo hoja en el cual debería ubicarse el nuevo elemento.
2. Si el nodo hoja tiene menos elementos que el máximo número de elementos legales, entonces hay lugar para uno más. Inserte el nuevo elemento en el nodo, respetando el orden de los elementos.
3. De otra forma, el nodo debe ser dividido en dos nodos. La división se realiza de la siguiente manera:
   (a) Se escoge el valor medio entre los elementos del nodo y el nuevo elemento.
   (b) Los valores menores que el valor medio se colocan en el nuevo nodo izquierdo, y los valores mayores que el valor medio se colocan en el nuevo nodo derecho; el valor medio actúa como valor separador.
   (c) El valor separador se debe colocar en el nodo padre, lo que puede provocar que el padre sea dividido en dos, y así sucesivamente.

Si las divisiones de nodos suben hasta la raíz, se crea una nueva raíz con un único elemento como valor separador, y dos hijos. Es por esto por lo que la cota inferior del tamaño de los nodos no se aplica a la raíz. El máximo número de elementos por nodo es $U-1$. Así que debe ser posible dividir el número máximo de elementos $U-1$ en dos nodos legales. Si este número fuera impar, entonces $U=2L$, y cada uno de los nuevos nodos tendrían $(U-2)/2 = L-1$ elementos, y por lo tanto serían nodos legales. Si $U-1$ fuera par, $U=2L-1$, así que habría $2L-2$ elementos en el nodo. La mitad de este número es $L-1$, que es el número mínimo de elementos permitidos por nodo.

Un algoritmo mejorado admite una sola pasada por el árbol desde la raíz, hasta el nodo donde la inserción tenga lugar, dividiendo todos los nodos que estén llenos encontrados a su paso. Esto evita el costo de volver a cargar en memoria los nodos padres (lo que puede llegar a ser caro si los nodos se encuentran en una memoria secundaria). Sin embargo, para usar este algoritmo mejorado, debemos ser capaces de enviar un elemento al nodo padre y dividir el resto $U-2$ elementos en 2 nodos legales, sin añadir un nuevo elemento. Esto requiere que $U=2L$ en lugar de $U=L-1$, lo que explica por qué algunos libros de texto imponen este requisito en la definición de árboles-B.

### Eliminación

La eliminación de un elemento es directa si no se requiere corrección para garantizar sus propiedades. Hay dos estrategias populares para eliminar un nodo de un árbol B.

- localizar y eliminar el elemento, y luego corregir, o
- hacer una única pasada de arriba a abajo por el árbol, pero cada vez que se visita un nodo, reestructurar el árbol para que cuando se encuentre el elemento a ser borrado, pueda eliminarse sin necesidad de continuar reestructurando

Se pueden dar dos problemas al eliminar elementos. Primero, el elemento puede ser un separador de un nodo interno. Segundo, puede suceder que al borrar el elemento número de elementos del nodo quede debajo de la cota mínima. Estos problemas se tratan a continuación en orden.

### Eliminación en un nodo hoja

- Busque el valor a eliminar.
- Si el valor se encuentra en un nodo hoja, se elimina directamente la clave, posiblemente dejándolo con muy pocos elementos; por lo que se requerirán cambios adicionales en el árbol.

**Eliminación en un nodo interno** (recursivamente) del nuevo nodillo.

En el segundo caso, uno de los dos nodos hijos tienen un número de elementos mayor que el mínimo. Entonces izquierdo o el menor elemento de el nuevo separador.

- Como se ha eliminado un elemento de un nodo hoja, se tar este caso de manera equivalente.

### Rebalanceo después de la eliminación

Si al eliminar un elemento de un nodo hoja el nodo se ha quedado con menos elementos que el mínimo permitido, algunos elementos se deben redistribuir. En algunos casos el cambio lleva la deficiencia al nodo padre, y la redistribución se debe aplicar iterativamente hacia arriba del árbol, quizá incluso hasta a la raíz. Dado que la cota mínima en el número de elementos no se aplica a la raíz, el problema desaparece cuando llega a ésta.

### Construcción Inicial

En aplicaciones, es frecuentemente útil construir un árbol-B para representar un gran número de datos existentes y después actualizarlo de forma creciente usando operaciones estándar de los árboles-B. En este caso, el modo más eficiente para construir el árbol-B inicial no sería insertar todos los elementos en el conjunto inicial sucesivamente, sino construir el conjunto inicial de nodos hoja directamente desde la entrada, y después construir los nodos internos a partir de este conjunto. Inicialmente, todas las hojas excepto la última tienen un elemento más, el cual será utilizado para construir los nodos internos. Por ejemplo, si los nodos hoja tienen un tamaño máximo de 4 y el conjunto inicial es de enteros desde el 1 al 24, tenemos que construir inicialmente 5 nodos hoja conteniendo 5 valores cada uno (excepto el último que contiene 4):

Construiremos el siguiente nivel hacia arriba desde las hojas tomando el último elemento de cada hoja excepto el último. De nuevo, cada nodo excepto el último contendrá un valor más. En el ejemplo, es supuesto que los nodos internos contienen como mucho 2 valores (por lo que pueden tener 3 hijos). Luego el siguiente nivel de nodos internos nos quedaría de la siguiente manera:

Este proceso se continuará hasta que alcancemos un nivel con un solo nodo y no está sobrecargado. En nuestro ejemplo solo nos quedaría el nivel de la raíz:

### 1.10.6 Notas

Cada nodo tendrá siempre entre *L* y *U* hijos incluidos con una excepción: El nodo raíz debe tener entre 2 y *U* hijos. En otras palabras, la raíz está exenta de la restricción del límite inferior. Esto permite al árbol sostener un pequeño número de elementos. Un nodo raíz con un solo hijo no tendría sentido, ya que podríamos añadírselo a la raíz. Un nodo raíz sin hijos es también innecesario, ya que un árbol sin hijos se suele representar sin raíz.

#### Multi-modo:combinar y dividir

Es posible modificar el algoritmo anterior, cuando tratamos de encontrar más elementos para un nodo al que le faltan, examinamos a los hermanos, y si alguno tiene más del valor mínimo de números, reordenamos los valores de los hermanos de un extremo a otro para rellenar al mínimo el nodo al que le faltan. De la misma manera, cuando un nodo se divide, los elementos extra pueden ser movidos cerca, por ejemplo a hermanos menos poblados; o la división puede dar lugar a un número de hermanos, redistribuyendo los elementos entre ellos en lugar de dividir un nodo.

En la práctica, el uso más común de los árboles-B implica mantener los nodos una memoria secundaria, donde será lento acceder a un nodo que no haya sido usado con anterioridad. Utilizando solo divisiones y combinaciones, disminuimos el número de nodos que se necesitan para la mayoría de situaciones comunes, pero podrían ser útiles en otras.

#### Relación entre U y L

Es casi universal el dividir nodos eligiendo un elemento medio y creando dos nuevos nodos. Esto limita la relación entre *L* y *U*. Si intentamos insertar un elemento dentro de un nodo con *U* elementos, esto conlleva una redistribución de *U* elementos. Uno de estos, el intermedio, será trasladado al nodo padre, y los restantes serán divididos equitativamente, en la medida de lo posible, entre los dos nuevos nodos.

Por ejemplo, en un árbol-B 2-3, añadiendo un elemento a un nodo que ya contiene 3 hijos, y por consiguiente 2 valores separadores (padres), da lugar a 3 valores (los dos separadores y el nuevo valor). El valor medio se convierte en el nuevo separador (padre), y los otros valores se hacen independientes y con 2 hijos. Por lo general, si *U* es impar, cada uno de los nuevos nodos tienen (*U*+2)/2 hijos. Si *U* es par, unos tiene *U*/2 hijos y el otro *U*/2+1.

Si un nodo está completo y se divide exactamente en 2 nodos, *L* debe tener un tamaño permitido, lo suficiente pequeño, una vez q el nodo ha sido divido. También es posible dividir nodos completos en más de dos nodos nuevos. Eligiendo dividir un nodo en más de 2 nodos nuevos requerirá un valor más pequeño para *L* para el mismo valor de *U*. Como *L* se hace más pequeño, esto permite que haya más espacio sin usar en los nodos. Esto disminuirá la frecuencia de división de nodos, pero de la misma manera aumentará la cantidad de memoria que se necesita para almacenar el mismo número de valores, y el número de nodos que tienen que ser examinados para una operación particular.

#### Acceso concurrente

Lehman y Yao nos mostraron que, uniendo los bloques de árboles en cada nivel con un puntero al siguiente nivel (en una estructura de árbol donde los permisos, de lectura de los bloques del árbol, se pueden evitar porque el árbol desciende desde la raíz hasta las hojas por búsqueda e inserción), los permisos de escritura solo se requieren cuando un bloque del árbol es modificado. Minimizar los permisos a un nodo colgante simple solo durante su modificación, ayuda a maximizar el acceso concurrente por múltiples usuarios. Este dato se tiene en cuenta en bases de datos como, por ejemplo, ISAM (Métodos Indexados de Acceso Secuencial).

### 1.10.7 Véase también

- Árbol
- Árbol binario
- Árbol-B+
- Árbol-B*
- Partición de espacio binario

- Árbol rojo-negro
- Árbol AA
- Skip list

### 1.10.8 Referencias

### 1.10.9 Enlaces externos

- Ejemplo de funcionamiento del árbol 2-3-4 (la variante más simple de árbol-B) (en inglés)
- Applet de ejemplo del funcionamiento del árbol B (en inglés)
- Tutorial sobre árboles B
- Indexación de datos con árbol B en Java

## 1.11 Montículo (informática)

En computación, un **montículo** (*heap* en inglés) es una estructura de datos del tipo árbol con información perteneciente a un conjunto ordenado. Los montículos máximos tienen la característica de que cada nodo padre tiene un valor mayor que el de cualquiera de sus nodos hijos, mientras que en los montículos mínimos, el valor del nodo padre es siempre menor al de sus nodos hijos.

Un árbol cumple la condición de montículo si satisface dicha condición y además es un árbol binario completo. Un árbol binario es completo cuando todos los niveles están llenos, con la excepción del último, que se llena desde la izquierda hacia la derecha.

En un montículo de prioridad, el mayor elemento (o el menor, dependiendo de la relación de orden escogida) está siempre en el nodo raíz. Por esta razón, los montículos son útiles para implementar colas de prioridad. Una ventaja que poseen los montículos es que, por ser árboles completos, se pueden implementar usando arreglos (arrays), lo cual simplifica su codificación y libera al programador del uso de punteros.

La eficiencia de las operaciones en los montículos es crucial en diversos algoritmos de recorrido

### 1.11.1 Operaciones

Las operaciones más importantes o básicas en un montículo son la de inserción y la de eliminación de uno o varios elementos.

#### Insertar

Monticulus: Esta operación parte de un elemento y lo inserta en un montículo aplicando su criterio de ordenación.Si suponemos que el monticulo está estructurado de forma que la raíz es mayor que sus hijos, comparamos el elemento a insertar (incluido en la primera posición libre) con su padre.Si el hijo es menor que el padre,entonces el elemento es insertado correctamente, si ocurre lo contrario sustituimos el hijo por el padre.

¿Y si la nueva raíz sigue siendo más grande que su nuevo padre?. Volvemos a hacer otra vez dicho paso hasta que el montículo quede totalmente ordenado. En la imagen adjunta vemos el ejemplo de cómo realmente se inserta un elemento en un montículo. Aplicamos la condición de que cada padre sea mayor que sus hijos, y siguiendo dicha regla el elemento a insertar es el 12. Es mayor que su padre, siguiendo el método de ordenación, sustituimos el elemento por su padre que es 9 y así quedaría el montículo ordenado.

Ahora veremos la implementación en varios lenguajes de programación del algoritmo de inserción de un elemento en un montículo.

En *Maude* el insertar se realiza a través de un constructor:

op insertarHeap : X$Elt Heap{X} -> HeapNV{X} eq insertarHeap(R, crear) = arbolBin(R, crear, crear) . eq insertarHeap(R1, arbolBin(R2, I, D)) = if ((altura(I) > altura(D)) and not estaLleno?(I)) or (((altura(I) == altura(D)) and estaLleno?(D)) then arbolBin(max(R1, R2),insertarHeap(min(R1, R2), I), D) else arbolBin(max(R1, R2), I,insertarHeap(min(R1, R2), D)) fi .

En pseudolenguaje quedaría:

PROC Flotar ( M, i ) MIENTRAS (i>1) ^ (M.Vector_montículo[i div 2] < M.Vector montículo[i] HACER intercambiar M.Vector montículo[i div 2] ^ M.Vector_montículo[i] i = i div 2 FIN MIENTRAS FIN PROC PROC Insertar ( x, M ) SI M.Tamaño_montículo = Tamaño_máximo ENTONCES error Montículo lleno SINO M.Tamaño_montículo = M.Tamaño_montículo + 1 M.Vector_montículo[M.Tamaño montículo] = x Flotar ( M, M.Tamaño_montículo ) FIN PROC

En Java el código sería el siguiente:

public void insertItem(Object k, Object e) throws InvalidKeyException { if(!comp.isComparable(k)) throw new InvalidKeyException("Invalid Key"); Position z = T.add(new Item(k, e)); Position u; while(!T.isRoot(z)) { // bubbling-up u = T.parent(z); if(comp.isLessThanOrEqualTo(key(u),key(z))) break; T.swapElements(u, z); z = u; } }

#### Eliminar

En este caso eliminaremos el elemento máximo de un montículo. La forma más eficiente de realizarlo sería buscar el elemento a borrar, colocarlo en la raíz e intercambiarlo por el máximo valor de sus hijos satisfaciendo así la propiedad de montículos de máximos. En el ejemplo representado vemos como 19 que es el elemento máximo es el sujeto a eliminar.

Se puede observar que ya está colocado en la raíz al ser un montículo de máximos, los pasos a seguir son:

1. Eliminar el elemento máximo (colocado en la raíz).
2. Hemos de subir el elemento que se debe eliminar, para cumplir la condición de montículo a la raíz, que ha quedado vacía.
3. Una vez hecho esto queda el último paso el cual es ver si la raíz tiene hijos mayores que ella si es así, aplicamos la condición y sustituimos el padre por el mayor de sus progenitores.

A continuación veremos la especificación de eliminar en distintos lenguajes de programación.

En Maude el código será el siguiente:

eq eliminarHeap(crear) = crear . eq eliminarHeap(HNV) = hundir(arbolBin(ultimo(HNV), hijoIzq(eliminarUltimo(HNV)), hijoDer(eliminarUltimo(HNV)) )

Donde hundir es una operación auxiliar que coloca el nodo en su sitio correspondiente.

En código Java:

public Object removeMin() throws PriorityQueueEmptyException { if(isEmpty()) throw new PriorityQueueEmptyException("Priority Queue Empty!"); Object min = element(T.root()); if(size() == 1) T.remove(); else { T.replaceElement(T.root(), T.remove()); Position r = T.root(); while(T.isInternal(T.leftChild(r))) { Position s; if(T.isExternal(T.rightChild(r)) || comp.isLessThanOrEqualTo(key(T.leftChild(r)),key(T.rightChild(r)))) s = T.leftChild(r); else s = T.rightChild(r); if(comp.isLessThan(key(s), key(r))) { T.swapElements(r, s); r = s; } else break; } } }

Tras haber especificado ambas operaciones y definir lo que es un montículo sólo nos queda por añadir que una de las utilizaciones más usuales del tipo heap (montículo) es en el algoritmo de ordenación de heapsort. También puede ser utilizado como montículo de prioridades donde la raíz es la de mayor prioridad.

### 1.11.2 Véase también

- Montículo binario
- Montículo binómico

- Montículo de Fibonacci
- Montículo suave
- Montículo 2-3

## 1.12 Montículo de Fibonacci

En Informática, un **Montículo de Fibonacci** (o Heap de Fibonacci) es una estructura de datos subconjunto de los montículos, que a su vez, son un subconjunto especial dentro de los bosques de árboles. Resulta similar a un montículo binomial, pero dispone de una mejor relación entre el coste y su amortización. Los montículos de Fibonacci fueron desarrollados en 1984 por Michael L. Fredman y Robert E. Tarjan y publicados por primera vez en una revista científica en 1987. El nombre de montículos de Fibonacci viene de la sucesión de Fibonacci, que se usa en pruebas comparativas de tiempo (Benchmarking).

En particular, las operaciones Insertar, Encontrar el mínimo, Decrementar la clave, y la Unión trabajan con tiempo constante amortizado. Las operaciones Borrar y Borrar el mínimo tienen un coste O(log $n$) como coste amortizado. Esto significa que, empezando con una estructura de datos vacía, cualquier secuencia de $a$ operaciones del primer grupo y $b$ operaciones del segundo grupo tardarían $O(a + b \log n)$. En un montículo binomial cualquier secuencia de operaciones tardarían $O((a + b)\log (n))$. Un montículo de Fibonacci es mejor que un montículo binomial cuando $b$ es asintóticamente más pequeño que $a$.

El montículo de Fibonacci puede ser utilizado para mejorar el tiempo de ejecución asintótico del algoritmo de Dijkstra para calcular el camino más corto en un grafo y el algoritmo de Prim para calcular el árbol mínimo de un grafo.

## 1.13 Estructura de un Montículo de Fibonacci

Un Heap de Fibonacci es una colección de árboles que satisfacen la propiedad del orden mínimo del montículo (que para abreviar se suele utilizar el anglicismo “Min-Heap”), es decir, a grandes rasgos, la clave de un hijo es siempre mayor o igual que la de su padre. Esto implica que la clave mínima está siempre en la raíz. Comparado con los montículos binomiales, la estructura de un montículo de Fibonacci es más flexible. Los árboles no tienen una forma predefinida y en un caso extremo el heap puede tener cada elemento en un árbol separado o en un único árbol de profundidad $n$. Esta flexibilidad permite que algunas operaciones puedan ser ejecutadas de una manera “perezosa”, posponiendo el trabajo para operaciones posteriores. Por ejemplo, la unión de dos montículos se hace simplemente concatenando las dos listas de árboles, y la operación *Decrementar Clave* a veces corta un nodo de su padre y forma un nuevo árbol.

Sin embargo, se debe introducir algún orden para conseguir el tiempo de ejecución deseado. En concreto, el grado de los nodos(el número de hijos) se tiene que mantener bajo: cada nodo tiene un grado máximo de $O(\log n)$ y la talla de un subárbol cuya raíz tiene grado $k$ es por lo menos $Fk_{+2}$, donde $Fk$ es un número de Fibonacci. Esto se consigue con la regla de que podemos cortar como mucho un hijo de cada nodo no raíz. Cuando es cortado un segundo hijo, el nodo también necesita ser cortado de su padre y se convierte en la raíz de un nuevo árbol. El número de árboles se decrementa en la operación *Borrar mínimo*, donde los árboles están unidos entre sí.

Como resultado de esta estructura, algunas operaciones pueden llevar mucho tiempo mientras que otras se hacen muy deprisa. En el análisis del coste de ejecución amortizado pretendemos que las operaciones muy rápidas tarden un poco más de lo que tardan. Este tiempo extra se resta después al tiempo de ejecución de operaciones más lentas. La cantidad de tiempo ahorrada para un uso posterior es medida por una función potencial. Esta función es:

$$\text{Potencial} = t + 2m$$

Donde $t$ es el número de árboles en el montículo de Fibonacci, y $m$ es el número de nodos marcados. Un nodo está marcado si al menos uno de sus hijos se cortó desde que el nodo se fue hecho hijo de otro nodo (todas las raíces están desmarcadas).

Además, la raíz de cada árbol en un montículo tiene una unidad de tiempo almacenada. Esta unidad de tiempo puede ser usada más tarde para unir este árbol a otro con coste amortizado 0. Cada nodo marcado también tiene dos unidades de tiempo almacenadas. Una puede ser usada para cortar el nodo de su padre. Si esto sucede, el nodo se convierte en una raíz y la segunda unidad de tiempo se mantendrá almacenada como en cualquier otra raíz.

## 1.14 Implementación de operaciones

Para permitir un Borrado y Concatenado rápido, las raíces de todos los árboles están unidas una lista de tipo circular doblemente enlazada. Los hijos de cada nodo también están unidos usando una lista. Para cada nodo, guardamos el número de hijos y si está marcado. Además guardamos un puntero a la raíz que contiene la clave mínima.

La operación **Encontrar Mínimo** es trivial porque guardamos el puntero al nodo que lo contiene. Esto no cambia el Potencial del Montículo, ya que el coste actual y amortizado es constante. Tal como se indica arriba, la **Unión** se implementa simplemente concatenando las listas de raíces de árboles de los dos Heaps. Esto se puede hacer en tiempo constante y no cambia su Potencia, resultando otra vez un tiempo constante amortizado. La operación **Insertar** trabaja creando un nuevo montículo con un elemento y haciendo la Unión. Esto se hace en tiempo constante, y el Potencial se incrementa en 1, ya que el número de árboles aumenta. El tiempo amortizado es constante igualmente.

La operación **Extraer Mínimo** (lo mismo que *Borrar Mínimo*) opera en tres fases. Primero cogemos la raíz con el elemento mínimo y la borramos. Sus hijos se convertirán en raíces de nuevos árboles. Si el número de hijos era $d$, lleva un tiempo $O(d)$ procesar todas las nuevas raíces y el Potencial se incrementa en $d-1$. El tiempo de ejecución amortizado en esta fase es $O(d) = O(\log n)$.

Sin embargo, para completar la extracción del mínimo, necesitamos actualizar el puntero a la raíz con la clave mínima. El problema es que hay $n$ raíces que comprobar. En la segunda fase decrementamos el número de raíces agrupando sucesivamente las raíces del mismo grado. Cuando dos raíces $u$ y $v$ tienen el mismo grado, hacemos que una de ellas sea hija de la otra de manera que la que tenga la clave menor siga siendo la raíz. Su grado se incrementará en uno. Esto se repite hasta que todas las raíces tienen un grado diferente. Para encontrar árboles del mismo grado eficientemente usamos un vector de longitud $O(\log n)$ en el que guardamos un puntero a una raíz de cada grado. Cuando una segunda raíz con el mismo grado es encontrada, las dos se unen y se actualiza el vector. El tiempo de ejecución actual es $O(\log n + m)$ donde $m$ es el número de raíces al principio de la segunda fase. Al final tendremos como mucho $O(\log n)$ raíces (porque cada una tiene grado diferente). Así pues el Potencial se decrementa al menos $m$ - $O(\log n)$ y el tiempo de ejecución amortizado es $O(\log n)$.

En la tercera fase, comprobamos cada una de las raíces restantes y encontramos el mínimo. Esto cuesta $O(\log n)$ y el potencial no cambia. El tiempo medio de ejecución amortizado para extraer el mínimo es por consiguiente $O(\log n)$.

La operación **Decrementar Clave** cogerá el nodo, decrementará la clave y se viola la propiedad del montículo (la nueva clave es más pequeña que la clave del padre), el nodo se corta de su padre. Si el padre no es una raíz, se marca. Si ya estaba marcado, se corta también y su padre se marca. Continuamos subiendo hasta que, o bien alcanzamos la raíz o un vértice no marcado. En el proceso creamos un número $k$ de nuevos árboles. El Potencial se reduce en al menos $k - 2$. El tiempo para realizar el corte es $O(k)$ y el tiempo de ejecución amortizado es constante.

Por último, la operación **Borrar** puede ser implementada simplemente decrementando la clave del elemento a borrar a menos infinito, convirtiéndolo en el mínimo de todo el montículo, entonces llamamos a Extraer Mínimo para borrarlo. El tiempo de ejecución amortizado de esta operación es $O(\log n)$.

## 1.15 Peor Caso

Aunque el tiempo total de ejecución de una secuencia de operaciones que empiezan por una estructura vacía viene determinado por lo explicado anteriormente, algunas, aunque muy pocas, operaciones de la secuencia pueden llevar mucho tiempo(en particular Decrementar Clave, Borrar y Borrar Mínimo tienen tiempo de ejecución lineal en el peor caso). Por este motivo los montículos de Fibonacci y otras estructuras con costes amortizados pueden no ser apropiadas para sistemas de tiempo real.

## 1.16 Resumen de Tiempos de Ejecución

(*)Tiempo Amortizado

## 1.17 Referencias

1. Fredman M. L. & Tarjan R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. Journal of the ACM 34(3), 596-615.

2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Chapter 20: Fibonacci Heaps, pp.476–497.

3. Brodal, G. S. 1996. Worst-case efficient priority queues. In Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (Atlanta, Georgia, United States, January 28 - 30, 1996). Symposium on Discrete Algorithms. Society for Industrial and Applied Mathematics, Philadelphia, PA, 52-58.

## 1.18 Enlaces externos

1. Simulación en Java de un Heap de Fibonacci

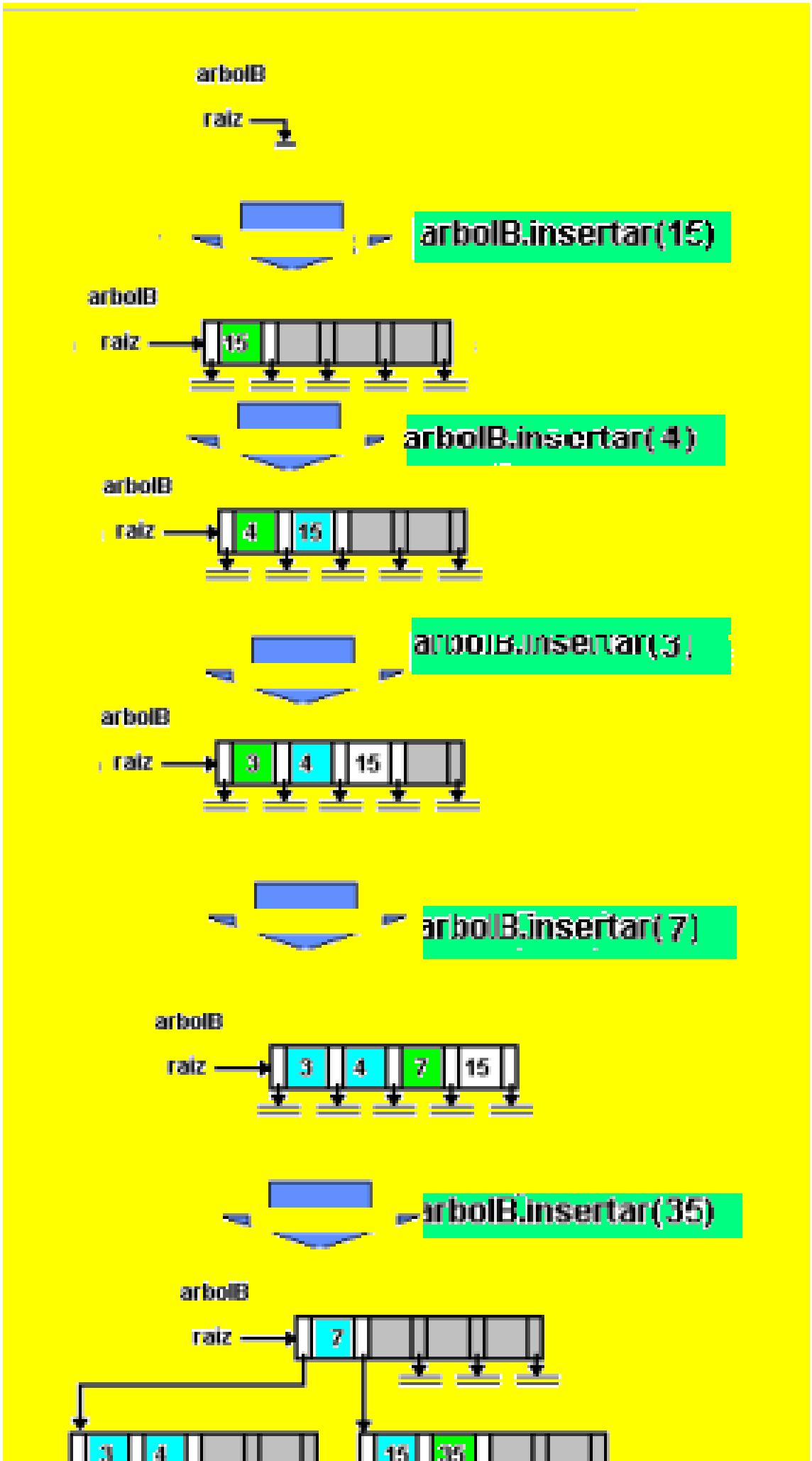
2. Implementación en C de un Heap de Fibonacci

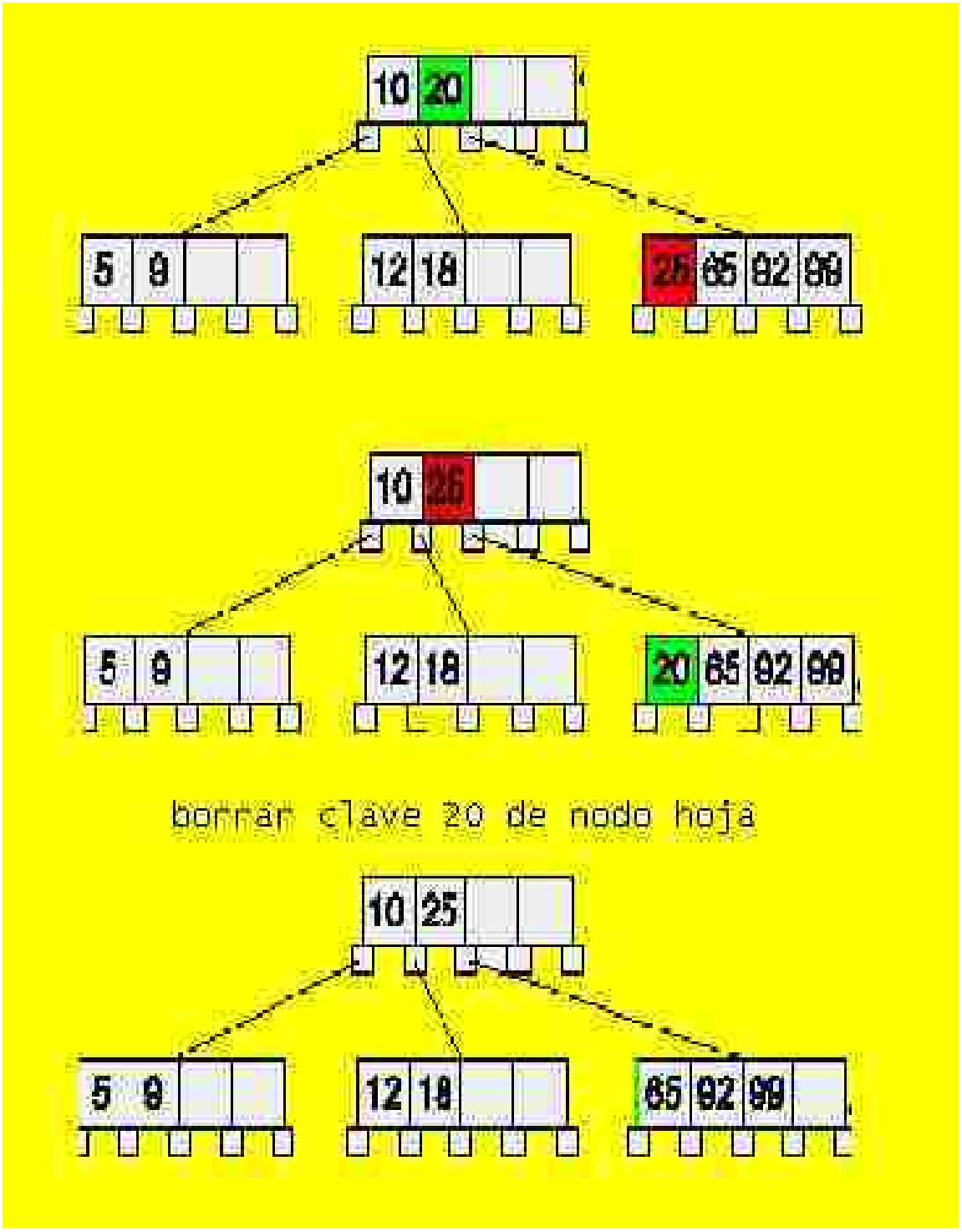
3. Algoritmo en pseudocódigo del montículo de Fibonacci

4. Pseudocódigo y explicación de J. Boyer (Dr. Dobb's Algorithm Alley)

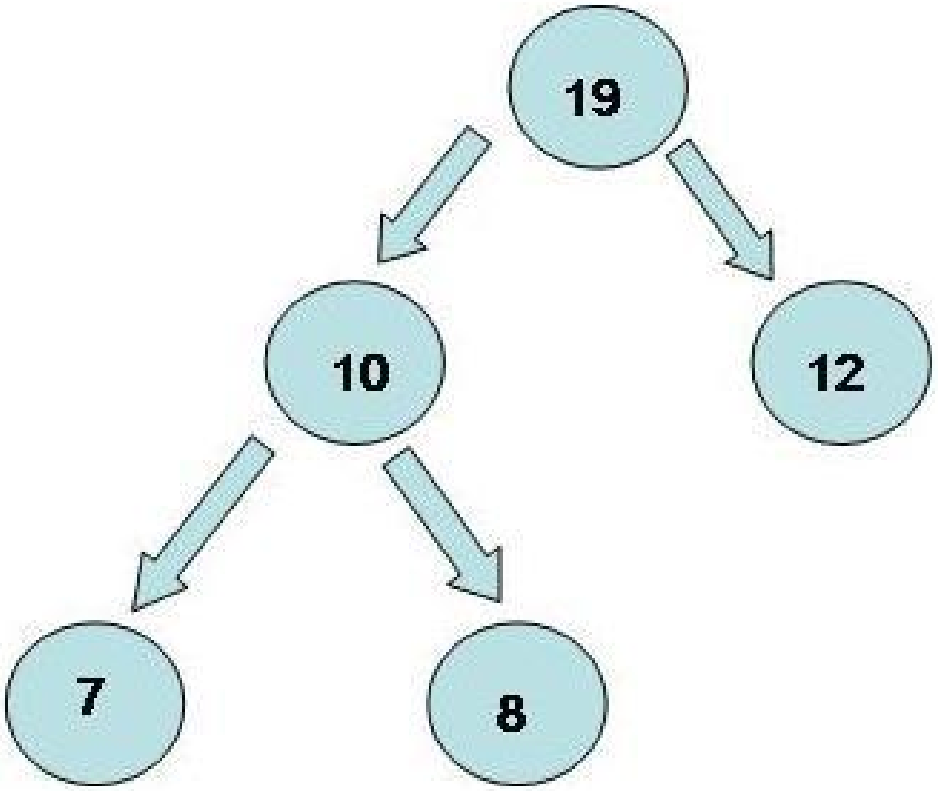
## 1.19 Véase también

1. Fibonacci (Leonardo de Pisa)

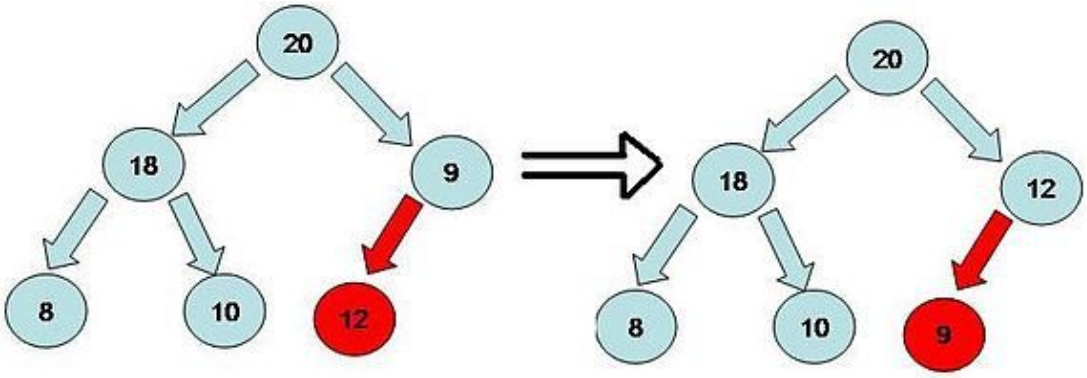

arbolB
raiz
arbolB.insertar(15)
arbolB
raiz
15
arbolB.insertar( 4 )
arbolB
raiz
4
15
arbolB.insertar( 3 )
arbolB
raiz
3
4
15
arbolB.insertar( 7 )
arbolB
raiz
3
4
7
15
arbolB.insertar(35)
arbolB
raiz
7




*eliminar clave 20 de un nodo interno.*




*Ejemplo de montículo de máximos.*




*Cómo se inserta un elemento en un montículo de máximos.*

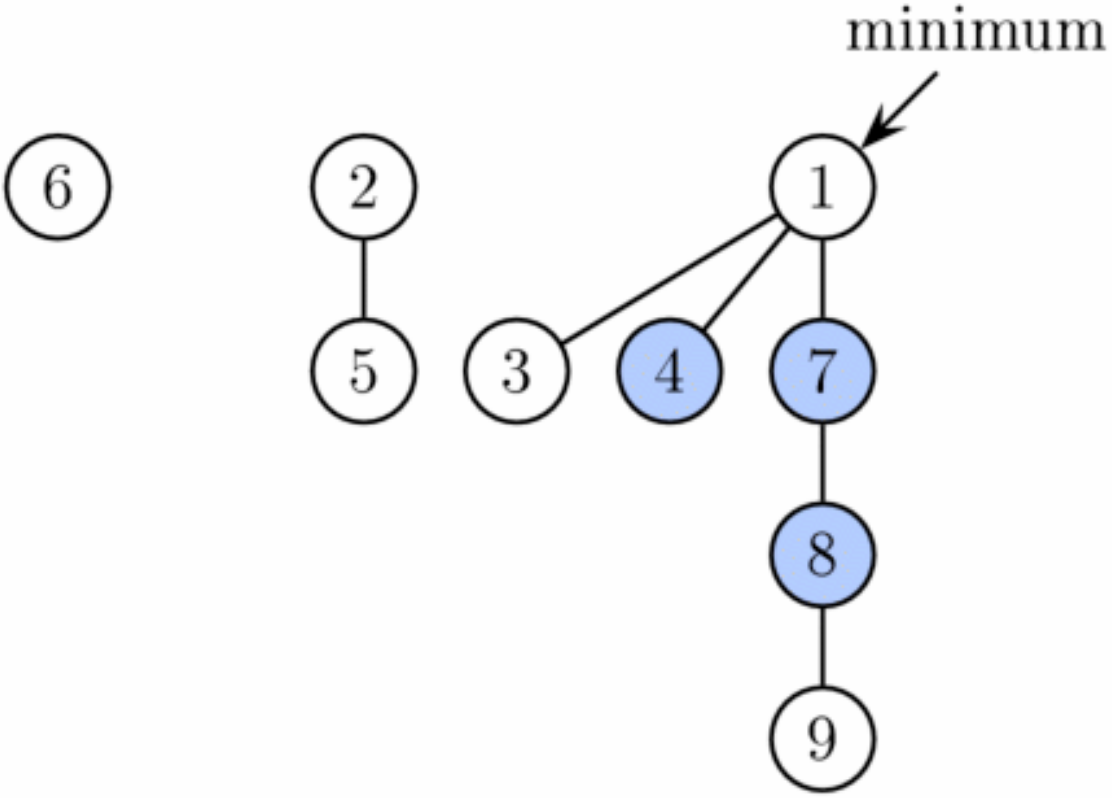
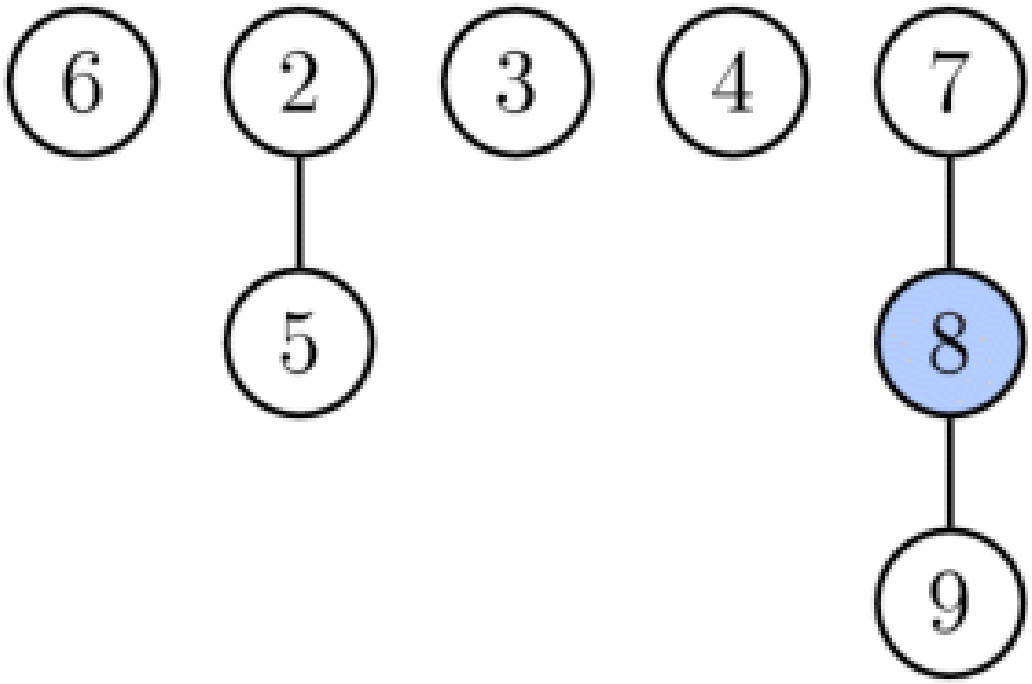
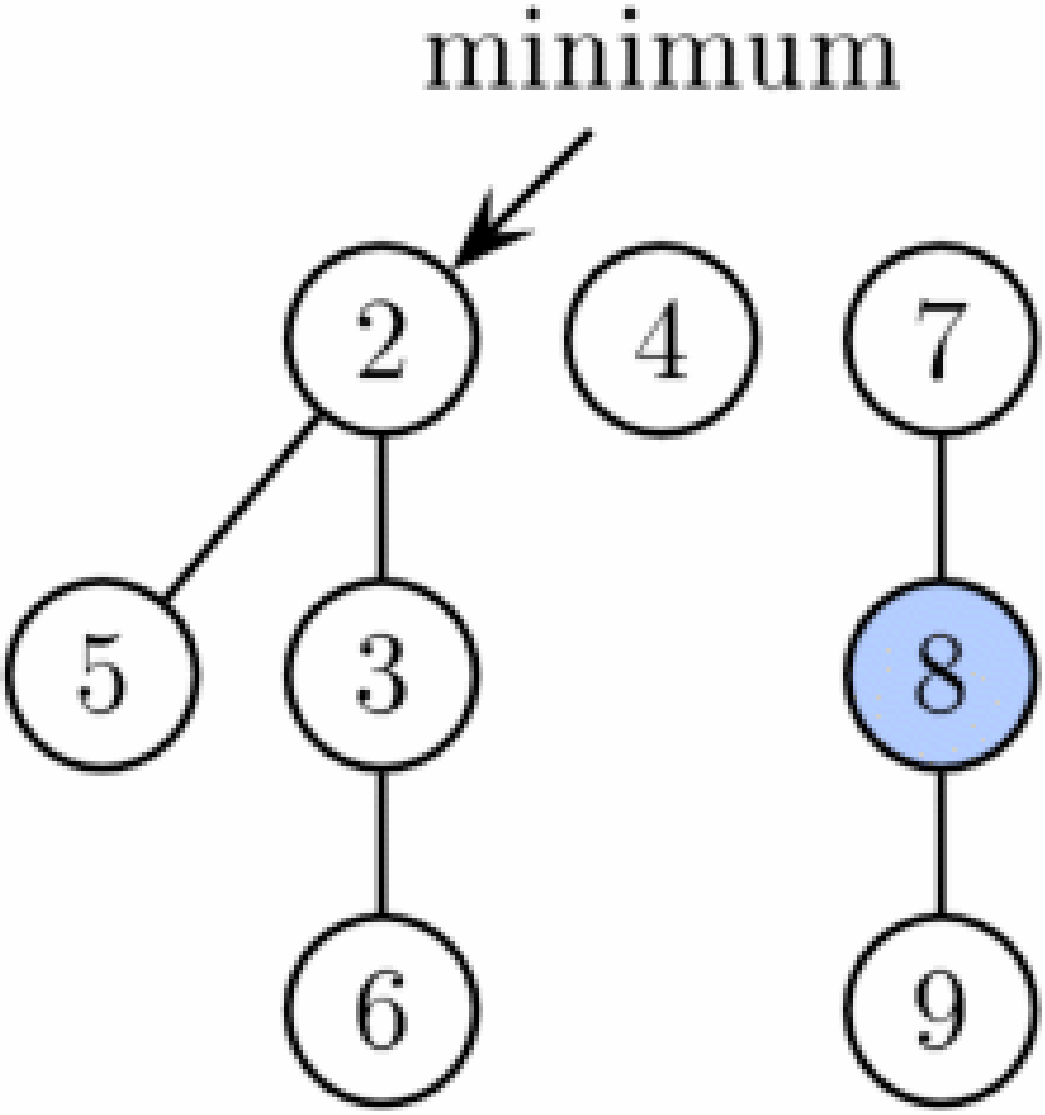


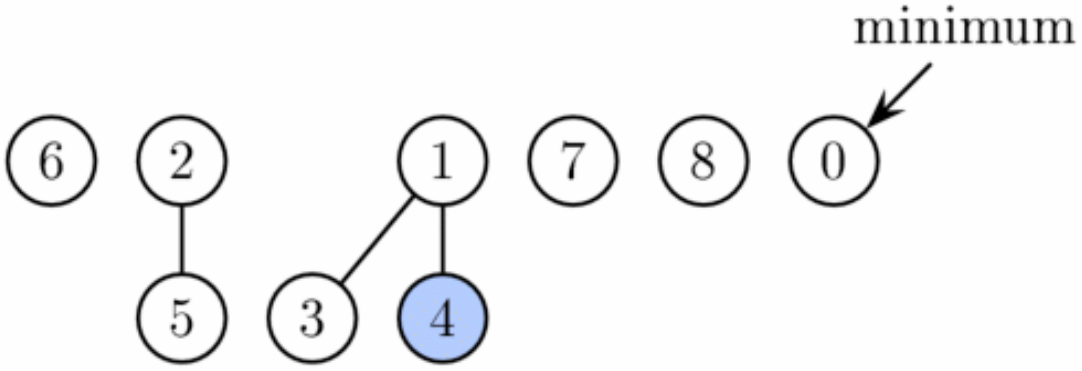

*Figura 1. Ejemplo de un montículo de Fibonacci. Tiene tres árboles de grados 0, 1 y 3. Tres vértices están marcados (mostrados en azul). Por lo tanto, el potencial de la pila es de 9.*




*El montículo de Fibonacci de la figura 1 después de la primera fase de extracción mínima. El nodo con clave 1 (el mínimo) se ha eliminado y sus hijos han formado árboles por separado.*




*El montículo de Fibonacci de la figura 1 después de extraer el mínimo. En primer lugar, los nodos 3 y 6 están unidos entre sí. Entonces el resultado se vincula con árboles arraigados en el nodo 2. Por último, el nuevo mínimo se encuentra.*

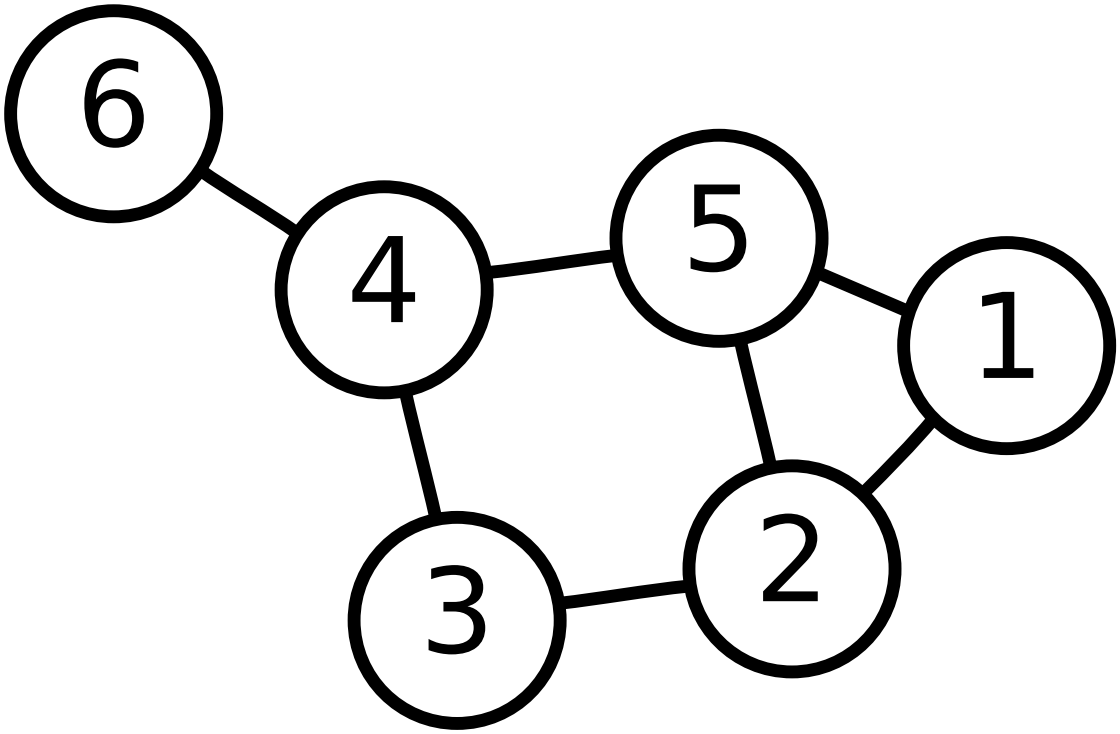



*El montículo de Fibonacci de la figura 1 después de la disminución de los principales nodos de 9 a 0. Este nodo, así como sus dos antecesores marcados se cortan del árbol enraizado en el 1 y se colocan como nuevas raíces.*

# Capítulo 2

# Gráficas

## 2.1 Grafo




*Grafo etiquetado con 6 vértices y 7 aristas.*

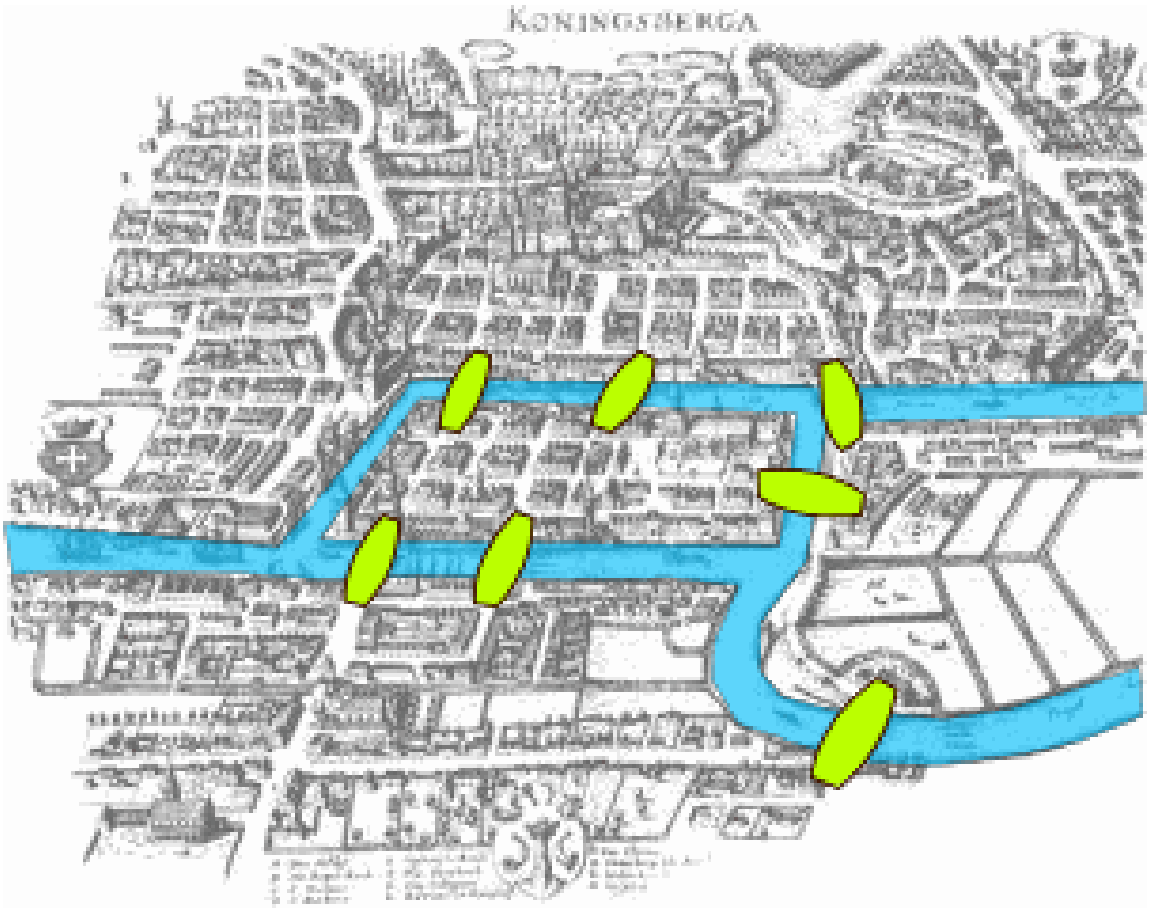
En matemáticas y ciencias de la computación, un **grafo** (del griego *grafos*: dibujo, imagen) es un conjunto de objetos llamados vértices o nodos unidos por enlaces llamados aristas o arcos, que permiten representar relaciones binarias entre elementos de un conjunto. Son objeto de estudio de la teoría de grafos.

Típicamente, un grafo se representa gráficamente como un conjunto de puntos (vértices o nodos) unidos por líneas (aristas).

Desde un punto de vista práctico, los grafos permiten estudiar las interrelaciones entre unidades que interactúan unas con otras. Por ejemplo, una red de computadoras puede representarse y estudiarse mediante un grafo, en el cual los vértices representan terminales y las aristas representan conexiones (las cuales, a su vez, pueden ser cables o conexiones inalámbricas).

Prácticamente cualquier problema puede representarse mediante un grafo, y su estudio trasciende a las diversas áreas de las ciencias exactas y las ciencias sociales.

### 2.1.1 Historia y problema de los puentes de Königsberg




*Los siete puentes de Königsberg.*

El primer artículo científico relativo a grafos fue escrito por el matemático suizo Leonhard Euler en 1736. Euler se basó en su artículo en el *problema de los puentes de Königsberg*. La ciudad de Kaliningrado, originalmente *Königsberg*, es famosa por sus siete puentes que unen ambas márgenes del río Pregel con dos de sus islas. Dos de los puentes unen la isla mayor con la margen oriental y otros dos con la margen occidental. La isla menor está conectada a cada margen por un puente y el séptimo puente une ambas islas. El problema planteaba lo siguiente: ¿es posible dar un paseo comenzando desde cualquiera de estas regiones, pasando por todos los puentes, recorriendo solo una vez cada uno y regresando al mismo punto de partida?

Abstrayendo este problema y planteándolo con la (entonces aún básica) teoría de grafos, Euler consigue demostrar que el grafo asociado al esquema de puentes de Königsberg no tiene solución, es decir, no es posible regresar al vértice de partida sin pasar por alguna arista dos veces.

De hecho, Euler resuelve el problema más general: ¿qué condiciones debe satisfacer un grafo para garantizar que se puede regresar al vértice de partida sin pasar por la misma arista más de una vez? Si definimos como «grado» al número de líneas que se encuentran en un punto de un grafo, entonces la respuesta al problema es que *los puentes de un pueblo se pueden atravesar exactamente una vez si, salvo a lo sumo dos, todos los puntos tienen un grado par*.

### 2.1.2 Definiciones

Un **grafo** $G$ es un par ordenado $G = (V, E)$ , donde:

- $V$ es un conjunto de vértices o nodos, y
- $E$ es un conjunto de aristas o arcos, que relacionan estos nodos.

Normalmente $V$ suele ser finito. Muchos resultados importantes sobre grafos no son aplicables para *grafos infinitos*.

Se llama **orden** del grafo $G$ a su número de vértices, $|V|$ .

El grado de un vértice o nodo $v \in V$ es igual al número de arcos que lo tienen como extremo.

Un bucle es una arista que relaciona al mismo nodo; es decir, una arista donde el nodo inicial y el nodo final coinciden.

Dos o más aristas son paralelas si relacionan el mismo par de vértices.

### Grafo no dirigido



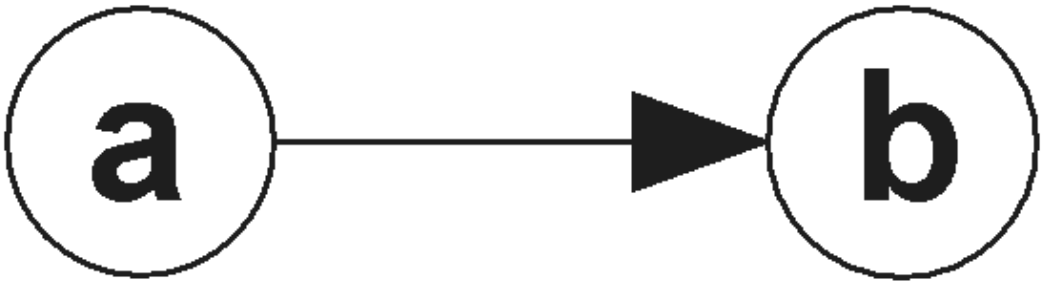

*Grafo no dirigido*

Un **grafo no dirigido** o **grafo propiamente dicho** es un grafo $G = (V, E)$ donde:

- $V \neq \emptyset$
- $E \subseteq \{x \in \mathcal{P}(V) : |x| = 2\}$ es un conjunto de *pares no ordenados* de elementos de $V$ .

Un **par no ordenado** es un conjunto de la forma $\{a, b\}$ , de manera que $\{a, b\} = \{b, a\}$ . Para los grafos, estos conjuntos pertenecen al conjunto potencia de $V$ , denotado $\mathcal{P}(V)$ , y son de cardinalidad 2.

### Grafo dirigido




*Grafo dirigido*

Un **grafo dirigido** o **digrafo** es un grafo $G = (V, E)$ donde:

- $V \neq \emptyset$
- $E \subseteq \{(a, b) \in V \times V : a \neq b\}$ es un conjunto de pares ordenados de elementos de $V$ .

Dada una arista $(a, b)$ , $a$ es su *nodo inicial* y $b$ su *nodo final*.

Por definición, los grafos dirigidos no contienen *bucles*.

Un **grafo mixto** es aquel que se define con la capacidad de poder contener aristas dirigidas y no dirigidas. Tanto los grafos dirigidos como los no dirigidos son casos particulares de este.

#### Variantes sobre las definiciones principales

Algunas aplicaciones requieren extensiones más generales a las dos propuestas clásicas de grafos. Aunque la definición original los permite, según la aplicación concreta pueden ser válidos o no. A veces $V$ o $E$ pueden ser un multiconjunto, pudiendo haber más de una arista entre cada par de vértices. La palabra *grafo* (a secas) puede permitir o no múltiples aristas entre cada par de vértices, dependiendo del autor de la referencia consultada. Si se quiere remarcar la inexistencia de múltiples aristas entre cada par de vértices (y en el caso no dirigido, excluir bucles) el grafo puede llamarse **simple**. Por otra parte, si se quiere asegurar la posibilidad de permitir múltiples aristas, el grafo puede llamarse **multigrafo** (a veces se utiliza el término **pseudografo** para indicar que se permiten tanto bucles como múltiples aristas entre cada par de vértices).

### 2.1.3 Propiedades

- **Adyacencia:** dos aristas son *adyacentes* si tienen un vértice en común, y dos vértices son **adyacentes** si una arista los une.
- **Incidencia:** una arista es *incidente* a un vértice si ésta lo une a otro.
- **Ponderación:** corresponde a una función que a cada arista le asocia un valor (costo, peso, longitud, etc.), para aumentar la expresividad del modelo. Esto se usa mucho para problemas de optimización, como el del vendedor viajero o del camino más corto.
- **Etiquetado:** distinción que se hace a los vértices y/o aristas mediante una marca que los hace unívocamente distinguibles del resto.

### 2.1.4 Ejemplos

La imagen es una representación del siguiente grafo:

- $V$:={1,2,3,4,5,6}
- $E$:={{1,2},{1,5},{2,3},{2,5},{3,4},{4,5},{4,6}}

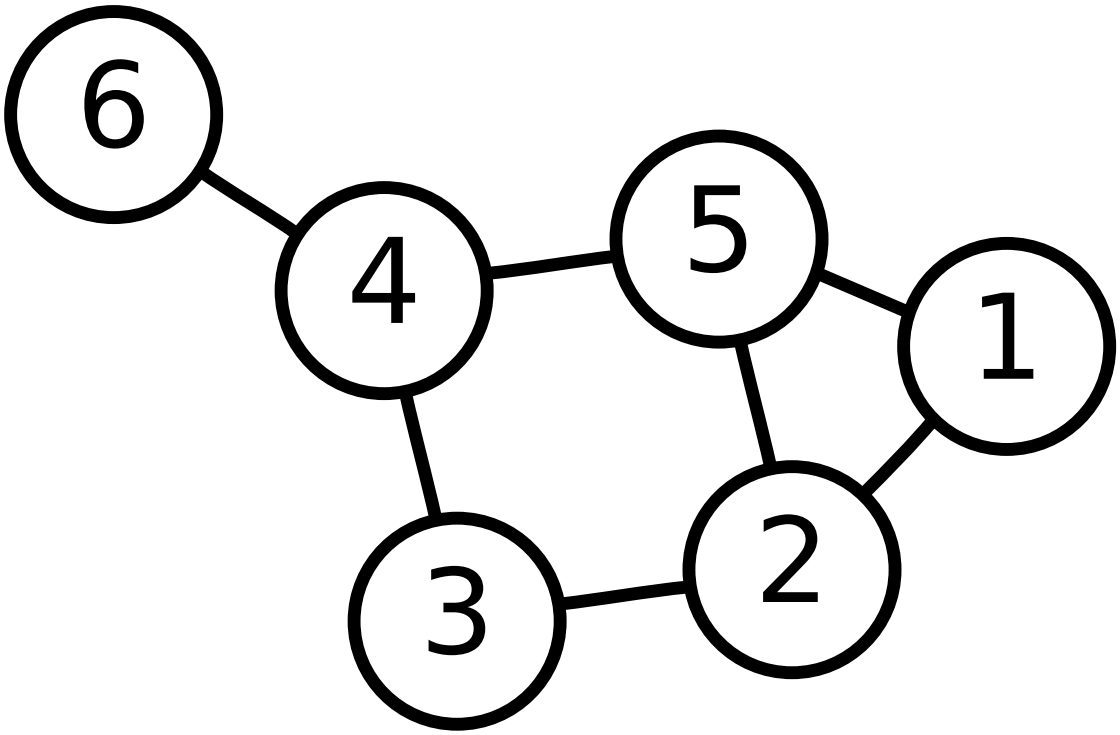
El hecho que el vértice 1 sea adyacente con el vértice 2 puede ser denotado como 1 ~ 2.

- En la teoría de las categorías una **categoría** puede ser considerada como un **multigrafo** dirigido, con los objetos como vértices y los morfismos como aristas dirigidas.
- En ciencias de la computación los grafos dirigidos son usados para representar máquinas de estado finito y algunas otras estructuras discretas.
- Una relación binaria $R$ en un conjunto $X$ es un grafo dirigido simple. Dos vértices $a$, $b$ en $X$ están conectados por una arista dirigida $ab$ si $aRb$.

### 2.1.5 Grafos particulares

Existen grafos que poseen propiedades destacables. Algunos ejemplos básicos son:

- Grafo nulo: aquel que no tiene vértices ni aristas. Nótese que algunas personas exigen que el conjunto de vértices no sea vacío en la definición de grafo.




- Grafo vacío: aquel que no tiene aristas.
- Grafo trivial: aquel que tiene un vértice y ninguna arista.
- *Grafo simple*: aquel que no posee bucles ni artistas paralelas. Consultar variantes en esta definición.
- Multigrafo (o pseudografo): G es multigrafo si y solo si no es simple. Consultar variantes en esta definición.
- Grafo completo: grafo simple en el que cada par de vértices están unidos por una arista, es decir, contiene todas las posibles aristas.
- Grafo bipartito: sea $(W, X)$ una partición del conjunto de vértices $V$ , es aquel donde cada arista tiene un vértice en $W$ y otro en $X$ .
- Grafo bipartito completo: sea $(W, X)$ una partición del conjunto de vértices $V$ , es aquel donde cada vértice en $W$ es adyacente *sólo* a cada vértice en $X$ , y viceversa.
- Grafo plano: aquel que puede ser dibujado en el plano cartesiano sin cruce de aristas.
- Árbol: grafo conexo sin ciclos.
- Grafo rueda: grafo con $n$ vértices que se forma conectando un único vértice a todos los vértices de un ciclo-$(n-1)$.
- Grafo perfecto aquel que el número cromático de cada subgrafo inducido es igual al tamaño del mayor clique de ese subgrafo.

Una generalización de los grafos son los llamados hipergrafos.

### 2.1.6 Véase también

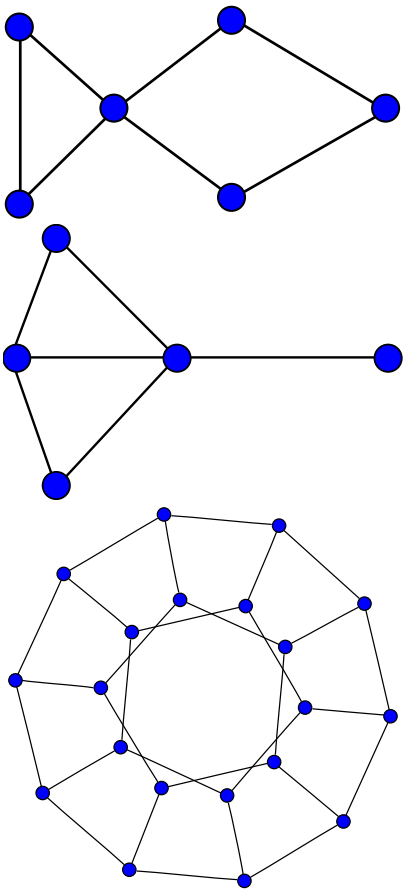
- Portal:Matemática. Contenido relacionado con **Matemática**.
- Portal:Geometría. Contenido relacionado con **Geometría**.

- Grafo social
- Teoría de grafos

### 2.1.7 Enlaces externos

- Wikimedia Commons alberga contenido multimedia sobre **Grafo**. Commons

## 2.2 Teoría de grafos

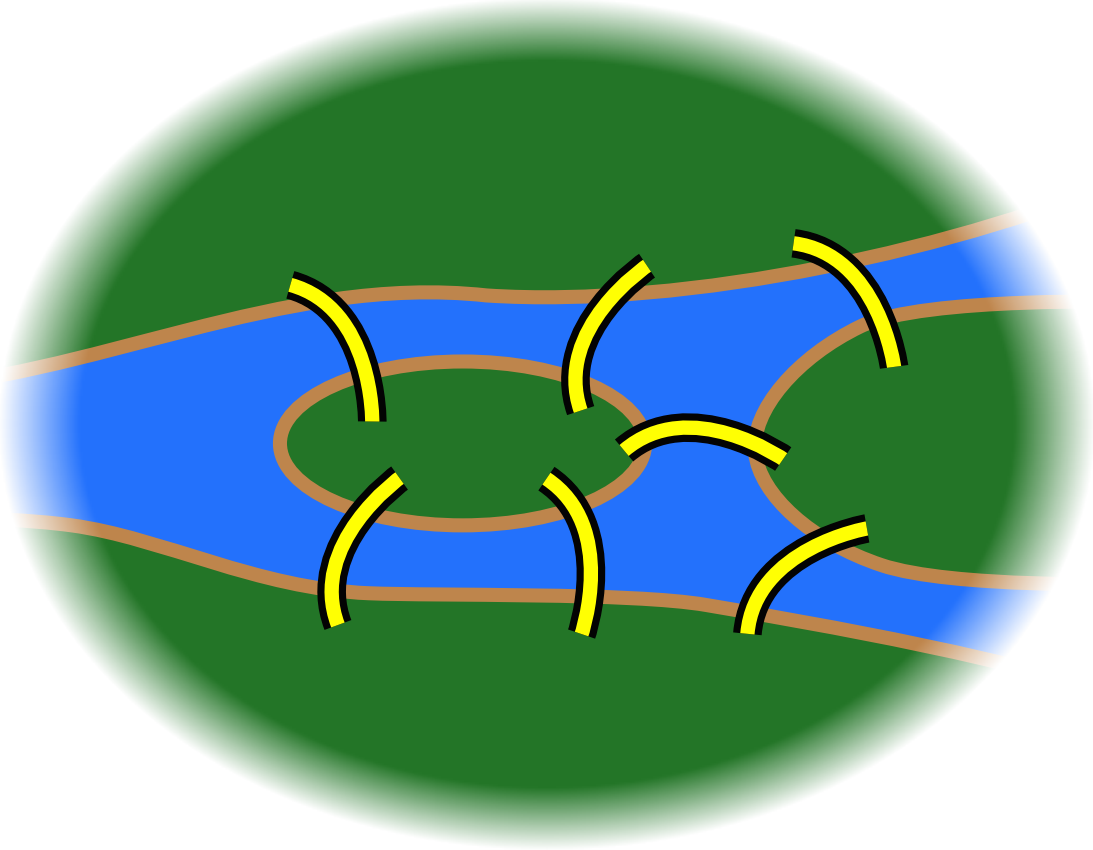


Los *grafos* son el objeto de estudio de esta rama de las matemáticas. Arriba el grafo pez, en medio el grafo arco y abajo el grafo dodecaedro.

La **teoría de grafos** (también llamada **teoría de las gráficas**) es un campo de estudio de las matemáticas y las ciencias de la computación, que estudia las propiedades de los grafos (también llamadas *gráficas*, que no se debe confundir con las gráficas que tienen una acepción muy amplia) estructuras que constan de dos partes, el conjunto de vértices, nodos o puntos; y el conjunto de aristas, líneas o lados (*edges* en inglés) que pueden ser orientados o no.

La teoría de grafos es una rama de las matemáticas discretas y de las matemáticas aplicadas, y es un tratado que usa diferentes conceptos de diversas áreas como combinatoria, álgebra, probabilidad, geometría de polígonos, aritmética y topología.

Actualmente ha tenido mayor preponderancia en el campo de la informática, las ciencias de la computación y telecomunicaciones.



*Los 7 puentes del río Pregel en Königsberg.*

### 2.2.1 Historia

El origen de la teoría de grafos se remonta al siglo XVIII con el problema de los puentes de Königsberg, el cual consistía en encontrar un camino que recorriera los siete puentes del río Pregel (54°42′12″N 20°30′56″E / 54.70333, 20.51556) en la ciudad de Königsberg, actualmente Kaliningrado, de modo que se recorrieran todos los puentes pasando una sola vez por cada uno de ellos. El trabajo de Leonhard Euler sobre el problema titulado *Solutio problematis ad geometriam situs pertinentis*[1] (*La solución de un problema relativo a la geometría de la posición*) en 1736, es considerado el primer resultado de la teoría de grafos. También se considera uno de los primeros resultados topológicos en geometría (que no depende de ninguna medida). Este ejemplo ilustra la profunda relación entre la teoría de grafos y la topología.

Luego, en 1847, Gustav Kirchhoff utilizó la teoría de grafos para el análisis de redes eléctricas publicando sus leyes de los circuitos para calcular el voltaje y la corriente en los circuitos eléctricos, conocidas como leyes de Kirchhoff, considerado la primera aplicación de la teoría de grafos a un problema de ingeniería.

En 1852 Francis Guthrie planteó el problema de los cuatro colores el cual afirma que es posible, utilizando solamente cuatro colores, colorear cualquier mapa de países de tal forma que dos países vecinos nunca tengan el mismo color. Este problema, que no fue resuelto hasta un siglo después por Kenneth Appel y Wolfgang Haken en 1976, puede ser considerado como el nacimiento de la teoría de grafos. Al tratar de resolverlo, los matemáticos definieron términos y conceptos teóricos fundamentales de los grafos.

En 1857, Arthur Cayley estudió y resolvió el problema de enumeración de los isómeros, compuestos químicos con idéntica composición (fórmula) pero diferente estructura molecular. Para ello representó cada compuesto, en este caso hidrocarburos saturados $C_nH_{2n+2}$, mediante un grafo árbol donde los vértices representan átomos y las aristas la existencia de enlaces químicos.

El término *«grafo»*, proviene de la expresión *H«graphic notation»* usada por primera vez por Edward Frankland[2] y posteriormente adoptada por Alexander Crum Brown en 1884, y hacía referencia a la representación gráfica de los

enlaces entre los átomos de una molécula.

El primer libro sobre teoría de grafos fue escrito por Dénes Kőnig y publicado en 1936.[3]

### 2.2.2 Aplicaciones

Gracias a la teoría de grafos se pueden resolver diversos problemas como por ejemplo la síntesis de circuitos secuenciales, contadores o sistemas de apertura. Se utiliza para diferentes áreas por ejemplo, Dibujo computacional, en toda las áreas de Ingeniería.

Los grafos se utilizan también para modelar trayectos como el de una línea de autobús a través de las calles de una ciudad, en el que podemos obtener caminos óptimos para el trayecto aplicando diversos algoritmos como puede ser el algoritmo de Floyd.

Para la administración de proyectos, utilizamos técnicas como técnica de revisión y evaluación de programas (PERT) en las que se modelan los mismos utilizando grafos y optimizando los tiempos para concretar los mismos.

La teoría de grafos también ha servido de inspiración para las ciencias sociales, en especial para desarrollar un concepto no metafórico de red social que sustituye los nodos por los actores sociales y verifica la posición, centralidad e importancia de cada actor dentro de la red. Esta medida permite cuantificar y abstraer relaciones complejas, de manera que la estructura social puede representarse gráficamente. Por ejemplo, una red social puede representar la estructura de poder dentro de una sociedad al identificar los vínculos (aristas), su dirección e intensidad y da idea de la manera en que el poder se transmite y a quiénes.

Se emplea en problemas de control de producción, para proyectar redes de ordenadores, para diseñar módulos electrónicos modernos y proyectar sistemas físicos con parámetros localizados (mecánicos, acústicos y eléctricos).

Se usa para la solución de problemas de genética y problemas de automatización de la proyección (SAPR). Apoyo matemático de los sistemas modernos para el procesamiento de la información. Acude en las investigaciones nucleares (técnica de diagramas de Feynman).[4]

Los grafos son importantes en el estudio de la biología y hábitat. El vértice representa un hábitat y las aristas (o "edges" en inglés) representa los senderos de los animales o las migraciones. Con esta información, los científicos pueden entender cómo esto puede cambiar o afectar a las especies en su hábitat.

- Mapas conceptuales
- Plano de estaciones del metro.
- Plano de autopistas.
- Circuito eléctrico
- Sociograma de una red social
- Topología de red de computadores
- Organigramas
- Isomeros
- Arquitectura de redes de telefonía móvil
- Draws de eliminación directa (ej: tenis)

### 2.2.3 Tipos de grafos

- **Grafo simple.** o simplemente *grafo* es aquel que acepta una sola arista uniendo dos vértices cualesquiera. Esto es equivalente a decir que una arista cualquiera es la única que une dos vértices específicos. Es la definición estándar de un grafo.
- **Multigrafo**. o *pseudografo* son grafos que aceptan más de una arista entre dos vértices. Estas aristas se llaman *múltiples* o *lazos* (*loops* en inglés). Los *grafos simples* son una subclase de esta categoría de grafos. También se les llama *grafos no-dirigido*.

- **Grafo dirigido**. Son grafos en los cuales se ha añadido una *orientación* a las aristas, representada gráficamente por una flecha
- **Grafo etiquetado**. Grafos en los cuales se ha añadido un *peso* a las aristas (número entero generalmente) o un *etiquetado* a los vértices.
- **Grafo aleatorio**. Grafo cuyas aristas están asociadas a una *probabilidad*.
- **Hipergrafo**. Grafos en los cuales las aristas tienen más de dos extremos, es decir, las aristas son incidentes a 3 o más vértices.
- **Grafo infinito**. Grafos con conjunto de vértices y aristas de *cardinal infinito*.

### 2.2.4 Representación de grafos

Existen diferentes formas de representar un grafo (simple), además de la geométrica y muchos métodos para almacenarlos en una computadora. La estructura de datos usada depende de las características del grafo y el algoritmo usado para manipularlo. Entre las estructuras más sencillas y usadas se encuentran las listas y las matrices, aunque frecuentemente se usa una combinación de ambas. Las listas son preferidas en grafos dispersos porque tienen un eficiente uso de la memoria. Por otro lado, las matrices proveen acceso rápido, pero pueden consumir grandes cantidades de memoria.

#### Estructura de lista

- **lista de incidencia** - Las aristas son representadas con un vector de pares (ordenados, si el grafo es dirigido), donde cada par representa una de las aristas.[5]
- **lista de adyacencia** - Cada vértice tiene una lista de vértices los cuales son adyacentes a él. Esto causa redundancia en un grafo no dirigido (ya que A existe en la lista de adyacencia de B y viceversa), pero las búsquedas son más rápidas, al costo de almacenamiento extra.
- **lista de grados** - También llamada *secuencia de grados* o *sucesión gráfica* de un grafo no-dirigido es una secuencia de números, que corresponde a los grados de los vértices del grafo.

#### Estructuras matriciales

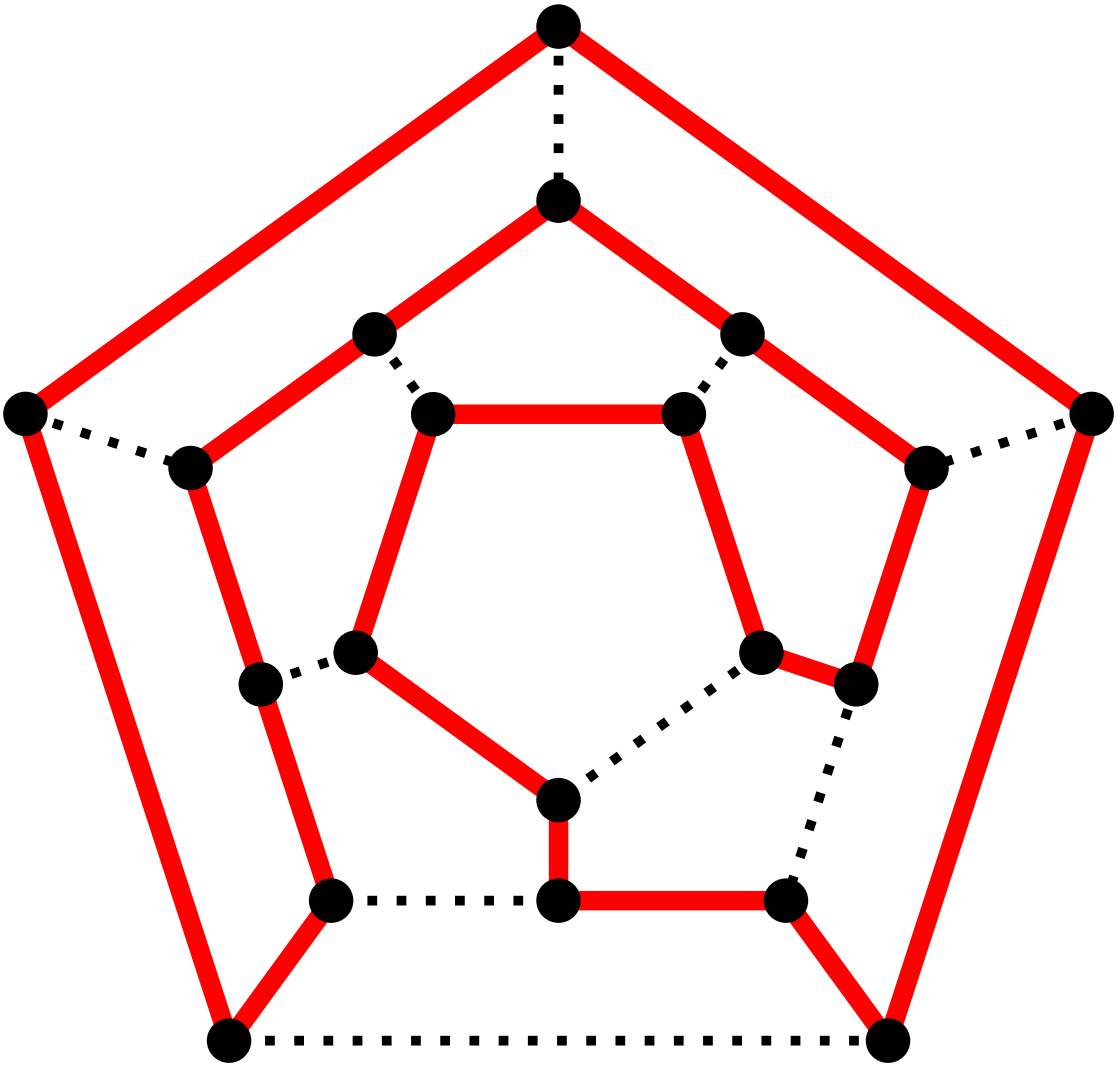
- **Matriz de adyacencia** - El grafo está representado por una matriz cuadrada M de tamaño $n^2$ , donde $n$ es el número de vértices. Si hay una arista entre un vértice x y un vértice y, entonces el elemento $m_{x,y}$ es 1, de lo contrario, es 0.
- **Matriz de incidencia** - El grafo está representado por una matriz de A (aristas) por V (vértices), donde [vértice, arista] contiene la información de la arista (1 - conectado, 0 - no conectado)

### 2.2.5 Problemas de teoría de grafos

#### Ciclos y caminos hamiltonianos

Un **ciclo** es una sucesión de aristas adyacentes, donde no se recorre dos veces la misma arista, y donde se regresa al punto inicial. Un **ciclo hamiltoniano** tiene además que recorrer todos los vértices exactamente una vez (excepto el vértice del que parte y al cual llega).

Por ejemplo, en un museo grande (al estilo del Louvre), lo idóneo sería recorrer todas las salas una sola vez, esto es buscar un ciclo hamiltoniano en el grafo que representa el museo (los vértices son las salas, y las aristas los corredores o puertas entre ellas).



*Ejemplo de un ciclo Hamiltoniano.*

Se habla también de Camino hamiltoniano si no se impone regresar al punto de partida, como en un museo con una única puerta de entrada. Por ejemplo, un caballo puede recorrer todas las casillas de un tablero de ajedrez sin pasar dos veces por la misma: es un camino hamiltoniano. Ejemplo de un ciclo hamiltoniano en el grafo del dodecaedro.

Hoy en día, no se conocen métodos generales para hallar un ciclo hamiltoniano en tiempo polinómico, siendo la búsqueda por fuerza bruta de todos los posibles caminos u otros métodos excesivamente costosos. Existen, sin embargo, métodos para descartar la existencia de ciclos o caminos hamiltonianos en grafos pequeños.

El problema de determinar la existencia de ciclos hamiltonianos, entra en el conjunto de los NP-completos.

### Grafos planos

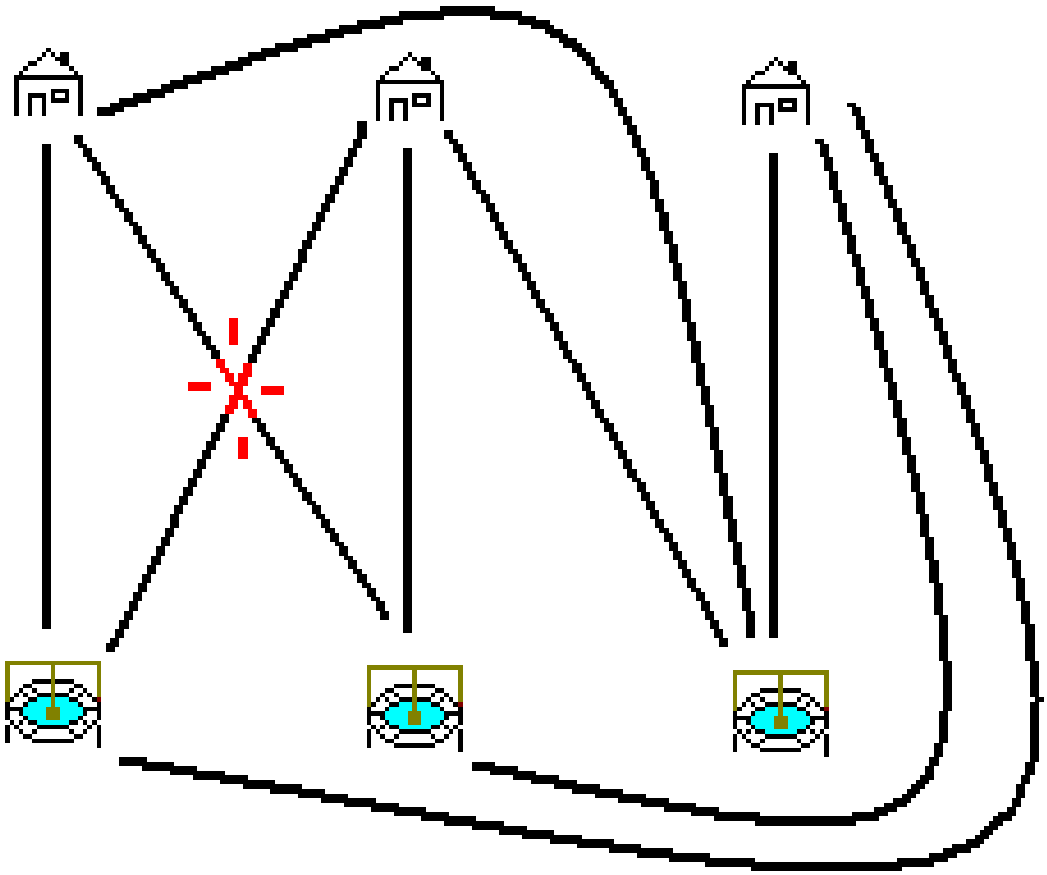
Cuando un grafo o multigrafo se puede dibujar en un plano sin que dos segmentos se corten, se dice que es plano.

Un juego muy conocido es el siguiente: Se dibujan tres casas y tres pozos. Todos los vecinos de las casas tienen el derecho de utilizar los tres pozos. Como no se llevan bien en absoluto, no quieren cruzarse jamás. ¿Es posible trazar los nueve caminos que juntan las tres casas con los tres pozos sin que haya cruces?

Cualquier disposición de las casas, los pozos y los caminos implica la presencia de al menos un cruce.

Sea $K_n$ el grafo completo con $n$ vértices, $K_{n,\,p}$ es el grafo bipartito de $n$ y $p$ vértices.

El juego anterior equivale a descubrir si el grafo bipartito completo $K_{3,3}$ es **plano**, es decir, si se puede dibujar en un



*Un grafo es **plano** si se puede dibujar sin cruces de aristas. El* problema de las tres casas y los tres pozos *tiene solución sobre el toro, pero no en el plano.*

plano sin que haya cruces, siendo la respuesta que no. En general, puede determinarse que un grafo *no* es plano, si en su diseño puede encontrase una estructura análoga (conocida como *menor*) a $K_5$ o a $K_{3,3}$.

Establecer qué grafos son planos no es obvio, y es un problema que tiene que ver con topología.

### Coloración de grafos

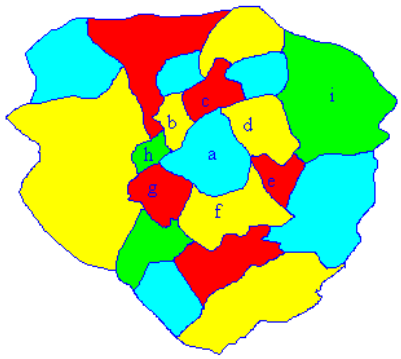
Si G=(V, E) es un grafo no dirigido, una coloración propia de G, ocurre cuando coloreamos los vértices de G de modo que si {a, b} es una arista en G entonces a y b tienen diferentes colores. (Por lo tanto, los vértices adyacentes tienen colores diferentes). El número mínimo de colores necesarios para una coloración propia de G es el número cromático de G y se escribe como C (G). Sea G un grafo no dirigido sea λ el número de colores disponibles para la coloración propia de los vértices de G. Nuestro objetivo es encontrar una función polinomial P (G,λ), en la variable λ, llamada polinomio cromático de G, que nos indique el número de coloraciones propias diferentes de los vértices de G, usando un máximo de λ colores.

Descomposición de polinomios cromáticos. Si G=(V, E) es un grafo conexo y e pertenece a E, entonces: P (G,λ)=P (G+e,λ)+P (G/e,λ), donde G/e es el grafo se obtene por contracción de aristas.

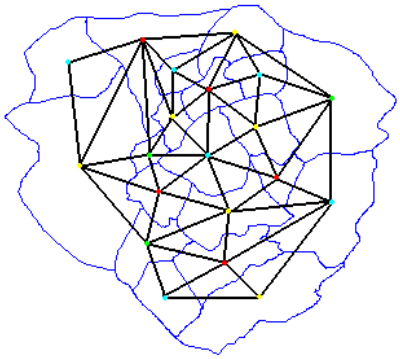
Para cualquier grafo G, el término constante en P (G,λ) es 0

Sea G=(V, E) con |E|>0 entonces, la suma de los coeficientes de P (G,λ) es 0.

Sea G=(V, E), con *a*, *b* pertenecientes al conjunto de vértices *V* pero {a, b}=e, no perteneciente a al conjunto de aristas E. Escribimos G+e para el grafo que se obtiene de G al añadir la arista e={a, b}. Al identificar los vértices a y b en G, obtenemos el subgrafo G++e de G.0000




**Teorema de los cuatro colores**
Mapa coloreado con 4-colores.



Grafo dual asociado al mapa con una 4-vértice coloración.

Otro problema famoso relativo a los grafos: ¿Cuántos colores son necesarios para dibujar un mapa político, con la condición obvia que dos países adyacentes no puedan tener el mismo color? Se supone que los países son de un solo pedazo, y que el mundo es esférico o plano. En un mundo en forma de toroide; el teorema siguiente no es válido:

Cuatro colores son siempre suficientes para colorear un mapa.

El mapa siguiente muestra que tres colores no bastan: Si se empieza por el país central **a** y se esfuerza uno en utilizar el menor número de colores, entonces en la corona alrededor de **a** alternan dos colores. Llegando al país **h** se tiene que introducir un cuarto color. Lo mismo sucede en **i** si se emplea el mismo método.

La forma precisa de cada país no importa; lo único relevante es saber qué país toca a qué otro. Estos datos están incluidos en el grafo donde los vértices son los países y las aristas conectan los que justamente son adyacentes. Entonces la cuestión equivale a atribuir a cada vértice un color distinto del de sus vecinos.

Hemos visto que tres colores no son suficientes, y demostrar que con cinco siempre se llega, es bastante fácil. Pero el teorema de los cuatro colores no es nada obvio. Prueba de ello es que se han tenido que emplear ordenadores para acabar la demostración (se ha hecho un programa que permitió verificar una multitud de casos, lo que ahorró muchísimo tiempo a los matemáticos). Fue la primera vez que la comunidad matemática aceptó una demostración asistida por ordenador, lo que creó en su día una cierta polémica dentro de la comunidad matemática.

### 2.2.6 Caracterización de grafos

**Grafos simples** Un grafo es *simple* si a lo más existe una arista uniendo dos vértices cualesquiera. Esto es equivalente a decir que una arista cualquiera es la única que une dos vértices específicos.

Un grafo que no es simple se denomina multigrafo.

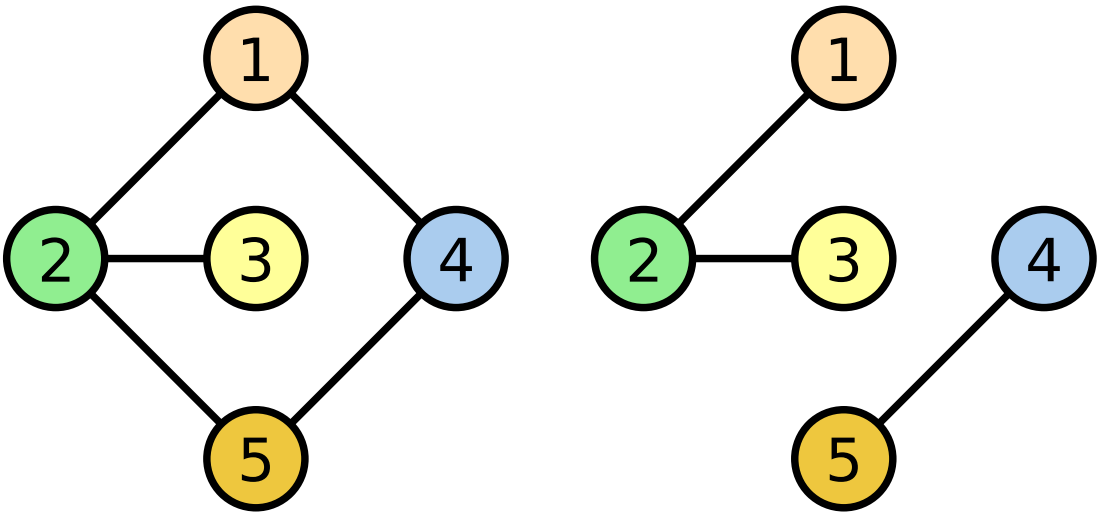
**Grafos conexos** Un grafo es conexo si cada par de vértices está conectado por un camino; es decir, si para cualquier par de vértices (a, b), existe al menos un camino posible desde *a* hacia *b*.

Un grafo es doblemente conexo si cada par de vértices está conectado por al menos dos caminos disjuntos; es decir, es conexo y no existe un vértice tal que al sacarlo el grafo resultante sea disconexo.

Es posible determinar si un grafo es conexo usando un algoritmo Búsqueda en anchura (BFS) o Búsqueda en profun-

didad (DFS).

En términos matemáticos la propiedad de un grafo de ser (fuertemente) conexo permite establecer con base en él una relación de equivalencia para sus vértices, la cual lleva a una partición de éstos en "componentes (fuertemente) conexas", es decir, porciones del grafo, que son (fuertemente) conexas cuando se consideran como grafos aislados. Esta propiedad es importante para muchas demostraciones en teoría de grafos.




*Grafo conexo y no conexo*

**Grafos completos** Un grafo es *completo* si existen aristas uniendo *todos* los pares posibles de vértices. Es decir, todo par de vértices (a, b) debe tener una arista *e* que los une.

El conjunto de los grafos completos es denominado usualmente $\mathbb{K}$ , siendo $\mathbb{K}_n$ el grafo completo de *n* vértices.

Un $\mathbb{K}_n$ , es decir, grafo completo de $n$ vértices tiene exactamente $\frac{n(n-1)}{2}$ aristas.

La representación gráfica de los $\mathbb{K}_n$ como los vértices de un polígono regular da cuenta de su peculiar estructura.

**Grafos bipartitos** Un grafo G es bipartito si puede expresarse como $G = \{V_1 \cup V_2, A\}$ (es decir, sus vértices son la unión de dos grupos de vértices), bajo las siguientes condiciones:

- $V_1$ y $V_2$ son disjuntos y no vacíos.
- Cada arista de A une un vértice de $V_1$ con uno de $V_2$.
- No existen aristas uniendo dos elementos de $V_1$; análogamente para $V_2$.

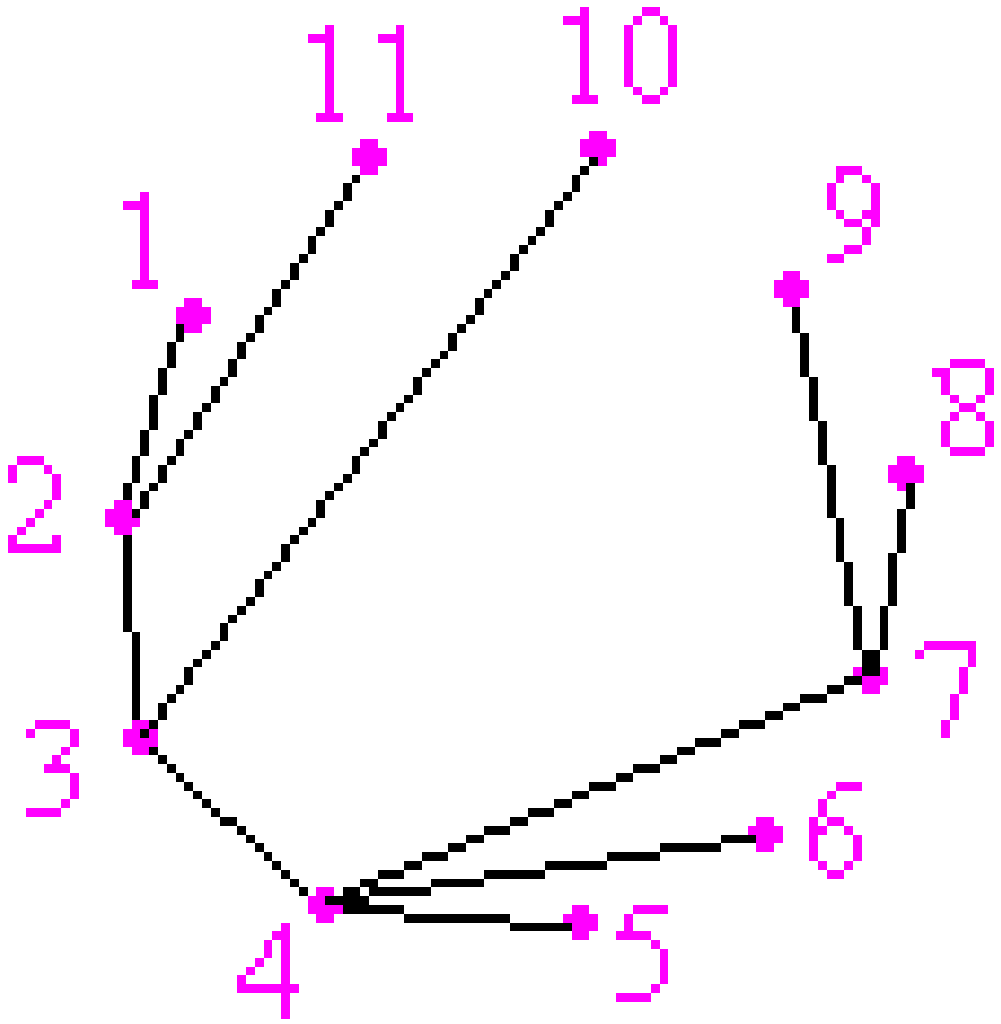
Bajo estas condiciones, el grafo se considera bipartito, y puede describirse informalmente como el grafo que une o relaciona dos conjuntos de elementos diferentes, como aquellos resultantes de los ejercicios y puzzles en los que debe unirse un elemento de la columna A con un elemento de la columna B.

### Homeomorfismo de grafos

Dos grafos $G_1$ y $G_2$ son homeomorfos si ambos pueden obtenerse a partir del mismo grafo con una sucesión de subdivisiones elementales de aristas.

### Árboles

Un grafo que no tiene ciclos y que conecta a todos los puntos, se llama un **árbol**. En un grafo con **n** vértices, los árboles tienen exactamente **n - 1** aristas, y hay $\mathbf{n^{n-2}}$ árboles posibles. Su importancia radica en que los árboles son grafos que




*Ejemplo de árbol.*

conectan todos los vértices utilizando el menor número posible de aristas. Un importante campo de aplicación de su estudio se encuentra en el análisis filogenético, el de la filiación de entidades que derivan unas de otras en un proceso evolutivo, que se aplica sobre todo a la averiguación del parentesco entre especies; aunque se ha usado también, por ejemplo, en el estudio del parentesco entre lenguas.

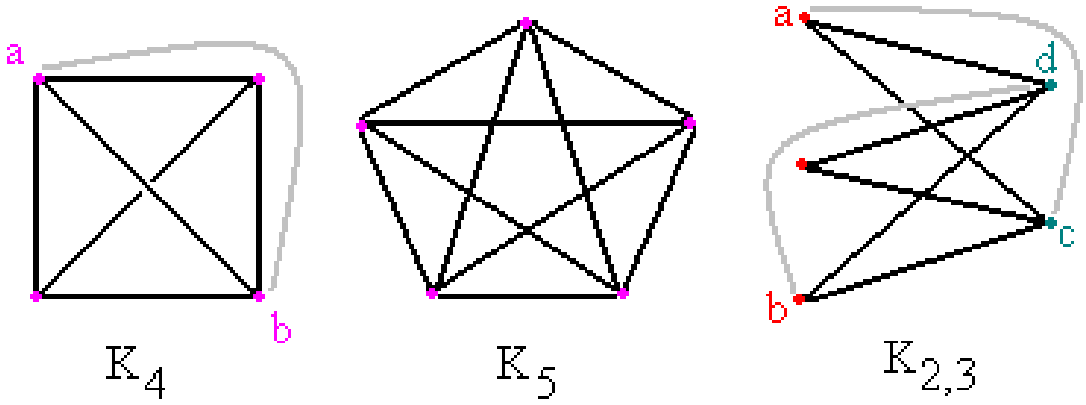
### Grafos ponderados o etiquetados

En muchos casos, es preciso atribuir a cada arista un número específico, llamado *valuación*, *ponderación* o *coste* según el contexto, y se obtiene así un **grafo valuado**.
Formalmente, es un grafo con una función v: A → **R.**.

Por ejemplo, un representante comercial tiene que visitar *n* ciudades conectadas entre sí por carreteras; su interés previsible será minimizar la distancia recorrida (o el tiempo, si se pueden prever atascos). El grafo correspondiente tendrá como vértices las ciudades, como aristas las carreteras y la valuación será la distancia entre ellas.
Y, de momento, no se conocen métodos generales para hallar un ciclo de valuación mínima, pero sí para los caminos desde *a* hasta *b*, sin más condición.

### Diámetro

En un grafo, la distancia entre dos vértices es el menor número de aristas de un recorrido entre ellos. El **diámetro**, en una figura como en un grafo, es la mayor distancia de entre todos los pares de puntos de la misma.




*En la figura se nota que $K_4$ es plano (desviando la arista* ab *al exterior del cuadrado), que $K_5$ no lo es, y que $K_{3,2}$ lo es también (desvíos en gris).*

El diámetro de los $K_n$ es 1, y el de los $K_{n,p}$ es 2. Un diámetro infinito puede significar que el grafo tiene una infinidad de vértices o simplemente que no es **conexo**. También se puede considerar el **diámetro promedio**, como el promedio de las distancias entre dos vértices.

El mundo de Internet ha puesto de moda esa idea del diámetro: Si descartamos los sitios que no tienen enlaces, y escogemos dos páginas *web* al azar: ¿En cuántos *clics* se puede pasar de la primera a la segunda? El resultado es el diámetro de la Red, vista como un grafo cuyos vértices son los sitios, y cuyas aristas son lógicamente los enlaces.

En el mundo real hay una analogía: tomando al azar dos seres humanos del mundo, ¿En cuántos saltos se puede pasar de uno a otro, con la condición de sólo saltar de una persona a otra cuando ellas se conocen personalmente? Con esta definición, se estima que el diámetro de la humanidad es de... ¡ocho solamente!

Este concepto refleja mejor la complejidad de una red que el número de sus elementos.

### 2.2.7 Algoritmos importantes

- Algoritmo de búsqueda en anchura (BFS)
- Algoritmo de búsqueda en profundidad (DFS)
- Algoritmo de búsqueda A*
- Algoritmo del vecino más cercano
- Ordenación topológica de un grafo
- Algoritmo de cálculo de los componentes fuertemente conexos de un grafo
- Algoritmo de Dijkstra
- Algoritmo de Bellman-Ford
- Algoritmo de Prim
- Algoritmo de Ford-Fulkerson
- Algoritmo de Kruskal
- Algoritmo de Floyd-Warshall

### 2.2.8 Investigadores relevantes en Teoría de grafos

- Alon, Noga
- Berge, Claude

- Bollobás, Béla
- Brightwell, Graham
- Chung, Fan
- Dirac, Gabriel Andrew
- Dijkstra, Edsger
- Edmonds, Jack
- Erdős, Paul
- Euler, Leonhard
- Faudree, Ralph
- Golumbic, Martin
- Graham, Ronald
- Harary, Frank
- Heawood, Percy John
- Kaufmann, Walter Arnold
- Kőnig, Dénes
- Kuratowski, Kazimierz
- Lovász, László
- Nešetřil, Jaroslav
- Rényi, Alfréd
- Ringel, Gerhard
- Robertson, Neil
- Seymour, Paul
- Szemerédi, Endre
- Thomas, Robin
- Thomassen, Carsten
- Turán, Pál
- Tutte, W. T.
- Whitney, Hassler

### 2.2.9 Véase también

- Grafo
- Anexo:Galería de grafos

### 2.2.10 Referencias

[1] Euler, L. (1736). «Solutio problematis ad geometriam situs pertinentis». *Commentarii Academiae Scientiarum Imperialis Petropolitanae* **8**. 128-140.

[2] http://booklens.com/l-r-foulds/graph-theory-applications pag 7

[3] Tutte, W.T. (2001), *Graph Theory*, Cambridge University Press, p. 30, ISBN 978-0-521-79489-3, http://books.google.com/books?id=uTGhooU37h4C&pg=PA30.

[4] Gorbátov:«Fundamentos de la matemática discreta»

[5] Ejemplo de una lista de incidencia

### 2.2.11 Enlaces externos

- Wikimedia Commons alberga contenido multimedia sobre **Teoría de grafos**. Commons
- Sobre los grafos VPT y los grafos EPT. Mazzoleni, María Pía. 30 de mayo de 2014.
- El contenido de este artículo incorpora material de una entrada de la **Enciclopedia Libre Universal**, publicada en español bajo la licencia Creative Commons Compartir-Igual 3.0.

## 2.3 Grafo dirigido

Un **grafo dirigido** o **digrafo** es un tipo de grafo en el cual las aristas tienen una dirección definida,[1] a diferencia del grafo generalizado, en el cual la dirección puede estar especificada o no.

Al igual que en el grafo generalizado, el grafo dirigido está definido por un par de conjuntos $G = (V, E)$ , donde:

- $V \neq \emptyset$ , un conjunto no vacío de objetos simples llamados vértices o nodos.
- $E \subseteq \{(a, b) \in V \times V : a \neq b\}$ es un conjunto de pares ordenados de elementos de $V$ denominados aristas o arcos, donde por definición un arco va del primer nodo *(a)* al segundo nodo *(b)* dentro del par.

A veces un digrafo es denominado **digrafo simple** para distinguirlo del caso general del multigrafo dirigido, donde los arcos constituyen un multiconjunto, en lugar de un conjunto. En este caso, puede haber más de un arco que una dos vértices en la misma dirección, distinguiéndose entre sí por su identidad, por su tipo (por ejemplo un tipo de arco representa relaciones de amistad mientras que el otro tipo representa mensajes enviados recientemente entre los nodos), o por un atributo como por ejemplo su importancia o *peso*.

A menudo también se considera que un digrafo simple es aquél en el que no están permitidos los bucles. Un bucle es un arco que une un vértice consigo mismo.

### 2.3.1 Terminología básica

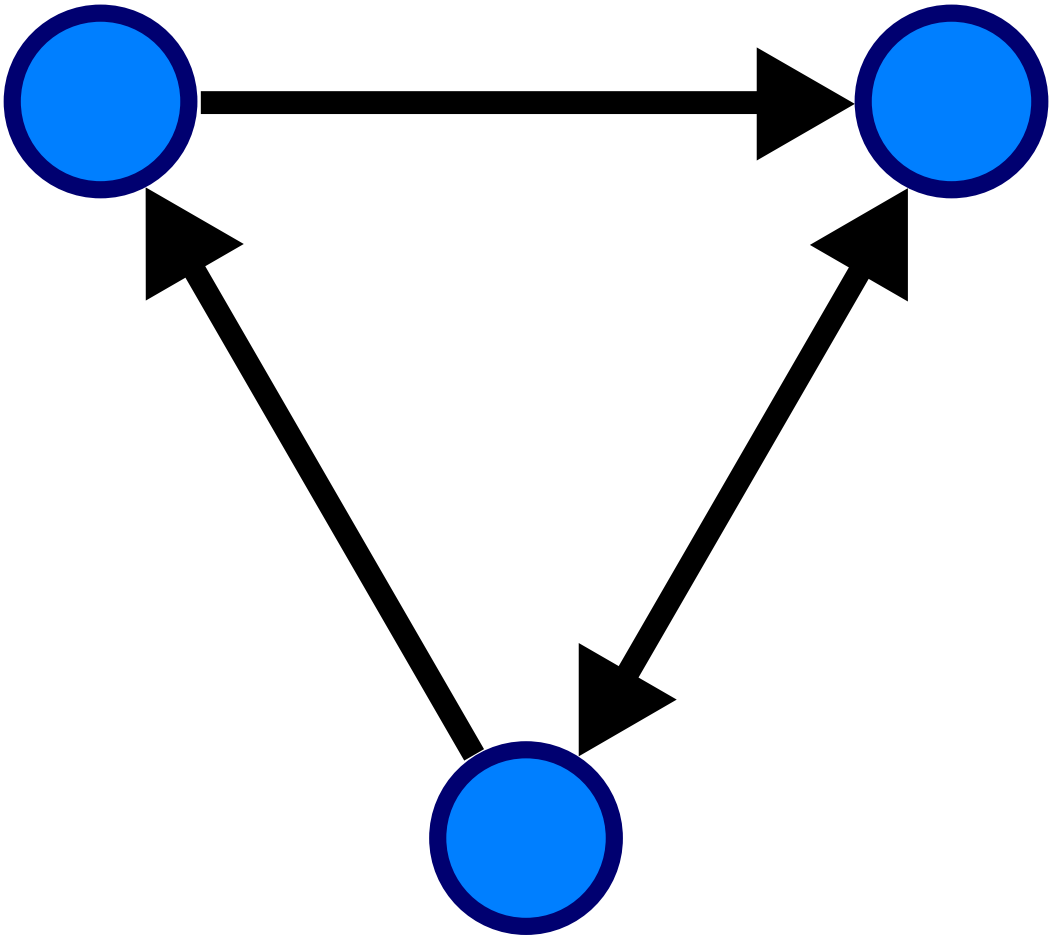
Un arco $e = (x, y)$ se considera dirigido **desde** *x* **hacia** *y*; *y* se denomina **cabeza** y *x* se denomina **cola** del arco.

*y* se denomina también un **sucesor directo** de *x*; correspondientemente, se denomina a *x* un **predecesor directo** de *y*.

Si existe un camino compuesto de uno o más arcos que una *x* con *y*, entonces a *y* se le denomina **sucesor** de *x*, al igual que a *x* se le denomina **predecesor** de *y*.

Al arco $(y, x)$ se le denomina **arco invertido** de $(x, y)$ .

Un grafo dirigido *G* es llamado **simétrico** si, para cualquier arco que pertenece a *G*, el arco invertido correspondiente también pertenece a *G*. Un grafo dirigido simétrico y sin bucles es equivalente a un grafo no dirigido; basta con reemplazar cada par de arcos dirigidos por un solo arco no dirigido.



*Un grafo dirigido.*

Una **orientación** de un grafo simple no dirigido se obtiene al asignar una orientación a cada uno de los arcos existentes. Un grafo dirigido construido de esta manera se denomina un **grafo orientado**. Una manera de distinguir entre un grafo simple dirigido y un grafo orientado es que si $x$ e $y$ son vértices, un grafo simple dirigido permite tanto $(x, y)$ como $(y, x)$ entre sus arcos, mientras que solo una de las dos posibilidades es admitida en un grafo orientado.[2][3]

Un **digrafo ponderado** es un digrafo en el que existen pesos asociados a cada uno de los arcos, de manera análoga al grafo ponderado. Un digrafo ponderado en el contexto de la teoría de grafos es denominado una **red**.

La matriz de adyacencia de un digrafo (con bucles y arcos múltiples permitidos) es una matriz compuesta por valores enteros, donde los índices de columnas y filas se corresponden con las identidades de los vertices $V$ . Un elemento de esta matriz, $a_{ij}$ representa el número de arcos existentes entre los nodos *i* y *j*. Un elemento en la diagonal de esta matriz, $a_{ii}$ representa el número de bucles que existen en el nodo *i*. La matriz de adyacencia de un digrafo es una representación única del digrafo, exceptuadas posibles permutacions de las filas y columnas.

Otra representación común de un digrafo es la matriz de incidencia.

### 2.3.2 Referencias

[1] Bang-Jensen,Gutin,2000. Diestel,2005, Section 1.10. Bondy,Murty,1976, Section 10.

[2] Diestel,2005, Section 1.10.

[3] Weisstein, Eric W. «Oriented Graph». En Weisstein, Eric W. *MathWorld* (en inglés). Wolfram Research.

## 2.4 Matriz de adyacencia

La **matriz de adyacencia** es una matriz cuadrada que se utiliza como una forma de representar relaciones binarias.

### 2.4.1 Construcción de la matriz a partir de un grafo

1. Se crea una matriz cero, cuyas columnas y filas representan los *nodos* del grafo.
2. Por cada arista que une a dos nodos, se suma 1 al valor que hay actualmente en la ubicación correspondiente de la matriz.

   Si tal arista es un bucle y el grafo es no dirigido, entonces se suma 2 en vez de 1.

Finalmente, se obtiene una matriz que representa el número de aristas (relaciones) entre cada par de nodos (elementos).

Existe una matriz de adyacencia única para cada grafo (sin considerar las permutaciones de filas o columnas), y viceversa.

#### Ejemplos

La siguiente tabla muestra dos grafos y su respectiva matriz de adyacencia. Note que en el primer caso, como se trata de un grafo no dirigido, la matriz obtenida es simétrica:

### 2.4.2 Propiedades de la matriz de adyacencia

- Para un grafo no dirigido la matriz de adyacencia es simétrica.
- El número de caminos *Ci,j*(*k*), atravesando *k* aristas desde el nodo *i* al nodo *j*, viene dado por un elemento de la potencia k-ésima de la matriz de adyacencia:

$$C_{i,j}(k) = [\mathbf{A}^k]_{ij}$$

### 2.4.3 Comparación con otras representaciones

Existen otras formas de representar relaciones binarias, como por ejemplo los pares ordenados o los grafos. Cada representación tiene sus virtudes y desventajas.

En particular, la matriz de adyacencia es muy utilizada en la programación, porque su naturaleza binaria y matricial calza perfecto con la de los computadores. Sin embargo, a una persona común y corriente se le hará mucho más sencillo comprender una relación descrita mediante grafos, que mediante matrices de adyacencia.

Otra representación matricial para las relaciones binarias es la matriz de incidencia.

### 2.4.4 Aplicaciones

La relación entre un grafo y el vector y valor propio de su correspondiente matriz de adyacencia se estudian en la *teoría espectral de grafos*.

### 2.4.5 Véase también

- Matriz de incidencia
- matriz laplaciana

## 2.5 Algoritmo de Floyd-Warshall

En informática, el **algoritmo de Floyd-Warshall**, descrito en 1959 por Bernard Roy, es un algoritmo de análisis sobre grafos para encontrar el camino mínimo en grafos dirigidos ponderados. El algoritmo encuentra el camino entre todos los pares de vértices en una única ejecución. El algoritmo de Floyd-Warshall es un ejemplo de programación dinámica.

### 2.5.1 Algoritmo

El algoritmo de Floyd-Warshall compara todos los posibles caminos a través del grafo entre cada par de vértices. El algoritmo es capaz de hacer esto con sólo $V^3$ comparaciones (esto es notable considerando que puede haber hasta $V^2$ aristas en el grafo, y que cada combinación de aristas se prueba). Lo hace mejorando paulatinamente una estimación del camino más corto entre dos vértices, hasta que se sabe que la estimación es óptima.

Sea un grafo $G$ con conjunto de vértices $V$ , numerados de 1 a N. Sea además una función caminoMinimo$(i, j, k)$ que devuelve el camino mínimo de $i$ a $j$ usando únicamente los vértices de 1 a $k$ como puntos intermedios en el camino. Ahora, dada esta función, nuestro objetivo es encontrar el camino mínimo desde cada $i$ a cada $j$ usando únicamente los vértices de 1 hasta $k + 1$ .

Hay dos candidatos para este camino: un camino mínimo, que utiliza únicamente los vértices del conjunto $(1...k)$ ; o bien existe un camino que va desde $i$ hasta $k + 1$ , y de $k + 1$ hasta $j$ , que es mejor. Sabemos que el camino óptimo de $i$ a $j$ que únicamente utiliza los vértices de 1 hasta $k$ está definido por caminoMinimo$(i, j, k)$ , y está claro que si hubiera un camino mejor de $i$ a $k + 1$ a $j$ , la longitud de este camino sería la concatenación del camino mínimo de $i$ a $k + 1$ (utilizando vértices de $(1...k)$ ) y el camino mínimo de $k + 1$ a $j$ (que también utiliza los vértices en $(1...k)$ ).

Por lo tanto, podemos definir $caminoMinimo(i, j, k)$ de forma recursiva:

caminoMinimo$(i, j, k)$ = min(caminoMinimo$(i, j, k - 1)$, caminoMinimo$(i, k, k - 1)$ + caminoMinimo$(k, j, k - 1)$);

caminoMinimo$(i, j, 0)$ = pesoArista$(i, j)$;

Esta fórmula es la base del algoritmo Floyd-Warshall. Funciona ejecutando primero $caminoMinimo(i, j, 1)$ para todos los pares $(i, j)$ , usándolos para después hallar $caminoMinimo(i, j, 2)$ para todos los pares $(i, j)$ ... Este proceso continúa hasta que $k = n$ , y habremos encontrado el camino más corto para todos los pares de vértices $(i, j)$ usando algún vértice intermedio.

### 2.5.2 Pseudocodigo

Convenientemente, cuando calculamos el k-esimo caso, se puede sobreescribir la información salvada en la computación k $-1$. Esto significa que el algoritmo usa memoria cuadrática. Hay que cuidar la inicialización de las condiciones:

1 /* Suponemos que la función *pesoArista* devuelve el coste del camino que va de i a j 2 (infinito si no existe). 3 También suponemos que $n$ es el número de vértices y *pesoArista(i,i)* = 0 4 */ 5 6 **int** camino[][]; 7 /* Una matriz bidimensional. En cada paso del algoritmo, *camino[i][j]* es el camino mínimo 8 de i hasta j usando valores intermedios de (1..k-1). Cada *camino[i][j]* es inicializado a 9 10 */ 11 12 **procedimiento** FloydWarshall () 13 para k: = 0 hasta n – 1 14 15 *camino*[i][j] = mín ( *camino[i][j]*, *camino[i][k]+camino[k][j]*) 16 17 fin para

### 2.5.3 Código en C++

// Declaraciones en el archivo .h int cn; //cantidad de nodos vector< vector<int> > ady; // Devuelve una matriz con las distancias minimas de cada nodo al resto de los vertices. vector< vector<int> > Grafo :: floydWarshall(){ vector< vector<int> > path = this->ady; for(int i = 0; i < cn; i++) path[i][i] = 0; for(int k = 0; k < cn; k++) for(int i = 0; i < cn; i++) for(int j = 0; j < cn; j++){ int dt = path[i][k] + path[k][j]; if(path[i][j] > dt) path[i][j] = dt; } return path; }

### 2.5.4 Comportamiento con ciclos negativos

Para que haya coherencia numérica, Floyd-Warshall supone que no hay ciclos negativos (de hecho, entre cualquier pareja de vértices que forme parte de un ciclo negativo, el camino mínimo no está bien definido porque el camino puede ser infinitamente pequeño). No obstante, si hay ciclos negativos, Floyd-Warshall puede ser usado para detectarlos. Si ejecutamos el algoritmo una vez más, algunos caminos pueden decrementarse pero no garantiza que, entre todos los vértices, caminos entre los cuales puedan ser infinitamente pequeños, el camino se reduzca. Si los números de la diagonal de la matriz de caminos son negativos, es condición necesaria y suficiente para que este vértice pertenezca a un ciclo negativo.

### 2.5.5 Ejemplo

Hallar el camino mínimo desde el vértice 3 hasta 4 en el grafo con la siguiente matriz de distancias:

$$D = \begin{bmatrix} 0 & 3 & 5 & 1 & \infty & \infty \\ 3 & 0 & \infty & \infty & 9 & \infty \\ 5 & \infty & 0 & 7 & 7 & 1 \\ 1 & \infty & 7 & 0 & \infty & 4 \\ \infty & 9 & 7 & \infty & 0 & \infty \\ \infty & \infty & 1 & 4 & \infty & 0 \end{bmatrix}$$

Aplicamos el algoritmo de Floyd-Warshall, y para ello en cada iteración fijamos un vértice intermedio.

**1ª Iteración:** nodo intermedio = 1

La matriz es simétrica, por lo que solamente hará falta calcular el triángulo superior de las distancias.

$d_{23} = \min(d_{23}, d_{21} + d_{13}) = 8$

$d_{24} = \min(d_{24}, d_{21} + d_{14}) = 4$

$d_{25} = \min(d_{25}, d_{21} + d_{15}) = 9$

$d_{26} = \min(d_{26}, d_{21} + d_{16}) = \infty$

$d_{32} = \min(d_{32}, d_{31} + d_{12}) = 8$

$d_{34} = \min(d_{34}, d_{31} + d_{14}) = 6$

$d_{35} = \min(d_{35}, d_{31} + d_{15}) = 7$

$d_{36} = \min(d_{36}, d_{31} + d_{16}) = 1$

$d_{45} = \min(d_{45}, d_{41} + d_{15}) = \infty$

$d_{46} = \min(d_{46}, d_{41} + d_{16}) = 4$

$d_{56} = \min(d_{56}, d_{51} + d_{16}) = \infty$

La matriz de distancia después de esta iteración es:

$$\mathcal{W}_1 = \begin{bmatrix} 0 & 3 & 5 & 1 & \infty & \infty \\ 3 & 0 & 8 & 4 & 9 & \infty \\ 5 & 8 & 0 & 6 & 7 & 1 \\ 1 & 4 & 6 & 0 & \infty & 4 \\ \infty & 9 & 7 & \infty & 0 & \infty \\ \infty & \infty & 1 & 4 & \infty & 0 \end{bmatrix}$$

**2ª Iteración:** nodo intermedio = 2

$d_{13} = \min(d_{13}, d_{12} + d_{23}) = 5$

$d_{14} = \min(d_{14}, d_{12} + d_{24}) = 1$

$d_{15} = \min(d_{15}, d_{12} + d_{25}) = 12$

$d_{16} = \min(d_{16}, d_{12} + d_{26}) = \infty$

$d_{34} = \min(d_{34}, d_{32} + d_{24}) = 6$

$d_{35} = \min(d_{35}, d_{32} + d_{25}) = 7$

$d_{36} = \min(d_{36}, d_{32} + d_{26}) = 1$

$d_{45} = \min(d_{45}, d_{42} + d_{25}) = 13$

$d_{46} = \min(d_{46}, d_{42} + d_{26}) = 4$

$d_{56} = \min(d_{56}, d_{52} + d_{26}) = \infty$

La matriz de distancia después de esta iteración es:

$$\mathcal{W}_2 = \begin{bmatrix} 0 & 3 & 5 & 1 & 12 & \infty \\ 3 & 0 & 8 & 4 & 9 & \infty \\ 5 & 8 & 0 & 6 & 7 & 1 \\ 1 & 4 & 6 & 0 & 13 & 4 \\ 12 & 9 & 7 & 13 & 0 & \infty \\ \infty & \infty & 1 & 4 & \infty & 0 \end{bmatrix}$$

**3ª Iteración:** nodo intermedio = 3

$d_{12} = \min(d_{12}, d_{13} + d_{32}) = 3$

$d_{14} = \min(d_{14}, d_{13} + d_{34}) = 1$

$d_{15} = \min(d_{15}, d_{13} + d_{35}) = 12$

$d_{16} = \min(d_{16}, d_{13} + d_{36}) = 6$

$d_{24} = \min(d_{24}, d_{23} + d_{34}) = 4$

$d_{25} = \min(d_{25}, d_{23} + d_{35}) = 9$

$d_{26} = \min(d_{26}, d_{23} + d_{36}) = 9$

$d_{45} = \min(d_{45}, d_{43} + d_{35}) = 13$

$d_{46} = \min(d_{46}, d_{43} + d_{36}) = 4$

$d_{56} = \min(d_{56}, d_{53} + d_{36}) = 8$

La matriz de distancia después de esta iteración es:

$$\mathcal{W}_3 = \begin{bmatrix} 0 & 3 & 5 & 1 & 12 & 6 \\ 3 & 0 & 8 & 4 & 9 & 9 \\ 5 & 8 & 0 & 6 & 7 & 1 \\ 1 & 4 & 6 & 0 & 13 & 4 \\ 12 & 9 & 7 & 13 & 0 & 8 \\ 6 & 9 & 1 & 4 & 8 & 0 \end{bmatrix}$$

**4ª Iteración:** nodo intermedio = 4

$d_{12} = \min(d_{12}, d_{14} + d_{42}) = 3$

$d_{13} = \min(d_{13}, d_{14} + d_{43}) = 5$

$d_{15} = \min(d_{15}, d_{14} + d_{45}) = 12$

$d_{16} = \min(d_{16}, d_{14} + d_{46}) = 5$

$d_{23} = \min(d_{23}, d_{24} + d_{43}) = 8$

$d_{25} = \min(d_{25}, d_{24} + d_{45}) = 9$

$d_{26} = \min(d_{26}, d_{24} + d_{46}) = 8$

$d_{35} = \min(d_{35}, d_{34} + d_{45}) = 7$

$d_{36} = \min(d_{36}, d_{34} + d_{46}) = 1$

$d_{56} = \min(d_{56}, d_{54} + d_{46}) = 8$

La matriz de distancia después de esta iteración es:

$$\mathcal{W}_4 = \begin{bmatrix} 0 & 3 & 5 & 1 & 12 & 5 \\ 3 & 0 & 8 & 4 & 9 & 8 \\ 5 & 8 & 0 & 6 & 7 & 1 \\ 1 & 4 & 6 & 0 & 13 & 4 \\ 12 & 9 & 7 & 13 & 0 & 8 \\ 5 & 8 & 1 & 4 & 8 & 0 \end{bmatrix}$$

**5ª Iteración:** nodo intermedio = 5

$d_{12} = \min(d_{12}, d_{15} + d_{52}) = 3$

$d_{13} = \min(d_{13}, d_{15} + d_{53}) = 5$

$d_{14} = \min(d_{14}, d_{15} + d_{54}) = 1$

$d_{16} = \min(d_{16}, d_{15} + d_{56}) = 5$

$d_{23} = \min(d_{23}, d_{25} + d_{53}) = 8$

$d_{24} = \min(d_{24}, d_{25} + d_{54}) = 4$

$d_{26} = \min(d_{26}, d_{25} + d_{56}) = 8$

$d_{34} = \min(d_{34}, d_{35} + d_{54}) = 6$

$d_{36} = \min(d_{36}, d_{35} + d_{56}) = 1$

$d_{46} = \min(d_{46}, d_{45} + d_{56}) = 4$

La matriz de distancia después de esta iteración es:

$$\mathcal{W}_5 = \mathcal{W}_4 = \begin{bmatrix} 0 & 3 & 5 & 1 & 12 & 5 \\ 3 & 0 & 8 & 4 & 9 & 8 \\ 5 & 8 & 0 & 6 & 7 & 1 \\ 1 & 4 & 6 & 0 & 13 & 4 \\ 12 & 9 & 7 & 13 & 0 & 8 \\ 5 & 8 & 1 & 4 & 8 & 0 \end{bmatrix}$$

**6ª Iteración:** nodo intermedio = 6

$d_{12} = \min(d_{12}, d_{16} + d_{62}) = 3$

$d_{13} = \min(d_{13}, d_{16} + d_{63}) = 5$

$d_{14} = \min(d_{14}, d_{16} + d_{64}) = 1$

$d_{15} = \min(d_{15}, d_{16} + d_{65}) = 12$

$d_{23} = \min(d_{23}, d_{26} + d_{63}) = 8$

$d_{24} = \min(d_{24}, d_{26} + d_{64}) = 4$

$d_{25} = \min(d_{25}, d_{26} + d_{65}) = 9$

$d_{34} = \min(d_{34}, d_{36} + d_{64}) = 5$

$d_{35} = \min(d_{35}, d_{36} + d_{65}) = 7$

$d_{45} = \min(d_{45}, d_{46} + d_{65}) = 12$

La matriz de distancia después de esta iteración es:

$$\mathcal{W}_6 = \begin{bmatrix} 0 & 3 & 5 & 1 & 12 & 5 \\ 3 & 0 & 8 & 4 & 9 & 8 \\ 5 & 8 & 0 & 5 & 7 & 1 \\ 1 & 4 & 5 & 0 & 12 & 4 \\ 12 & 9 & 7 & 12 & 0 & 8 \\ 5 & 8 & 1 & 4 & 8 & 0 \end{bmatrix}$$

Ya se han hecho todas las iteraciones posibles. Por tanto, el camino mínimo entre 2 vértices cualesquiera del grafo será el obtenido en la matriz final. En este caso, el camino mínimo entre 3 y 4 vale 5.

### 2.5.6 Análisis

Si utilizamos matrices booleanas, para encontrar todos los $n^2$ de $\mathcal{W}_k$ desde $\mathcal{W}_{k-1}$ se necesita hacer $2n^2$ operaciones binarias. Debido a que empezamos con $\mathcal{W}_0 = \mathcal{W}_{\mathcal{R}}$ y computamos la secuencia de $n$ matrices booleanas $\mathcal{W}_1$ , $\mathcal{W}_2$ , ... , $\mathcal{W}_n = \mathcal{M}_{\mathcal{R}^*}$ , el número total de operaciones binarias es de $n \times 2n^2 = 2n^3$ . Por lo tanto, la complejidad del algoritmo es $\Theta(n^3)$ y puede ser resuelto por una máquina determinista de Turing en tiempo polinómico.

### 2.5.7 Aplicaciones y generalizaciones

El algoritmo de Floyd-Warshall puede ser utilizado para resolver los siguientes problemas, entre otros:

- Camino mínimo en grafos dirigidos (algoritmo de Floyd).
- Cierre transitivo en grafos dirigidos (algoritmo de Warshall). Es la formulación original del algoritmo de Warshall. El grafo es un grafo no ponderado y representado por una matriz booleana de adyacencia. Entonces la operación de adición es reemplazada por la conjunción lógica(AND) y la operación menor por la disyunción lógica (OR).
- Encontrar una expresión regular dada por un lenguaje regular aceptado por un autómata finito (algoritmo de Kleene).
- Inversión de matrices de números reales (algoritmo de Gauss-Jordan).
- Ruta optima. En esta aplicación es interesante encontrar el camino del flujo máximo entre 2 vértices. Esto significa que en lugar de tomar los mínimos con el pseudocodigo anterior, se coge el máximo. Los pesos de las aristas representan las limitaciones del flujo. Los pesos de los caminos representan cuellos de botella; por ello, la operación de adición anterior es reemplazada por la operación mínimo.
- Comprobar si un grafo no dirigido es bipartito.

### 2.5.8 Implementación del algoritmo de Floyd-Warshall

- Implementación en C - joshuarobinson.net (en inglés).
- Implementación en PHP - julmis.julmajanne.com (gracias a Janne Mikkonen).
- Implementación en Java (explicación paso a paso) - explicación y applet disponible en pms.ifi.lmu.de (en inglés)

### 2.5.9 Referencias


- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L. (1990). *Introduction to Algorithms* (1ª Edición edición). MIT Press y McGraw-Hill. ISBN 0-262-03141-8.
  - Sección 26.2, “The Floyd–Warshall algorithm”, pág. 558–565;
  - Sección 26.4, “A general framework for solving path problems in directed graphs”, pág. 570–576.
- Floyd, Robert W. (junio de 1962). «Algorithm 97: Shortest Path». *Communications of the ACM* **5** (6): 345. doi 10.1145/367766.368168.
- Kleene, S. C. (1956). «Representation of events in nerve nets and finite automata». En C. E. Shannon and John McCarthy. *Automata Studies*. Princeton University Press. pp. 3–42.
- Warshall, Stephen (enero de 1962). «A theorem on Boolean matrices». *Journal of the ACM* **9** (1): 11–12. doi 10.1145/321105.321107.
- Kenneth H. Rosen (2003). *Discrete Mathematics and Its Applications, 5ª Edición*. Addison Wesley. ISBN 0-07-119881-4.

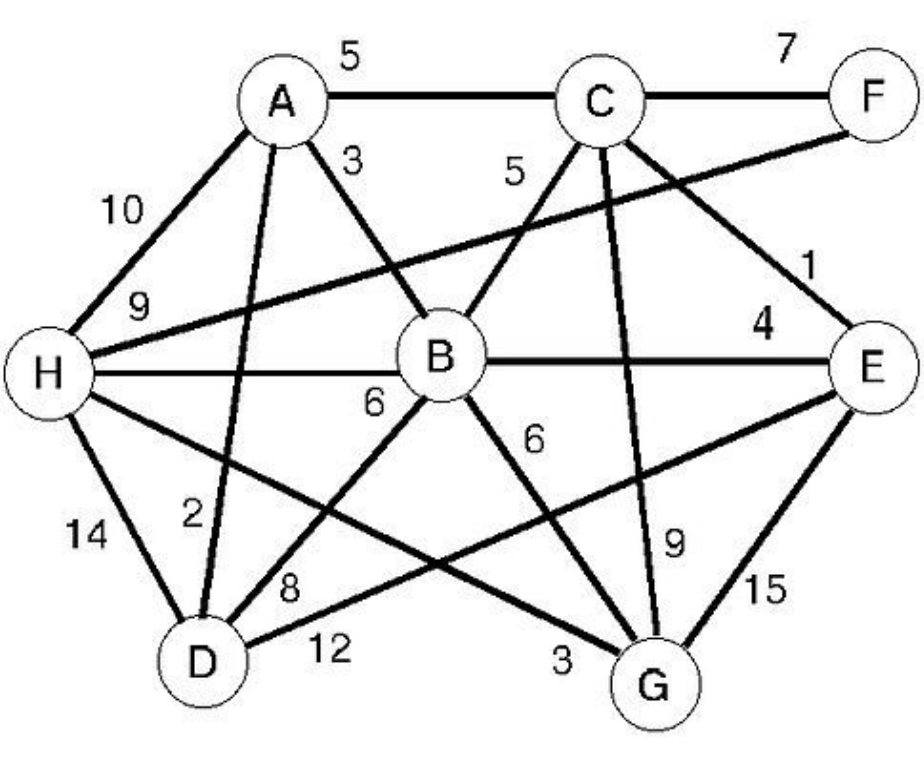
### 2.5.10 Véase también

- Algoritmo de Dijkstra
- Robert Floyd
- Lista de publicaciones de Robert W. Floyd

### 2.5.11 Enlaces externos

- Video Tutorial en VideoPractico.com de Floyd

## 2.6 Problema del camino más corto




*Ejemplo de Grafo Ponderado*

En la teoría de grafos, el **problema del camino más corto** es el problema que consiste en encontrar un camino entre dos vértices (o nodos) de tal manera que la suma de los pesos de las aristas que lo constituyen es mínima. Un ejemplo de esto es encontrar el camino más rápido para ir de una ciudad a otra en un mapa. En este caso, los vértices representarían las ciudades y las aristas las carreteras que las unen, cuya ponderación viene dada por el tiempo que se emplea en atravesarlas.

### 2.6.1 Definición

El problema del camino más corto puede ser definido para grafos no dirigidos, dirigidos o mixtos. La siguiente es una definición para grafos no dirigidos, en el caso de grafos dirigidos la definición de camino requiere que los vértices adyacentes estén conectados por una apropiada arista dirigida.

Dos vértices son adyacentes cuando poseen una arista común. Un [Camino (Teoría de grafos)|camino] en un grafo no dirigido es una secuencia de vértices $P = (v_1, v_2, \ldots, v_n) \in V \times V \times \ldots \times V$ tal que todo vértice $v_i$ es adyacente con el vértice $v_{i+1}$ . Un camino $P$ se dice que es de longitud $n$ si va desde $v_1$ hasta $v_n$ .

Sea $e_{i,j}$ la arista incidente con los vértices $v_i$ y $v_j$ . Dada una función de variable real ponderada $f : E \to \mathbb{R}$ y un grafo no dirigido $G$ , el camino más corto desde $v$ hasta $v'$ es el camino $P = (v_1, v_2, \ldots, v_n)$ (donde $v_1 = v$ y $v_n = v'$ ) sobre todos los posibles $n$ que minimiza la suma $\sum_{i=1}^{n-1} f(e_{i,i+1})$. Cuando cada arista en el grafo tiene un peso unitario o $f : E \to \{1\}$ , hallar el camino más corto es equivalente a encontrar el camino con menor número de aristas.

El problema es también conocido como el **problema de los caminos más cortos entre dos nodos**, para diferenciarlo de las siguientes generalizaciones:

- **El problema de los caminos más cortos desde un origen**, en el cual tenemos que encontrar los caminos más cortos de un vértice origen v a todos los demás vértices del grafo.
- **El problema de los caminos más cortos con un destino**, en el cual tenemos que encontrar los caminos más cortos desde todos los vértices del grafo a un único vértice destino, esto puede ser reducido al problema anterior invirtiendo el orden.
- **El problema de los caminos más cortos entre todos los pares de vértices**, el cual tenemos que encontrar los caminos más cortos entre cada par de vértices (v, v') en el grafo.

### 2.6.2 Algoritmos

Los algoritmos más importantes para resolver este problema son:

- **Algoritmo de Dijkstra**, resuelve el problema de los caminos más cortos desde un único vértice origen hasta todos los otros vértices del grafo.
- **Algoritmo de Bellman - Ford**, resuelve el problema de los caminos más cortos desde un origen si la ponderación de las aristas es negativa.
- **Algoritmo de Búsqueda A***, resuelve el problema de los caminos más cortos entre un par de vértices usando la heurística para intentar agilizar la búsqueda.
- **Algoritmo de Floyd - Warshall**, resuelve el problema de los caminos más cortos entre todos los vértices.
- **Algoritmo de Johnson**, resuelve el problema de los caminos más cortos entre todos los vértices y puede ser más rápido que el de Floyd-Warshall en grafos de baja densidad.
- **Algoritmo de Viterbi**, resuelve el problema del camino estocástico más corto con un peso probabilístico adicional en cada vértice.

***Anexo: Ejemplo de Algoritmo de Dijkstra***

***Anexo: Ejemplo de Algoritmo de Bellman - Ford*** Otros algoritmos y evaluaciones asociadas pueden se encontradas en el artículo de Cherkassky et al.[1]

### 2.6.3 Algoritmos para caminos más cortos desde un origen

**Grafos no dirigidos**

**Grafos dirigidos no ponderados**

**Grafos acíclicos dirigidos**

Un algoritmo usando ordenación topológica puede resolver el problema del camino más corto desde un origen en tiempo lineal, $\Theta(E + V)$, en DAG ponderados.[3]

**Grafos dirigidos con ponderación no negativa**

La siguiente tabla es tomada del Schrijver (2004).[4]

#### Grafos dirigidos con ponderaciones arbitrarias

### 2.6.4 Algoritmos para caminos más cortos entre todos los pares de vértices

El problema del camino más corto entre todos los pares de vértices encuentra los caminos que sean más cortos entre todas las parejas de vértices v, v' en el grafo. El problema para grafos dirigidos no ponderados fue introducido por Shimbel (1953), quien observó que podría ser resuelto por una serie lineal de multiplicaciones matriciales que toma un tiempo total de $O(V^4)$.

#### Grafos no dirigidos

#### Grafos dirigidos

### 2.6.5 Aplicaciones

Los algoritmos de los caminos más cortos se aplican para encontrar direcciones de forma automática entre lugares físicos, como las rutas de conducción en sitios de mapas web como MapQuest o Google Maps. Para estas aplicaciones están disponibles rápidos algoritmos especializados.[8]

Si un algoritmo representa una máquina abstracta no determinista como un grafo, donde los vértices describen estados, y las aristas posibles transiciones, el algoritmo del camino más cortos puede ser usado para encontrar una secuencia óptima de decisiones para llegar a un cierto estado final o para establecer límites más bajos en el tiempo necesario para alcanzar un estado dado. Por ejemplo, si los vértices representan los estados de un puzzle como el Cubo de Rubik y cada arista dirigida corresponde a un simple movimiento o giro, los algoritmos del camino más corto se pueden usar para encontrar la solución que utiliza el menor número posible de movimientos.

En el argot de las telecomunicaciones, a este algoritmo es también conocido como el problema del mínimo retraso, y con frecuencia va ligado con el problema del camino más ancho. Por ejemplo, el algoritmo podría buscar el camino más corto entre los más anchos, o el camino más ancho entre los más cortos.

Una aplicación más coloquial es la teoría de los "Seis grados de separación", a partir de la cual se intenta encontrar el camino más corto entre dos personas cualesquiera.

Otras aplicaciones incluyen la investigación de operaciones, instalaciones y facilidad de diseño, robótica, transporte y el diseño VLSI.

### 2.6.6 Problemas relacionados

En la geometría computacional, el problema del **camino euclidiano mas corto**, en el cual dados un conjunto de obstáculos poliédricos en un espacio euclídeo y dos puntos, trata de encontrar el camino más corto entre los puntos tal que no intersecta ninguno de los obstáculos.

El problema de viajante de comercio, es el problema que trata de encontrar el camino más corto que pasa sólo una vez por cada vértice y regresa al comienzo. A diferencia del problema del camino más corto, el cual puede ser resuelto en un tiempo polinomial en grafos sin ciclos negativos, este problema es NP-completo, y como tal, no tiene una resolución eficiente (ver Clases de complejidad P y NP). El problema de encontrar el camino más largo también es NP-completo.

El problema del viajero canadiense y el problema del camino estocástico más corto son generalizaciones donde el grafo no es completamente conocido por el viajero, cambia con el tiempo, o donde los recorridos son probabilisticos.

### 2.6.7 Véase también

- Búsqueda de ruta
- IEEE 802.1aq
- Red de flujo

### 2.6.8 Referencias


[1] Cherkassky, Boris V.; Goldberg, Andrew V.; Radzik, Tomasz (1996). «Shortest paths algorithms: theory and experimental evaluation». *Mathematical Programming*. Ser. A **73** (2): 129–174. doi:10.1016/0025-5610(95)00021-6. MR 1392160..

[2] Thorup, Mikkel (1999). «Undirected single-source shortest paths with positive integer weights in linear time». *Journal of the ACM (JACM)* **46** (3): 362–394. Consultado el 28 de noviembre de 2014.

[3] Papamanthou, Charalampos (2004). *Depth First Search & Directed Acyclic Graphs*. pp. 12–14. Consultado el 2 de mayo de 2015.

[4] Schrijver, Alexander (2004). *Combinatorial Optimization — Polyhedra and Efficiency*. Algorithms and Combinatorics **24**. Springer. ISBN 3-540-20456-3. Aquí: vol.A, sec.7.5b, p.103

[5] *Proceedings of the thirteenth annual ACM-SIAM symposium on Discrete algorithms*. 2002. pp.267–276

[6] *Theoretical Computer Science* **312**. 2004. pp.47–74

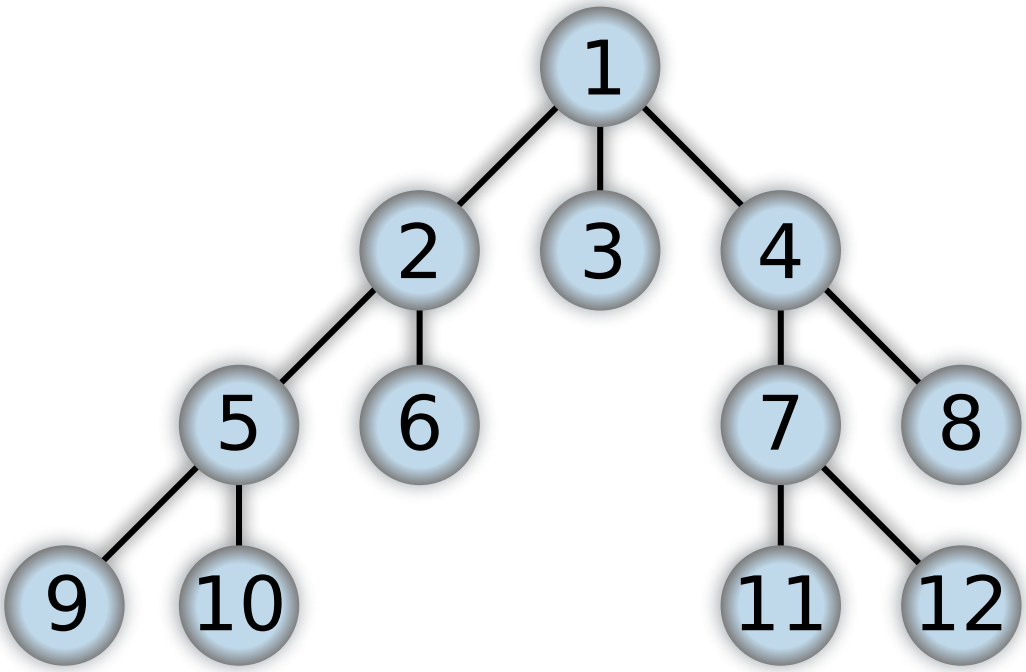
[7] *Proceedings of the 27th International Colloquium on Automata, Languages and Programming*. 2000. pp.61–72

[8] [Peter] Comprueba el valor del |enlaceautor= (ayuda) (March 23, 2009). *Fast route planning*. Google Tech Talk..

- Bellman, Richard (1958). «On a routing problem». *Quarterly of Applied Mathematics* **16**: 87–90. MR 0102435.
- Cormen, Thomas H. «Single-Source Shortest Paths and All-Pairs Shortest Paths». *Introduction to Algorithms* (2 edición). MIT Press. pp. 580–642.
- Dijkstra, E. W. (1959). «A note on two problems in connexion with graphs». *Numerische Mathematik* **1**: 269–271. doi:10.1007/BF01386390.
- Fredman, Michael Lawrence; Tarjan, Robert E. (1984). «Fibonacci heaps and their uses in improved network optimization algorithms». 25th Annual Symposium on Foundations of Computer Science. IEEE. pp. 338–346. doi:10.1109/SFCS.1984.715934. ISBN 0-8186-0591-X.
- Fredman, Michael Lawrence; Tarjan, Robert E. (1987). «Fibonacci heaps and their uses in improved network optimization algorithms». *Journal of the Association for Computing Machinery* **34** (3): 596–615. doi:10.1145/28869.28874.
- Leyzorek, M.; Gray, R. S.; Johnson, A. A.; Ladew, W. C.; Meaker, S. R., Jr.; Petry, R. M.; Seitz, R. N. (1957). *Investigation of Model Techniques — First Annual Report — 6 June 1956 — 1 July 1957 — A Study of Model Techniques for Communication Systems*. Cleveland, Ohio: Case Institute of Technology.
- Moore, E. F. (1959). «The shortest path through a maze». *Proceedings of an International Symposium on the Theory of Switching (Cambridge, Massachusetts, 2–5 April 1957)*. Cambridge: Harvard University Press. pp. 285–292.
- Shimbel, Alfonso (1953). «Structural parameters of communication networks». *Bulletin of Mathematical Biophysics* **15** (4): 501–507. doi:10.1007/BF02476438.


### 2.6.9 Enlaces externos

- Caminos más cortos - Dr J.B.Hayet
- Algebra matricial y Teoría de grafos - Luis M. Torres
- Demostración del Algoritmo de Dijkstra
- Demostración del Algoritmo de Bellman Ford
- Ejemplo de Ejercicio de Bellman Ford
- Aplicación para la resolución del algoritmo de Djkstra on-line
- Caminos más Cortos desde un Origen a muchos Destinos - Julio César López




*Búsqueda en anchura.*

## 2.7 Búsqueda en anchura

En Ciencias de la Computación, **Búsqueda en anchura** (en inglés *BFS - Breadth First Search*) es un algoritmo para recorrer o buscar elementos en un grafo (usado frecuentemente sobre árboles). Intuitivamente, se comienza en la raíz (eligiendo algún nodo como elemento raíz en el caso de un grafo) y se exploran todos los vecinos de este nodo. A continuación para cada uno de los vecinos se exploran sus respectivos vecinos adyacentes, y así hasta que se recorra todo el árbol.

Formalmente, *BFS* es un algoritmo de *búsqueda sin información*, que expande y examina todos los nodos de un árbol sistemáticamente para buscar una solución. El algoritmo no usa ninguna estrategia heurística.

Si las aristas tienen pesos negativos aplicaremos el algoritmo de Bellman-Ford en alguna de sus dos versiones.

### 2.7.1 Procedimiento

- Dado un vértice fuente **s**, *Breadth-first search* sistemáticamente explora los vértices de G para "descubrir" todos los vértices alcanzables desde **s**.
- Calcula la distancia (menor número de vértices) desde **s** a todos los vértices alcanzables.
- Después produce un árbol **BF** con raíz en **s** y que contiene a todos los vértices alcanzables.
- El camino desde **s** a cada vértice en este recorrido contiene el mínimo número de vértices. Es el camino más corto medido en número de vértices.
- Su nombre se debe a que expande uniformemente la frontera entre lo descubierto y lo no descubierto. Llega a los nodos de distancia k, sólo tras haber llegado a todos los nodos a distancia k-1.

### 2.7.2 Pseudocódigo

- La nomenclatura adicional utilizada es: Q = Estructura de datos cola

**BFS**(grafo G, nodo_fuente s) { *//recorremos todos los vértices del grafo inicializándolos a NO_VISITADO, //distancia INFINITA y padre de cada nodo NULL* **for** u ∈ V[G] **do** { estado[u] = NO_VISITADO; distancia[u] = INFINITO; /* distancia infinita si el nodo no es alcanzable */ padre[u] = NULL; } estado[s] = VISITADO; distancia[s] = 0; padre[s] = NULL; CrearCola(Q); /* nos aseguramos que la cola está vacía */ Encolar(Q, s); **while** !vacia(Q) **do** { *//extraemos el nodo **u** de la cola **Q** y exploramos todos sus nodos adyacentes* u = extraer(Q); **for** v ∈ adyacencia[u] **do** { **if** estado[v] == NO_VISITADO **then** { estado[v] = VISITADO; distancia[v] = distancia[u] + 1; padre[v] = u; Encolar(Q, v); } } } } *Falta recorrer vértices no adyacentes directa o indirectamente al vértice origen "s", pues la cola queda vacía sin los adyacentes restantes.

- El tiempo de ejecución es O(|V|+|E|). Nótese que cada nodo es puesto a la cola una vez y su lista de adyacencia es recorrida una vez también.

### 2.7.3 Referencias


- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Section 22.3: Depth-first search, pp.531 - 539.


### 2.7.4 Véase también

- Árbol (estructura de datos)
- Recorrido de árboles
- Búsqueda en profundidad
- Wikimedia Commons alberga contenido multimedia sobre **Búsqueda en anchura**. Commons

## 2.8 Búsqueda en profundidad

Una **Búsqueda en profundidad** (en inglés DFS o *Depth First Search*) es un algoritmo que permite recorrer todos los nodos de un grafo o árbol (teoría de grafos) de manera ordenada, pero no uniforme. Su funcionamiento consiste en ir expandiendo todos y cada uno de los nodos que va localizando, de forma recurrente, en un camino concreto. Cuando ya no quedan más nodos que visitar en dicho camino, regresa (Backtracking), de modo que repite el mismo proceso con cada uno de los hermanos del nodo ya procesado.

Análogamente existe el algoritmo de búsqueda en anchura (BFS o *Breadth First Search*).

### 2.8.1 Pseudocódigo

- **Pseudocódigo para grafos**

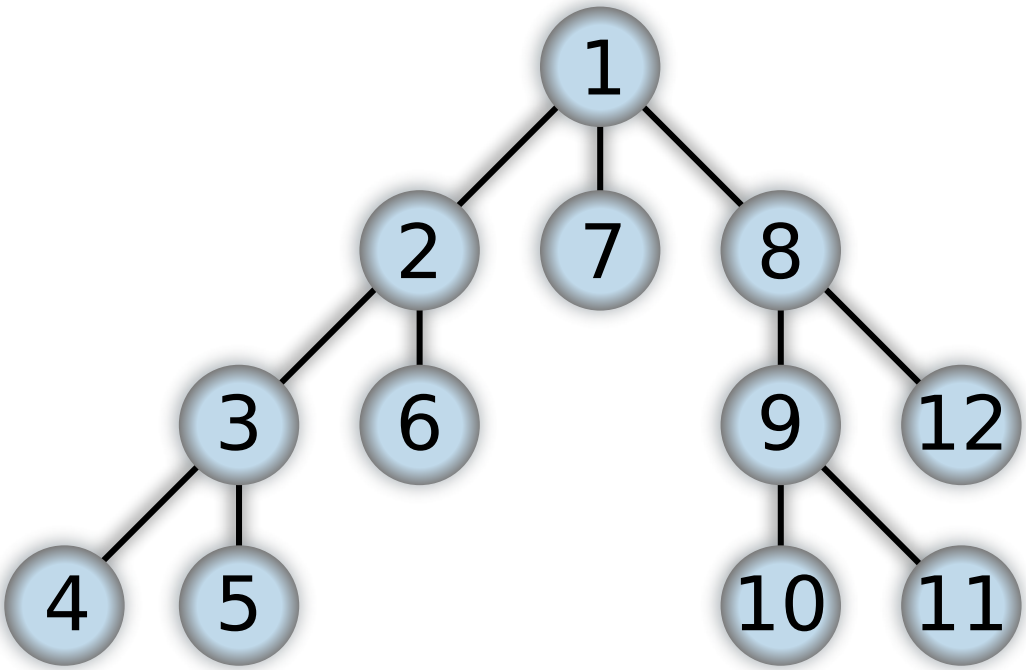
**DFS**(grafo G) **PARA CADA** vertice *u* ∈ V[G] **HACER** estado[*u*] ← NO_VISITADO padre[*u*] ← NULO tiempo ← 0 **PARA CADA** vertice *u* ∈ V[G] **HACER SI** estado[u] = NO_VISITADO **ENTONCES** DFS_Visitar(*u*,*tiempo*) **DFS_Visitar**(nodo *u*, int *tiempo*) estado[*u*] ← VISITADO tiempo ← tiempo + 1 d[u] ← tiempo **PARA CADA** *v* ∈ Vecinos[*u*] **HACER SI** estado[*v*] = NO_VISITADO **ENTONCES** padre[*v*] ← *u* DFS_Visitar(*v*,*tiempo*) estado[*u*] ← TERMINADO tiempo ← tiempo + 1 *f[u]* ← tiempo

### 2.8.2 Arcos DF

Si en tiempo de descubrimiento de *u* tenemos el arco **(u,v)**:

i. Si el estado de *v* es NO_VISITADO, entonces *(u,v)* ∈ DF,

El tiempo de ejecución es O(|V|+|E|)




*Búsqueda en profundidad.(Orden en el que se visitan los nodos)*

### 2.8.3 Véase también

- Árbol (estructura de datos)
- Recorrido de árboles
- Búsqueda en anchura

- Lema: Un grafo dirigido es cíclico si y sólo si al ejecutar DFS(G) produce al menos un arco hacia atrás.

### 2.8.4 Referencias


- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Section 22.3: Depth-first search, pp.540–549.

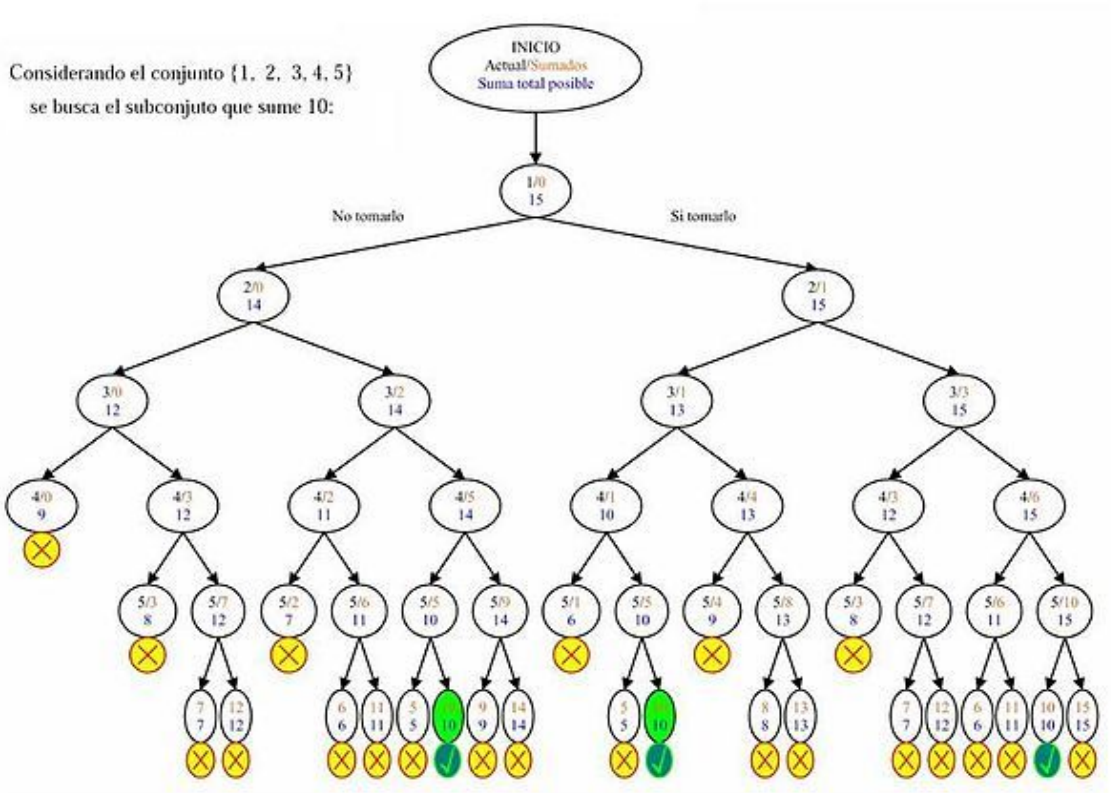

- Wikimedia Commons alberga contenido multimedia sobre **Búsqueda en profundidad**. Commons

## 2.9 Vuelta atrás

**Vuelta atrás** (*Backtracking*) es una estrategia para encontrar soluciones a problemas que satisfacen restricciones. El término "backtrack" fue acuñado por primera vez por el matemático estadounidense D. H. Lehmer en la década de 1950.

### 2.9.1 Concepto

En su forma básica, la idea de backtracking se asemeja a un recorrido en profundidad dentro de un grafo dirigido. El grafo en cuestión suele ser un árbol, o por lo menos no contiene ciclos. Sea cual sea su estructura, existe sólo implícitamente. El objetivo del recorrido es encontrar soluciones para algún problema. Esto se consigue construyendo




*Ejemplo de árbol de búsqueda. Nótese que el árbol no tiene por qué ser binario.*

soluciones parciales a medida que progresa el recorrido; estas soluciones parciales limitan las regiones en las que se puede encontrar una solución completa. El recorrido tiene éxito si, procediendo de esta forma, se puede definir por completo una solución. En este caso el algoritmo puede, o bien detenerse (si lo único que se necesita es una solución del problema) o bien seguir buscando soluciones alternativas (si deseamos examinarlas todas). Por otra parte, el recorrido no tiene éxito si en alguna etapa la solución parcial construida hasta el momento no se puede completar. En tal caso, el recorrido vuelve atrás exactamente igual que en un recorrido en profundidad, eliminando sobre la marcha los elementos que se hubieran añadido en cada fase. Cuando vuelve a un nodo que tiene uno o más vecinos sin explorar, prosigue el recorrido de una solución.

Algoritmo de Backtracking proc Backtracking (↕X[1 . . . i ]: TSolución, ↑ok: B) variables L: ListaComponentes inicio si EsSolución (X) entonces ok CIERTO de lo contrario ok FALSO L=Candidatos (X) mientras ¬ok ^ ¬Vacía (L) hacer X[i + 1] Cabeza (L); L Resto (L) Backtracking (X, ok) fin mientras fin si fin

Podemos visualizar el funcionamiento de una técnica de backtracking como la exploración en profundidad de un grafo.

Cada vértice del grafo es un posible estado de la solución del problema. Cada arco del grafo representa la transición entre dos estados de la solución (i.e., la toma de una decisión).

Típicamente el tamaño de este grafo será inmenso, por lo que no existirá de manera explícita. En cada momento sólo tenemos en una estructura los nodos que van desde el estado inicial al estado actual. Si cada secuencia de decisiones distinta da lugar a un estado diferente, el grafo es un árbol (el árbol de estados).

### 2.9.2 Enfoques

Los problemas que deben satisfacer un determinado tipo de restricciones son problemas completos, donde el orden de los elementos de la solución no importa. Estos problemas consisten en un conjunto (o lista) de variables a la que a cada una se le debe asignar un valor sujeto a las restricciones del problema. La técnica va creando todas las posibles combinaciones de elementos para obtener una solución. Su principal virtud es que en la mayoría de las implementaciones se puede evitar combinaciones, estableciendo funciones de acotación (o poda) reduciendo el tiempo de ejecución.

Vuelta atrás está muy relacionado con la búsqueda combinatoria.

### 2.9.3 Diseño e implementación

Esencialmente, la idea es encontrar la mejor combinación posible en un momento determinado, por eso, se dice que este tipo de algoritmo es una búsqueda en profundidad. Durante la búsqueda, si se encuentra una alternativa incorrecta, la búsqueda retrocede hasta el paso anterior y toma la siguiente alternativa. Cuando se han terminado las posibilidades, se vuelve a la elección anterior y se toma la siguiente opción (hijo [si nos referimos a un árbol]). Si no hay más alternativas la búsqueda falla. De esta manera, se crea un árbol implícito, en el que cada nodo es un estado de la solución (solución parcial en el caso de nodos interiores o solución total en el caso de los nodos hoja).

Normalmente, se suele implementar este tipo de algoritmos como un procedimiento recursivo. Así, en cada llamada al procedimiento se toma una variable y se le asignan todos los valores posibles, llamando a su vez al procedimiento para cada uno de los nuevos estados. La diferencia con la búsqueda en profundidad es que se suelen diseñar funciones de cota, de forma que no se generen algunos estados si no van a conducir a ninguna solución, o a una solución peor de la que ya se tiene. De esta forma se ahorra espacio en memoria y tiempo de ejecución.

### 2.9.4 Heurísticas

Algunas heurísticas son comúnmente usadas para acelerar el proceso. Como las variables se pueden procesar en cualquier orden, generalmente es más eficiente intentar ser lo más restrictivo posible con las primeras (esto es, las primeras con menores valores posibles). Este proceso poda el árbol de búsqueda antes de que se tome la decisión y se llame a la subrutina recursiva.

Cuando se elige qué valor se va a asignar, muchas implementaciones hacen un examen hacia delante (FC, Forward Checking), para ver qué valor restringirá el menor número posible de valores, de forma que se anticipa en a) preservar una posible solución y b) hace que la solución encontrada no tenga restricciones destacadas.

Algunas implementaciones muy sofisticadas usan una función de cotas, que examina si es posible encontrar una solución a partir de una solución parcial. Además, se comprueba si la solución parcial que falla puede incrementar significativamente la eficiencia del algoritmo. Por el uso de estas funciones de cota, se debe ser muy minucioso en su implementación de forma que sean poco costosas computacionalmente hablando, ya que lo más normal es que se ejecuten en para cada nodo o paso del algoritmo. Cabe destacar, que las cotas eficaces se crean de forma parecida a las funciones heurísticas, esto es, relajando las restricciones para conseguir mayor eficiencia.

Con el objetivo de mantener la solución actual con coste mínimo, los algoritmos vuelta atrás mantienen el coste de la mejor solución en una variable que va variando con cada nueva mejor solución encontrada. Así, si una solución es peor que la que se acaba de encontrar, el algoritmo no actualizará la solución. De esta forma, devolverá siempre la mejor solución que haya encontrado.

### 2.9.5 Ejemplos de aplicación de backtracking

SATISFIABILITY

Inicialmente A contiene la expresión booleana que constituye el problema.

Elegir subproblema de A, p.ejemplo : (x+y+z)(x'+y)(y'+z)(z'+x)(x'+y'+z').

Elegir una cláusula con mínimo número de literales.

Elegir una variable x, y, z,... dentro de la cláusula y crear 2 subproblemas reemplazando x=V y x=F.

En el caso x=V

Omitir las cláusulas donde aparece x. Omitir x' en las cláusulas que aparece x'.

En el caso x=F

Omitir las cláusulas donde aparece x'. Omitir x en las cláusulas que aparece x.

Test

Si no quedan cláusulas. STOP. (solución encontrada).

Si hay una cláusula vacía. DROP.

En otro caso añadir a A

Nota: Observemos que si encontramos a A vacío entonces la expresión booleana no puede ser satisfecha.

**HAMILTON CYCLE (VIAJANTE DE COMERCIO)**

En este caso los subproblemas S son caminos que parten de a y llegan a b a través de un sucesión de nodos T. (b es el mismo a lo largo de todo el algoritmo).

Inicialmente A contiene solamente el camino (a, vacío, b).

Elegimos un subproblema S cualquiera de A (y lo borramos de A) y añadimos ramas (c, a) del grafo (las c's son las adyacentes de a) . Estos caminos extendidos son los hijos. Ahora cada c juega el rol de a.

Examinamos c/ hijo:

Test:

1)Si G-T forma un camino hamiltoniano STOP (solución hallada)

2)Si G-T tiene un nodo de grado uno (excepto a y b) o si G-T-{a, b} es disconexo entonces DROP este subproblema .

3) Si 1) y 2) fallan add subproblema en A.

**EXACT COVER**

Dado un conjunto finito U y una familia se subconjuntos {Tj} de U definimos una matriz A donde cada fila se corresponde con un elemento ui de U y cada columna de A con un subconjunto Tj . Ponemos aij=1 si ui□U pertenece a Tj y aij=0 en caso contrario. Interpretamos que xj=1 significa que elegimos Tj y 0 en caso contrario.

Se trata de averiguar si es factible Ax=1 donde A y x son binarias y las componentes de 1 son unos.

S0= un vector de ceros (raíz del árbol)

Cada nodo S del árbol es una sucesión x cuyas primeras k componentes le han sido asignados un 1 o un 0 y el resto de componentes son ceros. Reemplazamos S por 2 subproblemas Si (i=1,2) poniendo xk+1 =1 y xk+1=0 respectivamente.

Test

if Ax=1 STOP

if Ax>1 DROP Si

if Ax<1 add Si to A

### 2.9.6 Ejemplos de problemas comunes resueltos usando Vuelta Atrás

- Sudoku
- Problema de los movimientos de un caballo
- Las ocho reinas

### 2.9.7 Aplicaciones

Vuelta atrás se usa en la implementación de los lenguajes de programación tales como Lenguaje de programación Planner y Prolog. Además, se usa en los análisis sintácticos de los compiladores. Su uso en inteligencia artificial ha sido muy importante, dando lugar a nuevos tipos de búsquedas como el A estrella.

### 2.9.8 Branch & Bound (Ramificación y poda)

Este método busca una solución como en el método de backtracking, pero cada solución tiene asociado un costo y la solución que se busca es una de mínimo costo llamada óptima. Además de ramificar una solución padre (branch) en hijos se trata de eliminar de consideración aquellos hijos cuyos descendientes tienen un costo que supera al óptimo buscado acotando el costo de los descendientes del hijo (bound). La forma de acotar es un arte que depende de cada

problema. La acotacion reduce el tiempo de búsqueda de la solución óptima al "podar" (pruning) los subarboles de descendientes costosos.

## 2.10 Ordenación topológica

Una **ordenación topológica** (**topological sort**, **topological ordering**, **topsort** o **toposort** en inglés) de un grafo acíclico *G* dirigido es una ordenación lineal de todos los nodos de *G* que conserva la unión entre vértices del grafo *G* original. La condición que el grafo no contenga ciclos es importante, ya que no se puede obtener ordenación topológica de grafos que contengan ciclos.

Usualmente, para clarificar el concepto se suelen identificar los nodos con tareas a realizar en la que hay una precedencia a la hora de ejecutar dichas tareas. La ordenación topológica por tanto es una lista en orden lineal en que deben realizarse las tareas.

Para poder encontrar la **ordenación topológica** del grafo *G* deberemos aplicar una modificación del algoritmo de búsqueda en profundidad (DFS).

### 2.10.1 Algoritmos

Los algoritmos usuales para el ordenamiento topológico tienen un tiempo de ejecución de la cantidad de nodos más la cantidad de aristas (O(|V|+|E|)).

Uno de los algoritmos, primero descrito por Kahn (1962), trabaja eligiendo los vértices del mismo orden como un eventual orden topológico. Primero, busca la lista de los "nodos iniciales" que no tienen arcos entrantes y los inserta en un conjunto S; donde al menos uno de esos nodos existe si el grafo es acíclico. Entonces:

L ← Lista vacía que contendrá luego los elementos ordenados. S ← Conjunto de todos los nodos sin aristas entrantes. **MIENTRAS** [*S* no es vacío]: *n* ← nodo extraído de *S* insertar *n* en *L* **PARA CADA** [nodo *m* con arista *e* de *n* a *m*]: Eliminar arista *e* del grafo **SI** [*m* no tiene más aristas entrantes]: insertar *m* en *S* **SI** [el grafo tiene más aristas]: *error: el grafo tiene al menos un ciclo* **SINO**: **RETORNAR** *L*

Si respeta la definición de GAD, ésta es una solución posible, listada en L (no es la única solución). De lo contrario el grafo contiene al menos un ciclo y por lo tanto un ordenamiento topológico es imposible.

Ha de tenerse en cuenta que, debido a la falta de unicidad del orden resultante, la estructura S puede ser simplemente un conjunto, una cola o una pila.

Dependiendo del orden que los nodos "n" son extraídos del conjunto S, hay una diferente posible solución.

Una alternativa al algoritmo visto para ordenamiento topológico está basado en DFS (del inglés búsqueda en profundidad). Para este algoritmo, las aristas están en dirección contraria al algoritmo anterior (y en dirección contraria a lo que muestra el diagrama del ejemplo). Hay un arco desde x a y si la tarea x depende de la tarea y (en otras palabras, si la tarea y debe completarse antes que la tarea x empiece). El algoritmo se repite a través de cada nodo del grafo, en un orden arbitrario, iniciando una búsqueda en profundidad que termina cuando llega a un nodo que ya ha sido visitado desde el comienzo del orden topológico.

La **ordenación topológica** no es única. Depende en qué orden recorras los nodos del grafo en el bucle *for* de la función *ORDENACIÓN_TOPOLÓGICA*. La nomenclatura adicional utilizada es: lista = Estructura de datos lista enlazada

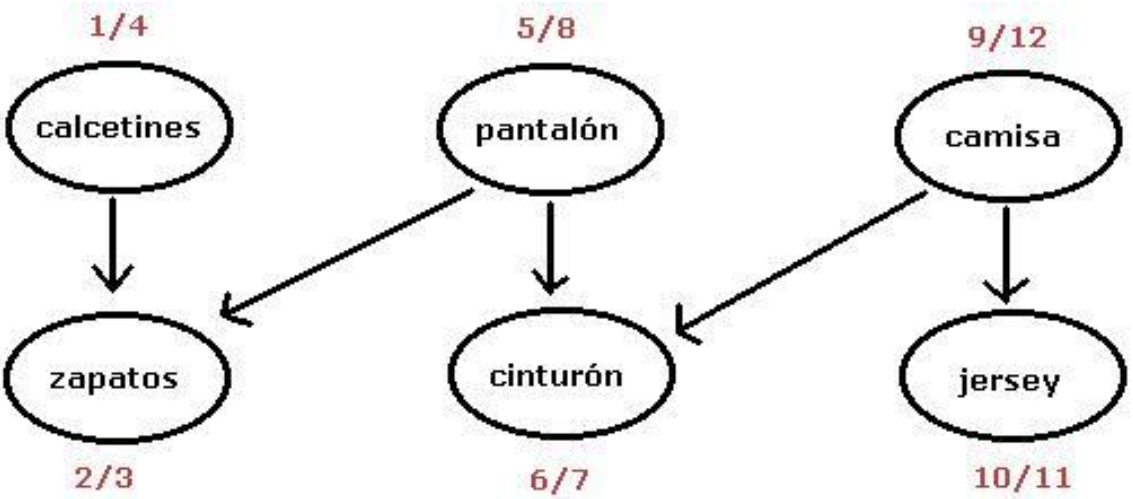
**ORDENACIÓN_TOPOLÓGICA**(grafo G) **for each** vertice $u \in$ V[G]**do** estado[$u$] = NO_VISITADO padre[$u$] = NULL tiempo =0 **for each** vertice $u \in$ V[G]**do if** estado[u] = NO_VISITADO **then** TOPOLÓGICO-Visitar($u$) **TOPOLÓGICO-Visitar**(nodo $u$) estado[$u$]=VISITADO tiempo = tiempo+1 distancia[u] = tiempo **for each** $v \in$ Adyacencia[$u$] **do if** estado[$v$]=NO_VISITADO **then** padre[$v$]=$u$ TOPOLÓGICO-Visitar($v$) estado[$u$] = TERMINADO tiempo = tiempo+1 finalización[u] = tiempo insertar (lista, u)

Al final de la ejecución del algoritmo se devuelve la lista enlazada de nodos, que corresponde con la **ordenación topológica del grafo** .

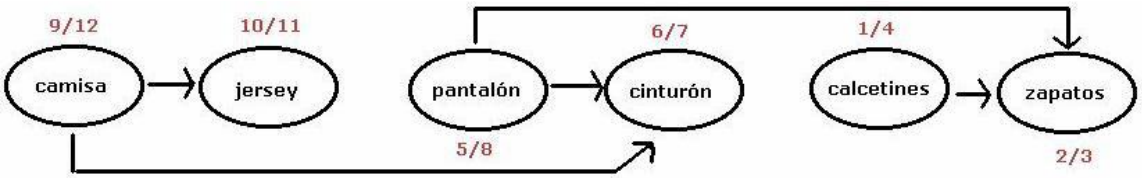
### 2.10.2 Ejemplos

- En rojo se muestran los siguientes tiempos: **distancia[u] / finalización[u]**

1. Ejecutamos el algoritmo **ORDENACIÓN_TOPOLÓGICA (grafo G)** sobre el siguiente grafo.




2. El algoritmo nos devuelve una lista enlazada con los nodos del grafo en orden decreciente en tiempo de finalización.




Grafo **ordenado topológicamente**. En él se pueden ver claramente las precedencias de las tareas:

- Ponerse la camisa antes que el cinturón y el jersey
- Ponerse el pantalón antes que los zapatos y el cinturón
- Ponerse los calcetines antes que los zapatos

La aplicación canónica del **orden topológico** es en programación, una secuencia de tareas; los algoritmos de ordenamiento topológico fueron estudiados por primera vez a los comienzos de los años 60 en el contexto de la técnica "PERT" (Técnica de Revisión y Evaluación de Programas del inglés) de programación en gestión de proyectos ((Jarnagin, ,1960)). Las tareas están representados por vértices, y hay un **arco** (o **arista**) desde *x* a *y* si la tarea *x* debe completarse antes que la tarea *y* comience (por ejemplo, cuando se lava la ropa, la lavadora debe terminar antes de ponerla a secar). Entonces, un **orden topológico** brinda un orden para ejecutar las tareas.

### 2.10.3 Véase también

### 2.10.4 Referencias


- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Section 22.3: Depth-first search, pp.540–549.


## 2.11 Algoritmo de Dijkstra

El **algoritmo de Dijkstra**, también llamado **algoritmo de caminos mínimos**, es un algoritmo para la determinación del camino más corto dado un vértice origen al resto de vértices en un grafo con pesos en cada arista. Su nombre se refiere a Edsger Dijkstra, quien lo describió por primera vez en 1959.

La idea subyacente en este algoritmo consiste en ir explorando todos los caminos más cortos que parten del vértice origen y que llevan a todos los demás vértices; cuando se obtiene el camino más corto desde el vértice origen, al resto de vértices que componen el grafo, el algoritmo se detiene. El algoritmo es una especialización de la búsqueda de costo uniforme, y como tal, no funciona en grafos con aristas de coste negativo (al elegir siempre el nodo con distancia menor, pueden quedar excluidos de la búsqueda nodos que en próximas iteraciones bajarían el costo general del camino al pasar por una arista con costo negativo).

### 2.11.1 Algoritmo

Teniendo un grafo dirigido ponderado de N nodos no aislados, sea x el nodo inicial, un vector D de tamaño N guardará al final del algoritmo las distancias desde x al resto de los nodos.

1. Inicializar todas las distancias en D con un valor infinito relativo ya que son desconocidas al principio, exceptuando la de x que se debe colocar en 0 debido a que la distancia de x a x sería 0.
2. Sea a = x (tomamos a como nodo actual).
3. Recorremos todos los nodos adyacentes de *a*, excepto los nodos marcados, llamaremos a estos nodos no marcados $v_i$.
4. Para el nodo actual, calculamos la distancia tentativa desde dicho nodo a sus vecinos con la siguiente fórmula: $dt(v_i) = D_a + d(a,v_i)$. Es decir, la distancia tentativa del nodo '$v_i$' es la distancia que actualmente tiene el nodo en el vector D más la distancia desde dicho el nodo 'a' (el actual) al nodo $v_i$. Si la distancia tentativa es menor que la distancia almacenada en el vector, actualizamos el vector con esta distancia tentativa. Es decir: Si $dt(v_i) < D_{vi} \rightarrow D_{vi} = dt(v_i)$
5. Marcamos como completo el nodo a.
6. Tomamos como próximo nodo actual el de menor valor en D (puede hacerse almacenando los valores en una cola de prioridad) y volvemos al paso 3 mientras existan nodos no marcados.

Una vez terminado al algoritmo, D estará completamente lleno.

### 2.11.2 Complejidad

Orden de complejidad del algoritmo: $O(|V|^2+|A|) = O(|V|^2)$ sin utilizar cola de prioridad, $O((|A|+|V|) \log |V|) = O(|A| \log |V|)$ utilizando cola de prioridad (por ejemplo un montículo). Por otro lado, si se utiliza un Montículo de Fibonacci, sería $O(|V| \log |V|+|A|)$.

Podemos estimar la complejidad computacional del algoritmo de Dijkstra (en términos de sumas y comparaciones). El algoritmo realiza a lo más n-1 iteraciones, ya que en cada iteración se añade un vértice al conjunto distinguido. Para estimar el número total de operaciones basta estimar el número de operaciones que se llevan a cabo en cada iteración. Podemos identificar el vértice con la menor etiqueta entre los que no están en $S_k$ realizando n-1 comparaciones o menos. Después hacemos una suma y una comparación para actualizar la etiqueta de cada uno de los vértices que no están en $S_k$. Por tanto, en cada iteración se realizan a lo sumo 2(n-1) operaciones, ya que no puede haber más de n-1 etiquetas por actualizar en cada iteración. Como no se realizan más de n-1 iteraciones, cada una de las cuales supone a lo más 2(n-1) operaciones, llegamos al siguiente teorema.

***TEOREMA:*** El Algoritmo de Dijkstra realiza $O(n^2)$ operaciones (sumas y comparaciones) para determinar la longitud del camino más corto entre dos vértices de un grafo ponderado simple, conexo y no dirigido con n vértices.

### 2.11.3 Pseudocódigo

Estructura de datos auxiliar: Q = Estructura de datos Cola de prioridad (se puede implementar con un montículo)

**DIJKSTRA** (Grafo *G*, nodo_fuente *s*) **para** $u \in V[G]$ **hacer** distancia[*u*] = INFINITO padre[*u*] = NULL distancia[*s*] = 0 adicionar (cola, (s, distancia[*s*])) **mientras que** cola no es vacía **hacer** *u* = extraer_mínimo(cola) **para** todos *v* ∈ adyacencia[*u*] **hacer si** distancia[*v*] > distancia[*u*] + peso (u, v) **hacer** distancia[*v*] = distancia[*u*] + peso (u, v) padre[*v*] = *u* adicionar(cola,(v, distancia[*v*]))

### 2.11.4 Otra versión en pseudocódigo sin cola de prioridad

**función Dijkstra** (Grafo **G**, nodo_salida **s**) //Usaremos un vector para guardar las distancias del nodo salida al resto entero distancia[n] //Inicializamos el vector con distancias iniciales booleano visto[n] //vector de boleanos para controlar los vértices de los que ya tenemos la distancia mínima **para cada** w ∈ V[G] **hacer Si** (no existe arista entre s y w) **entonces** distancia[w] = Infinito //puedes marcar la casilla con un $-1$ por ejemplo **Si_no** distancia[w] = peso (s, w) **fin si fin para** distancia[s] = 0 visto[s] = cierto //n es el número de vértices que tiene el Grafo **mientras que** (no_estén_vistos_todos) **hacer** vértice = coger_el_mínimo_del_vector distancia y que no esté visto; visto[vértice] = cierto; **para cada** w ∈ sucesores (G, vértice) **hacer si** distancia[w]>distancia[vértice]+peso (vértice, w) **entonces** distancia[w] = distancia[vértice]+peso (vértice, w) **fin si fin para fin mientras fin función**.

Al final tenemos en el vector distancia en cada posición la distancia mínima del vértice salida a otro vértice cualquiera.

### 2.11.5 Véase también

- Anexo:Ejemplo de Algoritmo de Dijkstra
- Algoritmo de Bellman-Ford

### 2.11.6 Enlaces externos

- Wikimedia Commons alberga contenido multimedia sobre **Algoritmo de Dijkstra**Commons.
- Presentación del Algoritmo de Dijkstra
- Applets en Java para probar el algoritmo de Dijkstra (Inglés)
- Graph módulo Perl en CPAN
- Bio::Coordinate::Graph módulo Perl en CPAN que implementa el algoritmo de Dijkstra
- Algoritmo de Dijkstra en Javascript Resolución online del Algoritmo de Dijkstra.
- Optimización del algoritmo del camino más corto entre todos los nodos de un grafo no dirigido Comparación entre el algoritmo de Dijkstra y el de Floyd-Warshall, y método para optimizar el primero
- Distintas implementaciones del algoritmo en RosettaCode.org

## 2.12 Algoritmo voraz

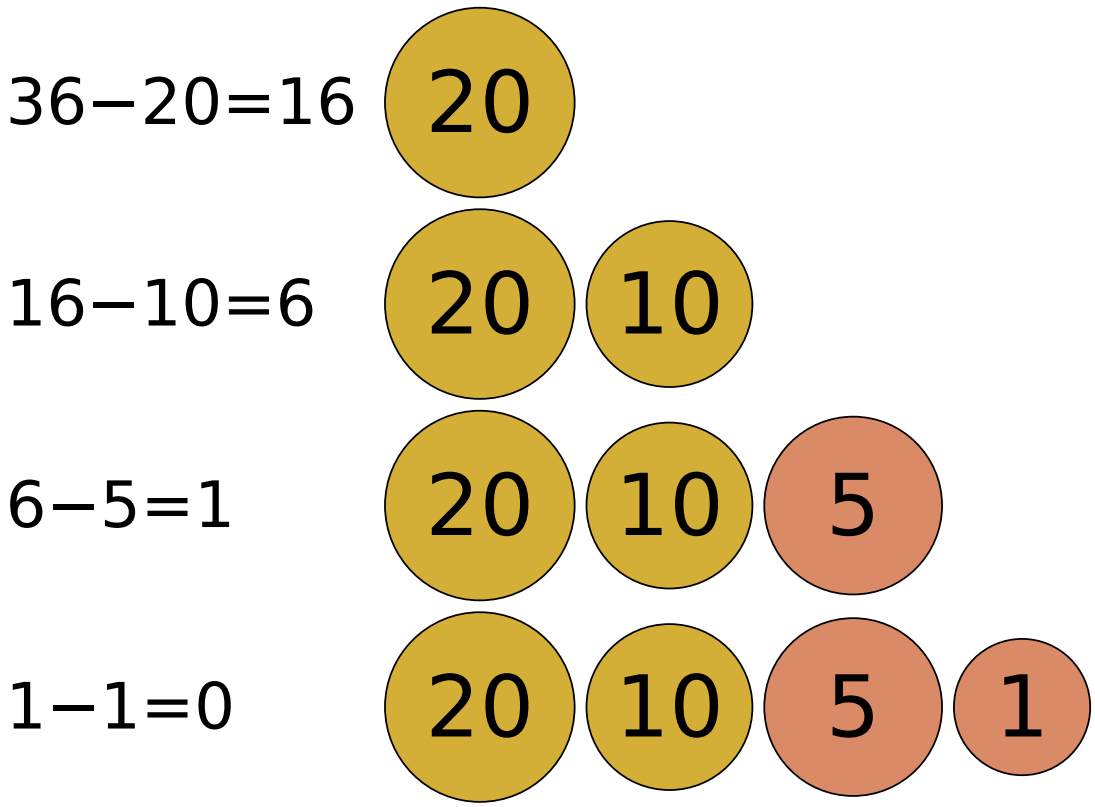
Un **algoritmo voraz** (también conocido como **ávido**, **devorador** o **goloso**) es aquel que, para resolver un determinado problema, sigue una heurística consistente en elegir la opción óptima en cada paso local con la esperanza de llegar a una solución general óptima. Este esquema algorítmico es el que menos dificultades plantea a la hora de diseñar y comprobar su funcionamiento. Normalmente se aplica a los problemas de optimización.

### 2.12.1 Esquema

Dado un conjunto finito de entradas $C$ , un algoritmo voraz devuelve un conjunto $S$ (seleccionados) tal que $S \subseteq C$ y que además cumple con las restricciones del problema inicial. A cada conjunto $S$ que satisfaga las restricciones se le suele denominar prometedor, y si este además logra que la función objetivo se minimice o maximice (según corresponda) diremos que $S$ es una *solución óptima*.

#### Elementos de los que consta la técnica

- El conjunto $C$ de **candidatos**, entradas del problema.
- Función **solución**. Comprueba, en cada paso, si el subconjunto actual de candidatos elegidos forma una solución (no importa si es óptima o no lo es).




*Un algoritmo voraz determina el mínimo número de monedas que debe devolverse en el cambio. En la figura se muestran los passo que un ser humano debería seguir para emular a un algoritmo voraz para acumular 36 céntimos usando sólo monedas de valores nominales de 1, 5, 10 y 20. La moneda del mayor valor menor que el resto debido es el óptimo local en cada paso. Nótese que en general el problema de devolución del cambio requiere programación dinámica o porgroamación lineal para encontrar una solución óptima. Sin embargo, en muchos sistemas monetarios, incluyendo el euro y el dolar estadonidense, son casos especiales donde el la estrategia del algoritmo voraz da con la solución óptima.*

- Función de **selección**. Informa de cuál es el elemento más prometedor para completar la solución. Éste no puede haber sido escogido con anterioridad. Cada elemento es considerado una sola vez. Luego, puede ser rechazado o aceptado y pertenecerá a $C \setminus S$ .
- Función de **factibilidad**. Informa si a partir de un conjunto se puede llegar a una solución. Lo aplicaremos al conjunto de seleccionados unido con el elemento más prometedor.
- Función **objetivo**. Es aquella que queremos maximizar o minimizar, el núcleo del problema.

#### Funcionamiento

El algoritmo escoge en cada paso al mejor elemento $x \in C$ posible, conocido como el **elemento más prometedor**. Se elimina ese elemento del conjunto de candidatos ( $C \leftarrow C \setminus \{x\}$ ) y, acto seguido, comprueba si la inclusión de este elemento en el conjunto de elementos seleccionados ( $S \cup \{x\}$ ) produce una solución factible.

En caso de que así sea, se incluye ese elemento en $S$ . Si la inclusión no fuera factible, se descarta el elemento. Iteramos el bucle, comprobando si el conjunto de seleccionados es una solución y, si no es así, pasando al siguiente elemento del conjunto de candidatos.

### 2.12.2 Ejemplos de algoritmos voraces

- Algoritmo de Kruskal
- Algoritmo de Prim

- Algoritmo de Dijkstra
- Algoritmo para la ubicación óptima

### 2.12.3 Temas relacionados

- Algoritmo
- Algoritmo heurístico

### 2.12.4 Referencias

- Brassard, Gilles; Bratley, Paul (1997). «Algoritmos voraces». *Fundamentos de Algoritmia*. Madrid: PRENTICE HALL. ISBN 84-89660-00-X.

### 2.12.5 Enlaces externos

Wikilibros

- Wikilibros alberga un libro o manual sobre **Algoritmia/Algoritmos voraces**.

En inglés:

- Definición del NIST

## 2.13 Problema del viajante

EL **Problema del Agente Viajero** (**TSP** por sus siglas en inglés) o problema del viajante, responde a la siguiente pregunta: Dada una lista de ciudades y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y regresa a la ciudad origen? Este es un problema NP-duro dentro en la optimización combinatoria, muy importante en la investigación de operaciones y en la ciencia de la computación.

El problema fue formulado por primera vez en 1930 y es uno de los problemas de optimización más estudiados. Es usado como prueba para muchos métodos de optimización. Aunque el problema es computacionalmente complejo, una gran cantidad de heurísticas y métodos exactos son conocidos, de manera que, algunas instancias desde cien hasta miles de ciudades pueden ser resueltas.

El TSP tiene diversas aplicaciones aún en su formulación más simple, tales como: la planificación, la logística y en la fabricación de microchips. Un poco modificado, aparece como: un sub-problema en muchas áreas, como en la secuencia de ADN. En esta aplicación, el concepto de "ciudad" representa, por ejemplo: clientes, puntos de soldadura o fragmentos de ADN y el concepto de "distancia" representa el tiempo de viaje o costo, o una medida de similitud entre los fragmentos de ADN. En muchas aplicaciones, restricciones adicionales como el límite de recurso o las ventanas de tiempo hacen el problema considerablemente difícil. El TSP es un caso especial de los Problemas del Comprador Viajante (travelling purchaser problem).

En la teoría de la complejidad computacional, la versión de decisión del TSP (donde, dado un largo "L", la tarea es decidir cuál grafo tiene un camino menor que L) pertenece a la clase de los problemas NP-completos. Por tanto, es probable que en el caso peor el tiempo de ejecución para cualquier algoritmo que resuelva el TSP aumente de forma exponencial con respecto al número de ciudades.

### 2.13.1 Historia

El origen de los problemas del agente viajero no está claro. Una guía para agentes viajeros de 1832 menciona el problema e incluye ejemplos de viajes a través de Alemania y Suiza, pero no contiene un tratamiento matemático del mismo.[1]



*William Rowan Hamilton*

El problema del agente viajero fue definido en los años 1800s por el matemático irlandés W. R. Hamilton y por el matemático británico Thomas Kirkman. El Juego Icosian de Hamilton fue un puzzle recreativo basado en encontrar un ciclo de Hamilton.[2] Todo parece indicar que la forma general del TSP fue estudiada, por primera vez por matemáticos en Viena y Harvard, durante los años 1930s. Destacándose Karl Menger, quien definió los problemas, considerando el obvio algoritmo de fuerza bruta, y observando la no optimalidad de la heurística de vecinos más cercanos:

> Denotamos por "Problema del Mensajero" (dado que en la práctica esta pregunta puede resolverse por cada cartero, aunque puede ser resuelta por varios viajeros) la pregunta es encontrar, para un conjunto finito de puntos de los cuales se conocen las distancias entre cada par, el camino más corto entre estos puntos. Por supuesto, el problema es resuelto por un conjunto finito de intentos. La regla que se debe seguir es que desde el punto inicial se va al punto más cercano a este, de ahí a su más cercano y así

sucesivamente, en general este algoritmo no retorna la ruta más corta.[3]

Hassler Whitney de la Universidad de Princeton introdujo el nombre “travelling salesman problema” poco después.[4]

Durante los años 1950 a 1960, el problema fue incrementando su popularidad entre el círculo de científicos de Europa y Estados Unidos. Una notable contribución fue la de George Dantzig, Delbert Ray Fulkerson y Selmer M. Johnson de la Corporación RAND en Santa Mónica, quienes expresaron el problema como Programación Lineal en Enteros y desarrollaron para solucionarlo el método de Planos Cortantes. Con este nuevo método, resolvieron una instancia con 49 ciudades, óptimamente, mediante la construcción de un recorrido y probando que no había un recorrido que pudiera ser más corto. En las siguientes décadas, el problema fue estudiado por muchos investigadores, matemáticos, científicos de la computación, químicos, físicos, etc.

Richard M. Karp mostró en 1972 que el Problema del Ciclo de Hamilton era un problema NP-completo, lo cual implica que el TSP sea un problema NP-duro. Esto tiene su explicación matemática por la evidente dificultad computacional para encontrar recorridos óptimos.

Gran progreso tuvo a finales de los 70s y principios de los 80s, donde Grötschel, Padberg, Rinaldi y otros, manejaron soluciones exactas para instancias con 2392 ciudades, usando Planos Cortantes y Ramificación y Acotación.

En los 90s, Applegate, Bixby, Chvátal, y Cook desarrollaraon el programa Concorde, el cual es usado en muchos de los registros de soluciones recientes. Gerhard Reinelt publicó el TSPLIB en 1991, una colección de instancias de pruebas de dificultad variable, la cual es usada por muchos grupos investigativos para comparar resultados. En 2006, Cook y otros, obtuvieron un recorrido óptimo para 85,900 ciudades dado por un problema de diseño de microchip, actualmente TSPLIB es la instancia más larga resuelta. Para otras muchas instancias con millones de ciudades, la solución puede ser encontrada garantizando que la solución contiene un 2-3% del recorrido óptimo.[5]

### 2.13.2 Descripción

#### Caso práctico

En el problema se presentan N! rutas posibles, aunque se puede simplificar ya que dada una ruta nos da igual el punto de partida y esto reduce el número de rutas a examinar en un factor N quedando (N-1)!. Como no importa la dirección en que se desplace el viajante, el número de rutas a examinar se reduce nuevamente en un factor 2. Por lo tanto, hay que considerar (N-1)!/2 rutas posibles.

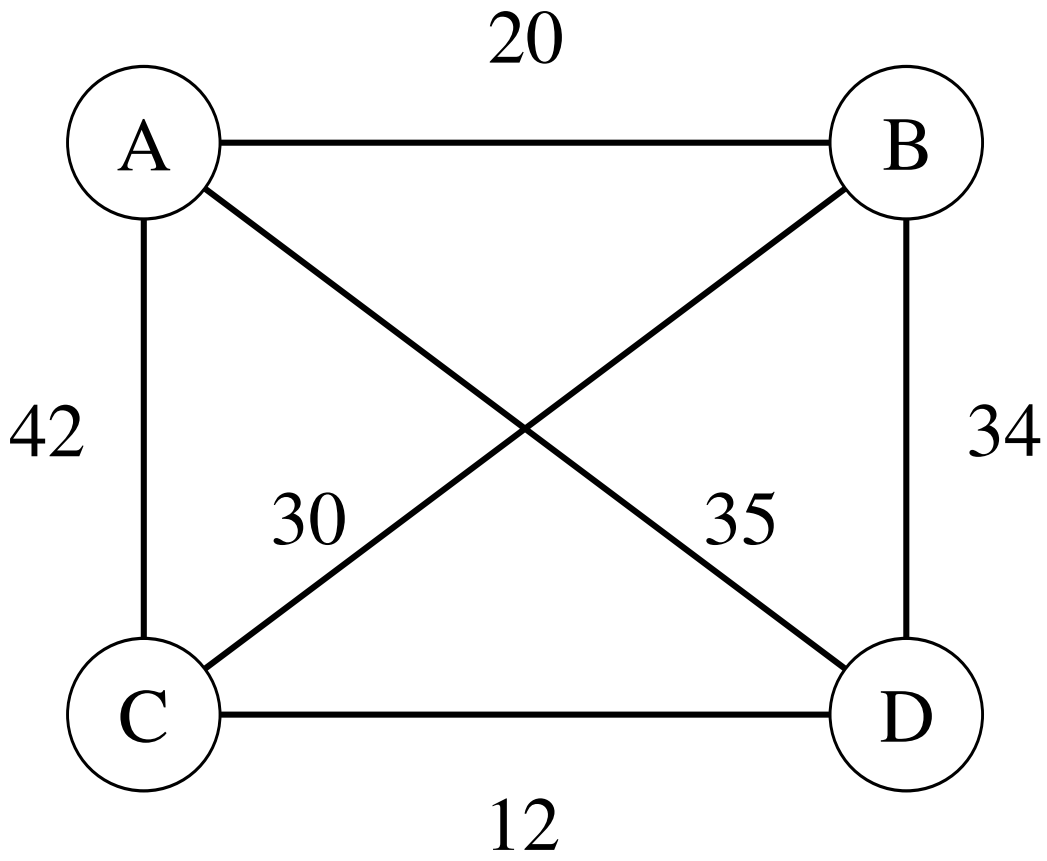
En la práctica, para un problema del viajante con 5 ciudades hay 12 rutas diferentes y no necesitamos un ordenador para encontrar la mejor ruta, pero apenas aumentamos el número de ciudades las posibilidades crece exponencialmente (en realidad, factorialmente):

- Para 10 ciudades hay 181.440 rutas diferentes
- Para 30 ciudades hay más de 4·10^31 rutas posibles. Un ordenador que calcule un millón de rutas por segundo necesitaría 10^18 años para resolverlo. Dicho de otra forma, si se hubiera comenzado a calcular al comienzo de la creación del universo (hace unos 13.400 millones de años) todavía no se habría terminado.

Puede comprobarse que por cada ciudad nueva que incorporemos, el número de rutas se multiplica por el factor N y crece exponencialmente (factorialmente). Por ello el problema pertenece a la clase de problemas NP-completos.

#### Como un problema de grafos

El TSP puede ser modelado como un grafo ponderado no dirigido, de manera que las ciudades sean los vértices del grafo, los caminos son las aristas y las distancias de los caminos son los pesos de las aristas. Esto es un problema de minimización que comienza y termina en un vértice especifico y se visita el resto de los vértices exactamente una vez. Con frecuencia, el modelo es un grafo completo (cada par de vértices es conectado por una arista). Si no existe camino entre un par de ciudades, se añade arbitrariamente una arista larga para completar el grafo sin afectar el recorrido óptimo.




*TSP simétrico con 4 ciudades*

### Asimétrico y simétrico

En el "'TSP simétrico"', la distancia entre un par de ciudades es la misma en cada dirección opuesta, formando un grafo no dirigido. Este problema reduce a la mitad el número de soluciones posibles. En el "'TSP asimétrico"', pueden no existir caminos en ambas direcciones o las distancias pueden ser diferentes, formando un grafo dirigido. Los accidentes de tráfico, las calles de un solo sentido y las tarifas aéreas para ciudades con diferentes costos de partida y arribo, son ejemplos de cómo esta simetría puede ser quebrada.

### Problemas relacionados

- Una formulación equivalente en términos de Teoría de grafos es: dado una grafo ponderado completo (donde los vértices representan las ciudades, las aristas representan los caminos y los pesos son el costo o las distancias de estos caminos), encontrar un ciclo de Hamilton con menor peso.

- Las restricciones de retorno a la ciudad de comienzo no cambia la complejidad computacional del problema, vea Problema del camino de Hamiltoniano.

- Otro problema relacionado es el Problema del agente viajero con cuello de botella (bottleneck TSP): Encontrar un ciclo de Hamilton en un grafo ponderado con el mínimo peso de las aristas más pesadas. El problema es de una considerable importancia en la práctica, en las áreas de la transportación y la logística. Un ejemplo clásico es en la fabricación de circuitos impresos: planificando una ruta de la máquina perforadora para perforar los huecos en un PCB. Otras son, las aplicaciones de perforado o maquinado en la robótica: las "ciudades" son los huecos de diferentes tamaños a perforar y el "costo de viaje" incluye el tiempo para reequipar el robot (problema del secuenciado del trabajo de una máquina simple).[6]

- La generalización del TSP trata con "estados" que tienen (una o más) "ciudades" y el viajante tiene que visitar exactamente una ciudad de cada "estado". También se conoce como el Problema del Político Viajero. Como una aplicación se considera, el perforado en la fabricación de semiconductores, vea e.g., Patente USPTO nº 7054798. Sorprendentemente, Behzad y Modarres demostraron que el problema generalizado del viajante puede ser transformado en el problema del viajante estándar con el mismo número de ciudades, pero modificando la Matriz de distancias

- El problema de ordenamiento secuencial trata con el problema de visitar un conjunto de ciudades donde se tiene en cuenta las relaciones de precedencias entre las ciudades.

- El problema del viajante comprador trata con un comprador que está cargado con un conjunto de productos. Él puede comprar estos productos en varias ciudades, pero tienen diferentes precios y no todas las ciudades ofrecen los mismos productos. El objetivo es encontrar una ruta entre un subconjunto de ciudades, los cuales minimicen el costo total (costo de viaje + costo de la compra) y habilite la compra de todos los productos requeridos.

### 2.13.3 Formulación de la programación lineal en enteros

El TSP puede ser formulado por la programación lineal en enteros.[7][8][9] Sea $x_{ij}$ igual 1, si existe el camino de ir de la *i* a la ciudad *j*, y 0 en otro caso, para el conjunto de ciudades 0,..., *n*. Sean $u_i$ para *i* = 1,..., *n* variables artificiales y sea $c_{ij}$ la distancia desde la ciudad *i* a la ciudad *j*. Entonces el modelo de programación lineal en enteros puede ser escrito como:

$$
\begin{aligned}
&\min \sum_{i=0}^{n} \sum_{j \neq i, j=0}^{n} c_{ij} x_{ij} \\
&0 \leq x_{ij} \leq 1 && i, j = 0, \ldots n \\
&x_{ij} \text{integer} && i, j = 0, \ldots n \\
&\sum_{i=0, i \neq j}^{n} x_{ij} = 1 && j = 0, \ldots, n \\
&\sum_{j=0, j \neq i}^{n} x_{ij} = 1 && i = 1, \ldots, n \\
&u_i - u_j + n x_{ij} \leq n - 1 && 1 \leq i \neq j \leq n.
\end{aligned}
$$

El primer conjunto de igualdades asegura que cada ciudad 0,..., *n* de salida llegue exactamente a una ciudad, y el segundo conjunto de igualdades aseguran que desde cada ciudad 1,..., *n* se salga exactamente hacia una ciudad (ambas restricciones también implican que exista exactamente una salida desde la ciudad 0.) La última restricción obliga a que un solo camino cubra todas las ciudades y no dos o más caminos disjuntos cubran conjuntamente todas las ciudades. Para probar esto se muestra en (1) que toda solución factible contiene solamente una secuencia cerrada de ciudades, y en (2) que para cada uno de los recorridos que cubren todas las ciudades, hay valores para todas las variables $u_i$ que satisfacen las restricciones.

Para probar que cada solución factible contiene solamente una secuencia cerrada de ciudades, es suficiente mostrar que cada sub-ruta en una solución factible pasa a través de la ciudad 0 (note que las igualdades aseguran que solamente pude haber un recorrido de ese tipo). Por tanto, si sumamos todas las desigualdades correspondiente a $x_{ij} = 1$ para cada sub-ruta de k pasos que no pasan a través de la ciudad 0, obtenemos $nk \leq (n-1)k$, lo cual es una contradicción.

Ahora, mostramos que para cada recorrido que cubre todas las ciudades, hay valores de las variables $u_i$ que satisfacen las restricciones.

Sin pérdida de generalidad, se define el recorrido con origen y fin en la ciudad 0. Escoger $u_i = t$ si la ciudad *i* es visitada en el paso *t* (*i*, *t* = 1, 2,..., n). Entonces $u_i - u_j \leq n - 1$ dado $u_i$ no puede ser mayor que *n* y $u_j$ no puede ser menor que 1; por lo tanto las restricciones se satisfacen siempre que $x_{ij} = 0$. Para $x_{ij} = 1$, $u_i - u_j + n x_{ij} = (t) - (t + 1) + n = n - 1$, se satisfacen las restricciones.

### 2.13.4 Calculando una solución

Las líneas tradicionales para atacar un problema NP-duro son las siguientes:

- Formular algoritmos para encontrar soluciones exactas (estos trabajan más rápidos en problemas con dimensiones pequeñas).
- Formular algoritmos heurísticos o "subóptimos" (por ejemplo: algoritmos que den aparentemente o probablemente buenas soluciones, pero no devuelven el óptimo).
- Encontrar los casos especiales para el problema ("subproblemas") para los cuales heurísticas mejores o algoritmos exactos son posibles.

#### Complejidad Computacional

El problema es NP-duro, y la versión del Problema de decisión ("dado el costo y un número x, decidir cuál es la ruta de viaje más barata que x") es NP-completo. El problema del viajante con cuello de botella es también NP-duro. El problema sigue siendo NP-duro aún para los casos donde las ciudades están en el plano con distancias Euclidianas, al igual que en otros casos restrictivos. Eliminando la condición de visitar cada ciudad exactamente una vez no se elimina la condición de problema NP-duro, ya que se ve fácilmente que en el caso planal hay un recorrido óptimo que visita cada ciudad una sola vez (de otra manera, por desigualdades triangulares, un atajo que se salta una visita repetida no incrementa la longitud del camino).

**Complejidad de una aproximación** En el caso general, encontrar el camino más corto del viajante es NP-completo. Si la medida de distancia es una métrica y es simétrica, el problema se convierte en APX-completo[10] y el Algoritmo de Christofides lo aproximan a 1.5.[11]

Si las distancias son restringidas a 1 y 2 (pero aun es una métrica) la proporción aproximada es de 8/7.[12] En el caso que sea asimétrico, es conocido solamente un rendimiento logarítmico, el mejor algoritmo actual logra una proporción de 0.814 log *n*;[13] esta es una pregunta abierta, si existe un factor constante de aproximación.

El correspondiente problema de aproximación, de encontrar el recorrido más largo del viajante es aproximable a 63/38.[14] Si la función de distancia es simétrica, el camino más largo puede aproximarse a 4/3 por una algoritmo determinista[15] y a $(33 + \epsilon)/25$ por un algoritmo aleatorio.[16]

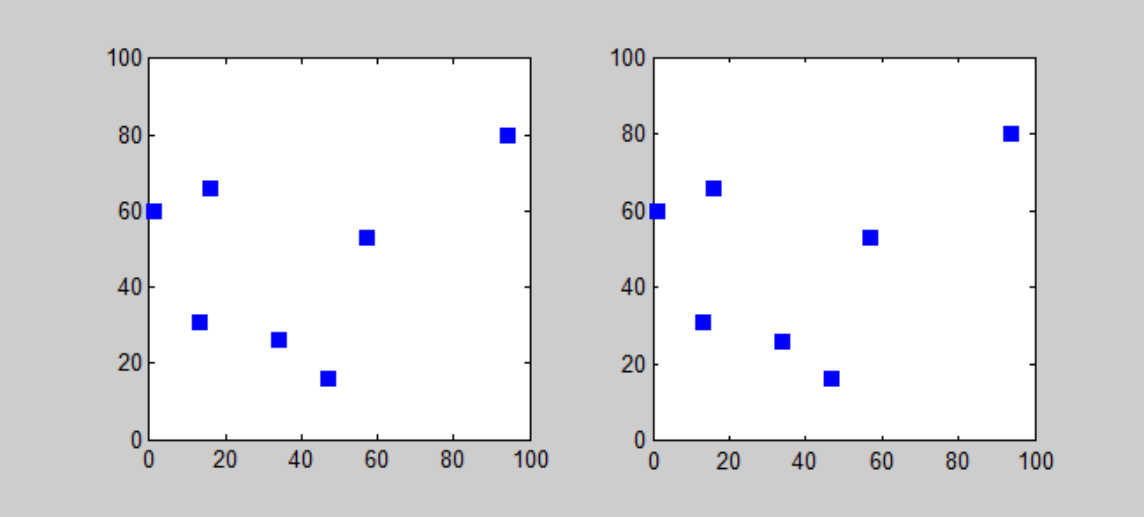
#### Algoritmos Exactos

La solución más directa puede ser, intentar todas las permutaciones (combinaciones ordenadas) y ver cuál de estas es la menor (usando una Búsqueda de fuerza bruta). El tiempo de ejecución es un factor polinómico de orden $O(n!)$ , el Factorial del número de ciudades, esta solución es impracticable para dado solamente 20 ciudades. Una de las mejores aplicaciones de la Programación dinámica es el algoritmo Held–Karp que resuelve el problema en $O(n^2 2^n)$ .[17]

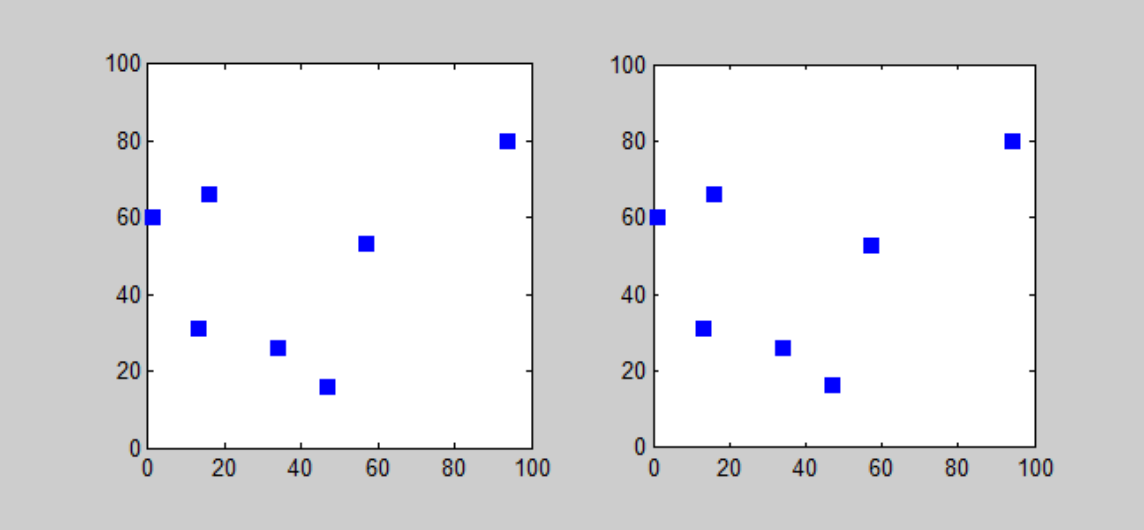
La mejora de estos límites de tiempos es difícil. Por ejemplo, no ha sido determinado si existe un algoritmo para el TSP que corra en un tiempo de orden $O(1.9999^n)$ [18]

Otras aproximaciones incluyen:

- Varios algoritmos de ramificación y acotación, los cuáles pueden ser usados para procesar TSP que contienen entre 40 y 60 ciudades.
- Algoritmos de mejoras progresivas (iterativas) los cuales utilizan técnicas de Programación lineal. Trabajan bien para más de 200 ciudades.
- Implementaciones de ramificación y acotación y un problema específico de generación de cortes (Ramificación y poda); este es el método elegido para resolver grandes instancias. Esta aproximación retiene el record vigente, resolviendo una instancia con 85,900 ciudades, vea Applegate *et al.* (2006).




*Solución de un TSP con 7 ciudades usando algoritmo fuerza bruta. Nota: Número de permutaciones: (7-1)!/2 = 360*




*Solución de un TSP con 7 ciudades usando un simple algoritmo de ramificación y acotación. Nota: El número de permutaciones es mucho menor que el de la búsqueda Fuerza Bruta*

Una solución exacta para 15,112 pueblos alemanes desde TSPLIB fue encontrada en 2001 usando el método de planos cortantes propuesto por George Dantzig, Ray Fulkerson, y Selmer M. Johnson en 1954, basados en la programación lineal. Los cálculos fueron realizados por una red de 110 procesadores ubicados en la Universidad Rice y en la Universidad de Princeton (vea el enlace externo Princeton). El tiempo total de cálculo fue equivalente a 22.6 años en un Procesador alpha de 500 MHz. En mayo de 2004, el problema del viajante de visitar todos los 24,978 poblados en Sweden fue resuelto: un recorrido de tamaño aproximado de 72,500 kilómetros fue encontrado y se probó que no existía un camino menor.[19]

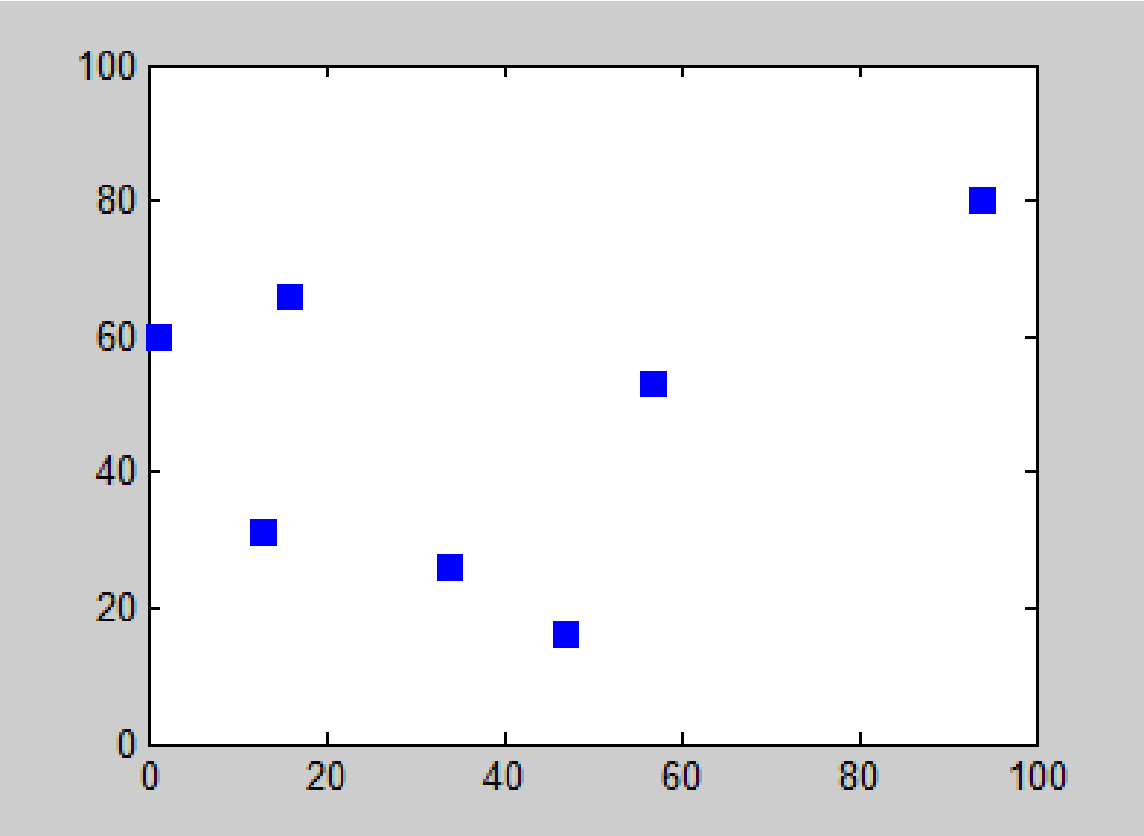
En marzo de 2005, el problema del viajante de visitar todos los 33,810 puntos en una tabla de circuitos fue resuelto usando *Concorde TSP Solver*: un recorrido de 66,048,945 unidades fue encontrado y se probó que no existía un recorrido menor. El cálculo tomo aproximadamente 15.7 años - CPU (Cook et al. 2006). En abril de 2006, una instancia con 85,900 puntos fue resuelta usando ´´Concorde TSP Solver´´, tomando 136 años- CPU, vea Applegate *et al.* (2006).

### Algoritmos Heurísticos y aproximados

Varios algoritmos heurísticos y aproximados que retornan rápidamente buenas soluciones han sido creados. Métodos modernos pueden encontrar soluciones para problema extremadamente largos (millones de ciudades) en un tiempo

razonable.[5]

Varias categorías de heurísticas son reconocidas:




*Algoritmo del vecino más próximo para un TSP con 7 ciudades. La solución cambia cuando el punto de inicio es cambiado*

**Heurísticas Constructivas** El Algoritmo del vecino más próximo (NN por sus siglas en inglés) o también llamado algoritmo glotón (greedy) permite al viajante elegir la ciudad no visitada más cercana como próximo movimiento. Este algoritmo retorna rápidamente una ruta corta. Para N ciudades aleatoriamente distribuidas en un plano, el algoritmo en promedio, retorna un camino de un 25% más largo que el menor camino posible.[20] Sin embargo, existen muchos casos donde la distribución de las ciudades dada hace que el algoritmo NN devuelva el peor camino (Gutin, Yeo, and Zverovich, 2002). Esto es verdad lo mismo para TSP simétricos que asimétricos (Gutin and Yeo, 2007). Rosenkrantz et al. [1977] mostró que el algoritmo NN tiene un factor de aproximación de orden $\Theta(\log |V|)$ para instancias que satisfacen la desigualdad triangular. Una variación del algoritmo NN, llamada operador de Fragmento más cercano (NF por sus siglas en inglés) la cual conecta un grupo (fragmento) de ciudades no visitadas más cercanas, puede encontrar la ruta más corta con iteraciones sucesivas.[21] El operador NF puede ser aplicado también, en la obtención de soluciones iniciales para el algoritmo NN para además ser mejorado en un modelo elitista, donde solamente son aceptadas las mejores soluciones.

Las construcciones basadas en un árbol de expansión mínima (minimum spanning tree) tienen una proporción de aproximación de 2. El Algoritmo de Christofides logra una proporción de 1.5.

Match Twice and Stitch (MTS) (Kahng, Reda 2004[22]), es otra heurística constructiva, que realiza dos comparaciones secuenciales (matchings), donde el segundo macheo es ejecutado después de eliminar todas las aristas del primer macheo, retornando un conjunto de ciclos. Los ciclos son unidos para producir el camino final.

### Mejora Iterativa

**Intercambio par a par** El intercambio par a par o técnica *2-opt* involucra en cada iteración la eliminación de dos aristas y el reemplazo de estas con dos aristas diferentes que reconecten los fragmentos creados por la eliminación de las aristas produciendo un camino nuevo más corto. Esto es un caso especial del método k-opt. Note

que, la etiqueta Lin–Kernighan es un nombre erróneo para el 2-opt muchas veces utilizado. Lin–Kernighan es realmente el método más general de k-opt.

**Heurística *k*-opt o heurística Lin-Kernighan** Toma un recorrido dado y elimina k aristas mutuamente disjuntas. Reconecta los fragmentos conformando el recorrido, sin dejar ningún subcamino disjunto (es decir, no conectar los dos extremos de un mismo fragmento). Esto, en efecto, simplifica las consideraciones del TSP convirtiéndolo en un problema más simple. Cada extremo de un fragmento tiene $2k - 2$ posibilidades de ser conectado: del total de $2k$ extremos de fragmentos disponibles, los dos extremos del fragmento que se está considerando son descartados. Tal restricción de $2k$- ciudades puede ser resulta con un método de fuerza bruta para encontrar el menor costo de reconectar los fragmentos originales. La técnica *k*-opt es un caso especial del *V*-opt o técnica Variable-opt. El método más popular del *k*-opt es 3-opt, y fue introducido por Shen Lin de Laboratorios Bell en 1965. Este es un caso especial de 3-opt donde las aristas son no disjuntas (dos de las aristas son adyacentes a la otra). En la práctica, es a menudo posible alcanzar mejoras sustanciales sobre el 2-opt sin el costo combinatorio del 3-opt general, restringiendo el 3-changes a este subconjunto especial en el cual dos de las aristas eliminadas son adyacentes. Este es llamado el dos y medio – opt (two-and-a-half-opt), típicamente cae a mitad entre el 2-opt y el 3-opt en términos de calidad del recorrido encontrado y el tiempo para encontrarlo.

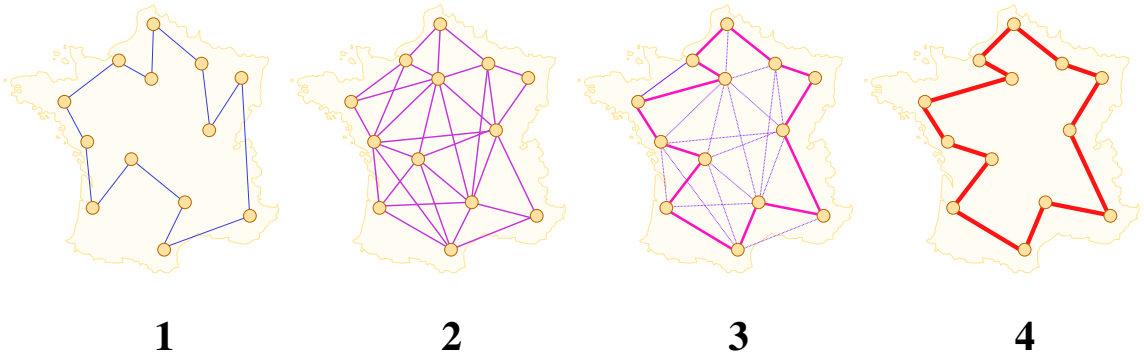
**Heurística *V*-opt** El método variable-opt está relacionado y es una generalización del método *k*-opt. Mientras el método *k*-opt elimina un número fijo k de aristas del recorrido original, el método variable-opt no fija el tamaño del conjunto de aristas eliminadas. En cambio, este método aumenta el conjunto a medida que el proceso de búsqueda continúa. El mejor método conocido en esta familia es el método Lin-Kernighan. Shen Lin y Brian Kernighan publicaron por primera vez este método en 1972, y fue la heurística más confiable para resolver el problema del viajante por aproximadamente dos décadas. Otros avances de los métodos variable-opt fueron desarrollados en Laboratorios Bell a finales de los 80s por David Johnson y su equipo de investigación. Estos métodos (a veces llamados Lin-Kernighan-Johnson) basados en el método Lin-Kernighan añaden ideas de la Búsqueda tabú y la Computación evolutiva. La técnica básica Lin-Jernighan da resultados que garantizan al menos el 3-opt. Los métodos Lin-Kernighan-Johnson calculan un camino de Lin-Kernighan y entonces perturban el camino por lo que ha sido descrito como una mutación que elimina las últimas cuatro aristas y reconecta el camino de una forma diferente y luego se aplica v-opt al nuevo camino. La mutación es a menudo suficiente para mover el camino desde el mínimo local identificado por Lin-Kernighan. Los métodos V-opt son considerados como la heurística más ponderosa para el problema y es capaz de enfrentar casos especiales, como el Problema del Ciclo de Hamilton y otros TSP no métricos para las cuales otras heurísticas fallan. Por muchos años Lin-Kernighan-Johnson ha identificado soluciones óptimas para todos los TSP donde una solución óptima era conocida y ha identificado la mejor solución para otros TSP sobre los cuales el método ha sido aplicado.

**Mejoras Aleatorias** Los algoritmos optimizados de cadenas de Márkov que usan heurísticas de búsquedas locales como sub-algoritmos pueden encontrar una ruta extremadamente cerca de la ruta óptima para instancias de 700 a 800 ciudades. EL TSP es la base para muchas heurísticas diseñadas para la optimización combinatoria como: los algoritmos genéticos, el recocido simulado, la Búsqueda tabú, la optimización por colonias de hormigas, la Inteligencia de enjambre, entre otras.

**Optimización por Colonia de Hormigas** El investigador de Inteligencia artificial, Marco Dorigo describió en 1997 un método que genera heurísticamente "buenas soluciones" para el TSP usando una simulación de una colonia de hormigas llamada ACS (Ant Colony System).[23] El cual modela el comportamiento observado en las hormigas reales de encontrar caminos cortos entre las fuentes de comida y su nido, emergió como un comportamiento la preferencia de cada hormiga de seguir el sendero de [Feromonas]] depositado por las otras hormigas.

ACS envía un gran número de hormigas (agentes virtuales) para explorar las posibles rutas en el mapa. Cada hormiga elige probabilísticamente la próxima ciudad a visitar basada en una heurística, combinando la distancia a la ciudad y la cantidad de feromonas depositadas en la arista hacia la ciudad. La hormiga exploradora, deposita feromonas en cada arista que ella cruce, hasta que complete todo el camino. En este punto la hormiga que completó el camino más corto deposita feromonas virtuales a lo largo de toda la ruta recorrida (actualización del camino global). La cantidad de feromonas depositadas es inversamente proporcional a la longitud del camino: el camino más corto, tiene más cantidad de feromonas.

**Casos Especiales**






*Algoritmo de optimización por Colonia de Hormigas para el TSP con 7 ciudades: Las líneas rojas y gruesas en el mapa de feromonas indican presencia de más feromonas*

**TSP métrico** En el "TSP métrico", también conocido como *delta-TSP* o Δ-TSP, las distancias entre las ciudades satisfacen la Desigualdad triangular.

Una restricción muy natural para el TSP es que las distancias entre las ciudades formen una Métrica que satisfagan las desigualdades triangulares; que significa que la conexión desde *A* hasta *B* no sea mayor que la ruta de "A" a "B" pasando por *C*:

$$d_{AB} \leq d_{AC} + d_{CB}$$

Las aristas se expanden y construyen una Métrica para el conjunto de vértices. Cuando las ciudades son vistas como puntos en el plano aparecen varias funciones de distancia y muchas instancias del TSP satisfacen los requerimientos de las mismas.

Los siguientes son ejemplos de TSP métricos para varias definiciones de las funciones de distancias:

- En el TSP Euclidiano la distancia entre dos ciudades es la Distancia euclidiana entre los puntos correspondientes.
- En el TSP rectilíneo la distancia entre dos ciudades es la suma de las diferencias de las coordenadas "x" e "y" respectivamente. Esta métrica es generalmente llamada Distancia de Manhattan.
- En la métrica máxima, la distancia entre dos puntos es el máximo de los valores absolutos de las diferencias de las coordenadas "x" e "y".

Las dos últimas métricas aparecen, por ejemplo, enrutando una máquina que perfora un conjunto dado de huecos en circuitos impresos (PCB por sus siglas en inglés). La métrica Manhattan corresponde a la máquina que ajusta a una

máquina que ajusta en primer lugar, la primera coordenada y luego la otra, así el tiempo de movimiento al nuevo punto es la suma de ambos movimientos. La métrica máxima corresponde a una máquina que ajusta ambas coordenadas simultáneamente, así el tiempo de moverse al nuevo punto es más despacio que los otros dos movimientos. En esta definición, el TSP no permite visitar ciudades dos veces, pero muchas aplicaciones no necesitan esta restricción. En tales casos, una instancia simétrica, no métrica puede ser reducida a una instancia con métrica. Este remplaza el grafo original con un grafo completo en el cual las distancias entre ciudades $d_{AB}$ es reemplazada por el camino más corto entre $A$ y $B$ en el grafo original.

La expansión del árbol de expansión mínima de la red $G$ is a natural es un límite inferior natural para expandir la ruta óptima, porque eliminando cualquier arista de la ruta óptima, devuelve un Camino de Hamilton, el cual es un árbol de expansión en $G$ . En el TSP con desigualdades triangulares es posible probar en términos de límites superiores del árbol de expansión mínima y diseñar un algoritmo que haga una verificación de los límites superiores en la expansión de la ruta. El primer ejemplo publicado (y el más simple) es el siguiente:

1. Construir un árbol de expansión mínima $T$ para $G$ .

2. Duplicar todas las aristas de $T$ . De manera que, dondequiera que haya una arista de *u* a *v*, adicionar una segunda arista de *v* a *u*. Devolviendo un Grafo Euleriano $H$ .

3. Encontrar un Circuito Euleriano $C$ en $H$ . Claramente, esta expansión es dos veces la expansión del árbol.

1. Convertir el circuito euleriano $C$ de $H$ en un ciclo hamiltoniano de $G$ de la siguiente manera: caminar a lo largo de $C$ , y cada vez que caiga en un vértice ya visitado, salte de él y trate de ir al próximo (a lo largo de $C$ ).

Es fácil probar que el último paso funciona. Además, gracias a la desigualdad triangular, cada salto del paso 4 es en efecto un camino corto, por ejemplo, la longitud del ciclo no se incrementa. Por lo cual, para un recorrido del TSP este no es más largo que el doble del recorrido óptimo.

El Algoritmo de Christofides sigue un diseño similar pero combina el árbol de expansión mínima con una solución de otro problema, mínimo pero del macheo perfecto. Esto retorna un camino del TSP que es a lo sumo 1.5 veces el óptimo. El algoritmo de Christofides fue una de los primeros algoritmos de aproximación, y fue en parte responsable por la imagen que se le dio a los algoritmos de aproximación como un acercamiento práctico a los problemas intratables. Como un hecho importante se tiene que, el término “algorithm” no fue comúnmente extendido a algoritmos de aproximación hasta más adelante el algoritmo Christofides fue inicialmente referido como la Heurística Christofides.

**TSP Euclidiano** El **TSP Euclidiano**, o **TSP planal**, es el TSP que utiliza la Distancia euclidiana.

El TSP euclidiano es en particular un caso del TSP métrico, dado que las distancias en un plano cumplen la desigualdad triangular.

Como el TSP general, el TSP Euclidiano es NP-duro. El problema con métrica discretizada (distancia redondeada por exceso a un entero), es NP-completo. Sin embargo, con respecto a esto es más fácil que el TSP métrico general. Por ejemplo, el árbol de expansión mínima del grafo asociado con una instancia del TSP Euclidiano es un árbol de expansión mínima euclidiano, y puede calcularse en un tiempo de O($n$ log $n$) para $n$ Sin embargo, con respecto a esto es más fácil que el TSP métrico general. Por ejemplo, el árbol de expansión mínima del grafo asociado con una instancia del TSP Euclidiano es un árbol de expansión mínima euclidiano, y puede calcularse en un tiempo de 2- aproximación simple para el TSP con desigualad triangular anterior, operar más rápidamente. En general, para cualquier $c > 0$, donde $d$ es el número de dimensiones en el espacio Euclidiano, existe un algoritmo polinomial que encuentra un camino de longitud a lo sumo $(1 + 1/c)$, el óptimo para instancia geométricas del TSP se da en tiempo $O\left(n(\log n)^{(O(c\sqrt{d}))^{d-1}}\right)$ ; este es llamado un esquema de aproximación a tiempo polinomial (PTAS por sus siglas en inglés). Sanjeev Arora y Joseph S. B. Mitchell se adjudicaron el Premio Gödel en 2010 por el descubrimiento de un PTAS para el TSP Euclidiano. En la práctica, heurísticas con pocas garantías continúan siendo usadas.

**TSP Asimétrico** En la mayoría de los casos, la distancia entre dos nodos en red del TSP es la misma en ambas direcciones. El caso donde la distancia de *A* a *B* no es igual que la distancia de *B* a *A* es llamado asimétrico. Una aplicación práctica de un TSP asimétrico es la optimización de rutas usando un enrutamiento calle-nivel (el cual es asimétrico por calles de un solo nivel, carreteras deslizantes, caminos de motos, etc.).

**Resolver por conversión al TSP simétrico** Resolver un grafo del TSP asimétrico puede ser algo complicado. Lo siguientes es una matriz de 3×3 que contiene todos los caminos ponderados entre los nodos *A*, *B* y *C*. Una opción es cambiar una matriz asimétrica de tamaño *N* por una matriz simétrica de tamaño 2*N*.[24]

Doblar el tamaño, cada nodo en el grafo es duplicado, creando un segundo *nodo fantasma*. Usando los nodos duplicados con pesos muy bajos, como −∞, proporciona rutas baratas con enlaces hacia atrás, al nodo real y permiten continuar la evaluación simétrica. La matriz original de 3×3 mostrada anteriormente es visible en la parte inferior izquierda y el inverso de la original en la parte superior derecha. Ambas copias de la matriz tienen sus diagonales reemplazadas por el menor costo de los caminos saltados, representados por −∞.

La matriz original de 3×3 produce dos ciclos hamiltonianos (un camino que visita todos los nodos una vez), particularmente *A*-*B*-*C*-*A* [costo 9] y *A*-*C*-*B*-*A* [costo 12]. Evaluando la versión simétrica de 6×6, el mismo problema ahora produce más caminos, incluyendo *A*-*A′*-*B*-*B′*-*C*-*C′*-*A*, *A*-*B′*-*C*-*A′*-*A*, *A*-*A′*-*B*-*C′*-*A* [todos los costos 9 – ∞].

Un tema importante sobre cada nueva secuencia se forma alternando los nodos (*A*, *B*, *C*) y sus simétricos (*A′*,*B′*,*C′*) y el enlace para ¨saltar¨ entre cualquier par relacionado (*A*-*A′*) es efectivamente libre. Una versión para el algoritmo puede usar cualquier peso para el camino *A*-*A′*, mientras que el peso sea menor que todos los otros pesos presentes en el grafo. Como el peso del camino *A*-*A′* es libre, el valor cero puede ser usado para representar su costo, si cero no está siendo usado para otro propósito (como el de designar caminos inválidos). En los dos ejemplos anteriores, no existen caminos entre los nodos, estos son mostrados con el espacio en blanco.

#### Evaluación Comparativa

Para evaluar los algoritmos del TSP, **TSPLIB** es una librería de instancias de ejemplos del TSP y de problemas relacionados, véase la referencia externa TSPLIB. Muchos de estas instancias son listas de ciudades actuales y diseños de circiutos impresos actuales.

### 2.13.5 Rendimiento humano en el TSP

El RSP, en particular la variante Euclidiana del problema, atrae la atención de los investigadores en Psicología cognitiva. Estos observan que los humanos son capaces de producir rápidamente buenas soluciones. El primer ejemplar del Journal of Problem Solving es dedicado al tema del rendimiento de los humanos en el TSP.

### 2.13.6 Longitud del camino en el TSP para conjuntos de puntos aleatorios en un cuadrado

Suponiendo N puntos aleatoriamente distribuidos en un cuadrado de 1×1 con *N*>>1. Considerar muchos de estos cuadrados. Suponer que conocemos el promedio del camino más corto (por ejemplo, en la solución del TSP) de cada cuadrado.

#### Límites Inferiores

$\frac{1}{2}\sqrt{N}$ Es un límite inferior obtenido por asumir *i*un punto en la secuencia e *i* tiene como próximo vecino el último en el camino.

$\left(\frac{1}{4}+\frac{3}{8}\right)\sqrt{N}$ Es un mejor límite inferior obtenido asumiendo que el último de *i*'s es el próximo de *i*'s, y el antiguo *i*'s es el que le sigue al próximo *i*'s.

$\sqrt{\frac{N}{2}}$ Es un aún mejor límite inferior obtenido dividiendo el camino en dos partes como *antes_de_i* y *después_de_i* cada parte contiene *N*/2 puntos, y luego eliminamos *antes_de_i* formando un conjunto de puntos relajado.

- David S. Johnson[25] obtuvo un límite inferior por el cálculo del siguiente experimento:

$0.7080\sqrt{N} + 0.522$ , donde 0.522 proviene de los puntos del límite del cuadrado que contiene menos vecinos.

- Christine L. Valenzuela and Antonia J. Jones[26] obtuvieron otro límite inferior con el siguiente experimento

$0.7078\sqrt{N} + 0.551$

### 2.13.7 TSP de Analyst

Este es un problema análogo en la teoría de las medidas geométricas la cual presenta la siguiente interrogante: ¿Bajo qué condiciones puede un subconjunto *E* del espacio Euclidiano contener en una curva rectificable (esto es, cuando una curva con longitud finita visita todos los puntos en "E")? Este problema es conocido como TSP de analyst (analyst's travelling salesman problem) o TSP geométrico.

### 2.13.8 Software Libres para resolver el TSP

### 2.13.9 Cultura Popular

*Travelling Salesman*, película del 2012 dirigida por Timothy Lanzone, es una historia de 4 matemáticos contratados por el gobierno norteamericano para resolver el más difícil problema en la historia de la ciencia de la computación: P vs. NP.[27]

### 2.13.10 Vea también

- Problema del cartero chino
- Problema de los puentes de Königsberg
- Problema de transporte

### 2.13.11 Notas

[1] "Der Handlungsreisende – wie er sein soll und was er zu tun hat, um Aufträge zu erhalten und eines glücklichen Erfolgs in seinen Geschäften gewiß zu sein – von einem alten Commis-Voyageur" (El viajante — cómo debe ser y lo que debería hacer para obtener comisiones y asegurar el feliz éxito en sus negocios — por una antigua ruta comercial)

[2] Una discusión del temprano trabajo de Hamilton y Kirkman puede ser encontrado en Graph Theory 1736-1936

[3] Citado y traducido al Inglés en Schrijver (2005). Original en Alemán: "Wir bezeichnen als *Botenproblem* (weil diese Frage in der Praxis von jedem Postboten, übrigens auch von vielen Reisenden zu lösen ist) die Aufgabe, für endlich viele Punkte, deren paarweise Abstände bekannt sind, den kürzesten die Punkte verbindenden Weg zu finden. Dieses Problem ist natürlich stets durch endlich viele Versuche lösbar. Regeln, welche die Anzahl der Versuche unter die Anzahl der Permutationen der gegebenen Punkte herunterdrücken würden, sind nicht bekannt. Die Regel, man solle vom Ausgangspunkt erst zum nächstgelegenen Punkt, dann zu dem diesem nächstgelegenen Punkt gehen usw., liefert im allgemeinen nicht den kürzesten Weg."

[4] Un tratamiento detallado de la conexión entre Menger y Whitney así como también el crecimiento en el estudio de TSP puede ser encontrado en Alexander Schrijver's 2005 artículo "On the history of combinatorial optimization (till 1960). Handbook of Discrete Optimization (K. Aardal, G.L. Nemhauser, R. Weismantel, eds.), Elsevier, Amsterdam, 2005, pp. 1–68.PS,PDF

[5] Rego, César; Gamboa, Dorabela; Glover, Fred; Osterman, Colin (2011), «Traveling salesman problem heuristics: leading methods, implementations and latest advances», *European Journal of Operational Research* **211** (3): 427–441, doi:10.1016/j.ejor.2010.09.010.

[6] Behzad, Arash; Modarres, Mohammad (2002), «New Efficient Transformation of the Generalized Traveling Salesman Problem into Traveling Salesman Problem», *Proceedings of the 15th International Conference of Systems Engineering (Las Vegas)*

[7] Papadimitriou, C.H.; Steiglitz, K. (1998), *Combinatorial optimization: algorithms and complexity*, Mineola, NY: Dover, pp.308-309.

[8] Tucker, A. W. (1960), "On Directed Graphs and Integer Programs", IBM Mathematical research Project (Princeton University)

[9] Dantzig, George B. (1963), *Linear Programming and Extensions*, Princeton, NJ: PrincetonUP, pp. 545–7, ISBN 0-691-08000-3, sixth printing, 1974.

[10] Papadimitriou (1983)

[11] Christofides (1976)

[12] Berman y Karpinski (2006)

[13] Kaplan (2004)

[14] Kosaraju (1994)

[15] Serdyukov (1984)

[16] Hassin (2000)

[17] Bellman (1960), Bellman (1962), Held y Karp (1962)

[18] Woeginger (2003)

[19] Work by David Applegate, AT&T Labs – Research, Robert Bixby, ILOG and Rice University, Vašek Chvátal, Concordia University, William Cook, University of Waterloo, and Keld Helsgaun, Roskilde University is discussed on their project web page hosted by the University of Waterloo and last updated in June 2004, here

[20] Johnson, D.S. and McGeoch, L.A.. "The traveling salesman problem: A case study in local optimization", Local search in combinatorial optimization, 1997, 215-310

[21] S. S. Ray, S. Bandyopadhyay and S. K. Pal, "Genetic Operators for Combinatorial Optimization in TSP and Microarray Gene Ordering," Applied Intelligence, 2007, 26(3). pp. 183-195.

[22] A. B. Kahng and S. Reda, "Match Twice and Stitch: A New TSP Tour Construction Heuristic," Operations Research Letters, 2004, 32(6). pp. 499–509. http://dx.doi.org/10.1016/j.orl.2004.04.001

[23] Marco Dorigo. Ant Colonies for the Traveling Salesman Problem. IRIDIA, Université Libre de Bruxelles. IEEE Transactions on Evolutionary Computation, 1(1):53–66. 1997. http://citeseer.ist.psu.edu/86357.html

[24] Roy Jonker and Ton Volgenant. "Transforming asymmetric into symmetric traveling salesman problems". *Operations Research Letters* **2**:161–163, 1983. doi 10.1016/0167-6377(83)90048-2

[25] David S. Johnson

[26] Christine L. Valenzuela and Antonia J. Jones

[27] Geere, Duncan. «'Travelling Salesman' movie considers the repercussions if P equals NP». Wired. Consultado el 26 de abril de 2012.

### 2.13.12 Referencias

- Applegate, D. L.; Bixby, R. M.; Chvátal, V.; Cook, W. J. (2006), *The Traveling Salesman Problem*, ISBN 0-691-12993-2.
- Arora, Sanjeev (1998), «Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems», *Journal of the ACM* **45** (5): 753–782, doi:10.1145/290179.290180.
- Bellman, R. (1960), «Combinatorial Processes and Dynamic Programming», en Bellman, R., Hall, M., Jr. (eds.), *Combinatorial Analysis, Proceedings of Symposia in Applied Mathematics 10,*, American Mathematical Society, pp. 217–249.
- Bellman, R. (1962), «Dynamic Programming Treatment of the Travelling Salesman Problem», *J. Assoc. Comput. Mach.* **9**: 61–63, doi:10.1145/321105.321111.
- Berman, Piotr; Karpinski, Marek (2006), «8/7-approximation algorithm for (1,2)-TSP», *Proc. 17th ACM-SIAM Symposium on Discrete Algorithms (SODA '06)*, pp. 641–648, doi:10.1145/1109557.1109627, Plantilla:ECCC, ISBN 0898716055.

- Christofides, N. (1976), *Worst-case analysis of a new heuristic for the travelling salesman problem*, Technical Report 388, Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh.
- Hassin, R.; Rubinstein, S. (2000), «Better approximations for max TSP», *Information Processing Letters* **75** (4): 181–186, doi:10.1016/S0020-0190(00)00097-1.
- Held, M.; Karp, R. M. (1962), «A Dynamic Programming Approach to Sequencing Problems», *Journal of the Society for Industrial and Applied Mathematics* **10** (1): 196–210, doi:10.1137/0110015.
- Kaplan, H.; Lewenstein, L.; Shafrir, N.; Sviridenko, M. (2004), «Approximation Algorithms for Asymmetric TSP by Decomposing Directed Regular Multigraphs», *In Proc. 44th IEEE Symp. on Foundations of Comput. Sci*, pp. 56–65.
- Kosaraju, S. R.; Park, J. K.; Stein, C. (1994), «Long tours and short superstrings'», *Proc. 35th Ann. IEEE Symp. on Foundations of Comput. Sci*, IEEE Computer Society, pp. 166–177.
- Orponen, P.; Mannila, H. (1987), «On approximation preserving reductions: Complete problems and robust measures'», *Technical Report C-1987–28, Department of Computer Science, University of Helsinki*.
- Papadimitriou, Christos H. (1977), «The Euclidean traveling salesman problem is NP-complete», *Theoretical Computer Science* **4** (3): 237–244, doi:10.1016/0304-3975(77)90012-3.
- Papadimitriou, C. H.; Yannakakis, M. (1993), «The traveling salesman problem with distances one and two», *Math. Oper. Res.* **18**: 1–11, doi:10.1287/moor.18.1.1.
- Serdyukov, A. I. (1984), «An algorithm with an estimate for the traveling salesman problem of the maximum'», *Upravlyaemye Sistemy* **25**: 80–86.
- Woeginger, G.J. (2003), «Exact Algorithms for NP-Hard Problems: A Survey», *Combinatorial Optimization – Eureka, You Shrink! Lecture notes in computer science, vol. 2570*, Springer, pp. 185–207.


### 2.13.13 También leer


- Adleman, Leonard (1994), «Molecular Computation of Solutions To Combinatorial Problems», *Science* **266** (5187): 1021–4, doi:10.1126/science.7973651, PMID 7973651, Bibcode: 1994Sci...266.1021A, http://www.usc.edu/dept/molecular-science/papers/fp-sci94.pdf
- Arora, S. (1998), «Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems», *Journal of the ACM* **45** (5): 753–782, doi:10.1145/290179.290180, http://graphics.stanford.edu/courses/cs468-06-winter/Papers/arora-tsp.pdf.
- Babin, Gilbert; Deneault, Stéphanie; Laportey, Gilbert (2005), *Improvements to the Or-opt Heuristic for the Symmetric Traveling Salesman Problem*, Cahiers du GERAD, G-2005-02, Montreal: Group for Research in Decision Analysis, http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.89.9953.
- Cook, William (2011), *In Pursuit of the Travelling Salesman: Mathematics at the Limits of Computation*, Princeton University Press, ISBN 978-0-691-15270-7.
- Cook, William; Espinoza, Daniel; Goycoolea, Marcos (2007), «Computing with domino-parity inequalities for the TSP», *INFORMS Journal on Computing* **19** (3): 356–365, doi:10.1287/ijoc.1060.0204.
- Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. (2001), «35.2: The traveling-salesman problem», *Introduction to Algorithms* (2nd edición), MIT Press and McGraw-Hill, pp. 1027–1033, ISBN 0-262-03293-7.
- Dantzig, G. B.; Fulkerson, R.; Johnson, S. M. (1954), «Solution of a large-scale traveling salesman problem», *Operations Research* **2** (4): 393–410, doi:10.1287/opre.2.4.393.
- Garey, M. R.; Johnson, D. S. (1979), «A2.3: ND22–24», *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W.H. Freeman, pp. 211–212, ISBN 0-7167-1045-5.
- Goldberg, D. E. (1989), «Genetic Algorithms in Search, Optimization & Machine Learning», *Reading: Addison-Wesley* (New York: Addison-Wesley), ISBN 0-201-15767-5, Bibcode: 1989gaso.book.....G.

- Gutin, G.; Yeo, A.; Zverovich, A. (2002), «Traveling salesman should not be greedy: domination analysis of greedy-type heuristics for the TSP», *Discrete Applied Mathematics* **117** (1–3): 81–86, doi:10.1016/S0166-218X(01)00195-0.
- Gutin, G.; Punnen, A. P. (2006), *The Traveling Salesman Problem and Its Variations*, Springer, ISBN 0-387-44459-9.
- Johnson, D. S.; McGeoch, L. A. (1997), «The Traveling Salesman Problem: A Case Study in Local Optimization», en Aarts, E. H. L.; Lenstra, J. K., *Local Search in Combinatorial Optimisation*, John Wiley and Sons Ltd, pp. 215–310.
- Lawler, E. L.; Lenstra, J. K.; Rinnooy Kan, A. H. G.; Shmoys, D. B. (1985), *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*, John Wiley & Sons, ISBN 0-471-90413-9.
- MacGregor, J. N.; Ormerod, T. (1996), «Human performance on the traveling salesman problem», *Perception & Psychophysics* **58** (4): 527–539, doi:10.3758/BF03213088, http://www.psych.lancs.ac.uk/people/uploads/TomOrmerod20030716T112601.pdf.
- Mitchell, J. S. B. (1999), «Guillotine subdivisions approximate polygonal subdivisions: A simple polynomial-time approximation scheme for geometric TSP, *k*-MST, and related problems», *SIAM Journal on Computing* **28** (4): 1298–1309, doi:10.1137/S0097539796309764, http://citeseer.ist.psu.edu/622594.html.
- Rao, S.; Smith, W. (1998), «Approximating geometrical graphs via 'spanners' and 'banyans'», *Proc. 30th Annual ACM Symposium on Theory of Computing*, pp. 540–550.
- Rosenkrantz, Daniel J.; Stearns, Richard E.; Lewis, Philip M., II (1977), «An Analysis of Several Heuristics for the Traveling Salesman Problem», *SIAM Journal on Computing* **6** (5): 563–581, doi:10.1137/0206041.
- Vickers, D.; Butavicius, M.; Lee, M.; Medvedev, A. (2001), «Human performance on visually presented traveling salesman problems», *Psychological Research* **65** (1): 34–45, doi:10.1007/s004260000031, PMID 11505612.
- Walshaw, Chris (2000), *A Multilevel Approach to the Travelling Salesman Problem*, CMS Press.
- Walshaw, Chris (2001), *A Multilevel Lin-Kernighan-Helsgaun Algorithm for the Travelling Salesman Problem*, CMS Press.


### 2.13.14 Enlaces externos

- Wikimedia Commons alberga contenido multimedia sobre **Problema del viajante**. Commons
- Solución al Problema del Vendedor Viajero
- Traveling Salesman Problem at University of Waterloo
- TSPLIB at the University of Heidelberg
- *Traveling Salesman Problem* by Jon McLoone at the Wolfram Demonstrations Project
- Source code library for the travelling salesman problem
- TSP solvers in R for symmetric and asymmetric TSPs. Implements various insertion, nearest neighbor and 2-opt heuristics and an interface to Concorde and Chained Lin-Kernighan heuristics.
- Traveling Salesman movie (on IMDB)
- "Traveling Salesman in Python and Linear Optimization, IBM Developerworks with Source Code" by Noah Gift

# Capítulo 3

# Text and image sources, contributors, and licenses

## 3.1 Text

- **Árbol (teoría de grafos)** *Fuente:* http://es.wikipedia.org/wiki/%C3%81rbol%20(teor%C3%ADa%20de%20grafos)?oldid=80226029 *Colaboradores:* Tano4595, Rondador, Skiel85, AlfonsoERomero, Maleiva, BOTijo, Paintman, Kn, CEM-bot, Alexav8, Ignacio Icke, Davius, Mcetina, Ingenioso Hidalgo, Clementito, Zifra, JAnDbot, Sergio Alvaré, Muro Bot, SieBot, Arlekean, Gato ocioso, Farisori, David0811, Dennis6492, Luisbona, Rubinbot, Ricardogpn, Itsukki, RedBot, DixonDBot, EmausBot, Spyglass007, MerlIwBot, KLBot2, Elvisor, Gonzalo.villarreal, Zhirose y Anónimos: 16
- **Árbol binario** *Fuente:* http://es.wikipedia.org/wiki/%C3%81rbol%20binario?oldid=81480871 *Colaboradores:* Sabbut, Angus, Sanbec, Dodo, Triku, Ascánder, JaNoX, Cinabrium, Porao, Periku, Chewie, Boticario, Petronas, JMPerez, Rembiapo pohyiete (bot), Chobot, Caiserbot, Yrbot, Dagavi, Vitamine, Mortadelo2005, Icvav, Gmarquez, Maldoror, Tomatejc, BOTpolicia, CEM-bot, Laura Fiorucci, Dvdgc, PabloCastellano, Templeir, Isha, Osiris fancy, JAnDbot, Muro de Aguas, Netito777, Pólux, Cinevoro, C'est moi, Galandil, Muro Bot, Tigrera, MetsBot~eswiki, StarBOT, Farisori, Eduardosalg, Fenrihr, Açipni-Lovrij, Jaag12, JoseTAD, UA31, AVBOT, NjardarBot, Diegusjaimes, MelancholieBot, Victormoz, Rimeju, Ptbotgourou, SuperBraulio13, Xqbot, Rodoelgrande, Pyr0, RedBot, Tux juanda95, Dinamik-bot, Humbefa, EmausBot, Johnanth, Alfredogtzh, Rosewitchy, Grillitus, Rezabot, MerlIwBot, Minsbot, JYBot, Ralgisbot, Rauletemunoz, Toxa Kniotee, Addbot y Anónimos: 144
- **Árbol binario de búsqueda** *Fuente:* http://es.wikipedia.org/wiki/%C3%81rbol%20binario%20de%20b%C3%BAsqueda?oldid=77830077 *Colaboradores:* Hashar, Zwobot, Porao, Chewie, FAR, LeonardoRob0t, Orgullobot~eswiki, Platonides, Yrbot, BOT-Superzerocool, Vitamine, YurikBot, Er Komandante, Fercufer, CEM-bot, Thijs!bot, BotOn, JAnDbot, Albries, NaBUru38, Rei-bot, Tirabo, 3coma14, Muro Bot, YonaBot, SieBot, Knzio, Drinibot, Bigsus-bot, BOTarate, Andresluna2007, Aluna2007, PixelBot, Leonpolanco, ErSame, Joanga, Loly bc15~eswiki, Apj, BotSottile, AVBOT, MastiBot, Ciberjovial, Luckas-bot, Alelapenya, SuperBraulio13, Xqbot, Ricardogpn, Bot0811, ViajeroEspacial, Yago AB, Mr.Ajedrez, Ripchip Bot, Humbefa, Invadibot, Bibliofilotranstornado, MetalGuns, Maucendon, Darkmeow, Wjuarezq, Addbot, Kimizombie y Anónimos: 46
- **Notación de infijo** *Fuente:* http://es.wikipedia.org/wiki/Notaci%C3%B3n%20de%20infijo?oldid=65237154 *Colaboradores:* GermanX, Muro Bot, RedBot, Bender2k14 y Addbot
- **Grafo completo** *Fuente:* http://es.wikipedia.org/wiki/Grafo%20completo?oldid=74852417 *Colaboradores:* Rondador, GermanX, Boja, Rdaneel, Thijs!bot, Escarbot, Yeza, Zifra, JAnDbot, Linkedark, VolkovBot, AlleborgoBot, Farisori, LucienBOT, MastiBot, DumZiBoT, Vic Fede, DSisyphBot, EABOT, Paladium, MerlIwBot, Ramzysamman, Elvisor, Legobot y Anónimos: 5
- **Notación polaca** *Fuente:* http://es.wikipedia.org/wiki/Notaci%C3%B3n%20polaca?oldid=81492846 *Colaboradores:* GermanX, CEM-bot, JAnDbot, TXiKiBoT, Marvelshine, Matdrodes, Shooke, Muro Bot, SieBot, Ensada, Drinibot, Bigsus-bot, DragonBot, Farisori, Global.minimum, Alexbot, Numen17, Louperibot, MastiBot, Diegusjaimes, Xpicto, Luckas-bot, Jkbw, Halfdrag, RedBot, Ripchip Bot, Elhalconingles, Elvisor, Addbot, Ineditable y Anónimos: 18
- **Notación polaca inversa** *Fuente:* http://es.wikipedia.org/wiki/Notaci%C3%B3n%20polaca%20inversa?oldid=81492869 *Colaboradores:* Sabbut, Jynus, GermanX, Banfield, Tamorlan, CEM-bot, Santhy, Rjelves, Escarbot, JAnDbot, Xsm34, Xrjunque, Urdangaray, Koldito, Shooke, Muro Bot, Edmenb, Sebasmagri, Jarisleif, DragonBot, Numen17, SilvonenBot, Louperibot, Nallimbot, DiegoFb, ArthurBot, Xqbot, Cally Berry, RedBot, KamikazeBot, EmausBot, KLBot2, Bender2k14, Andressote, YFdyh-bot, Syum90, Addbot y Anónimos: 20
- **Árbol binario de búsqueda auto-balanceable** *Fuente:* http://es.wikipedia.org/wiki/%C3%81rbol%20binario%20de%20b%C3%BAsqueda%20auto-balanceable?oldid=70830870 *Colaboradores:* Porao, Rembiapo pohyiete (bot), RobotQuistnix, Yrbot, Aliciadr, Cesarsorm, YurikBot, GermanX, Damifb, VolkovBot, Josell2, Muro Bot, PaintBot, PixelBot, AVBOT, Xqbot, EmausBot, Addbot y Anónimos: 4
- **Rotación de árboles** *Fuente:* http://es.wikipedia.org/wiki/Rotaci%C3%B3n%20de%20%C3%A1rboles?oldid=80557481 *Colaboradores:* Gejotape, Grillitus, MetroBot y Javier Borrego Fernandez C-512
- **Árbol-B** *Fuente:* http://es.wikipedia.org/wiki/%C3%81rbol-B?oldid=79056084 *Colaboradores:* Pino, JorgeGG, Hashar, Zwobot, Vcarceler, Ascánder, Avm, Elwikipedista, Aeb~eswiki, Dianai, Jecanre, Jac, Porao, Pixeltoo, LeonardoRob0t, Rembiapo pohyiete (bot), RobotQuistnix, Platonides, Superzerocool, Paradoja, Dibujon, Yrbot, Cesarsorm, YurikBot, Jgaray, Eduardo Lima, Chlewbot, Mencey, Juandiegocano, CEM-bot, Laura Fiorucci, Dbot, Ricardo cm, Crates, Xoneca, JAnDbot, Muro de Aguas, VolkovBot, Karras, Muro Bot, Rv53705, Loveless, Drinibot, Bigsus-bot, Nubecosmica, DorganBot, MetsBot~eswiki, Panypeces, Leonpolanco, Hichamaliouat~eswiki, Mr freeze360, AVBOT, DumZiBoT, Luckas-bot, LordboT, SuperBraulio13, Almabot, Grupoarbolb, Xqbot, Humbefa, GrouchoBot, Arthur 'Two Sheds' Jackson, MetroBot, Ralgisbot, YFdyh-bot, Ouro86, Addbot y Anónimos: 61

- **Montículo (informática)** *Fuente:* http://es.wikipedia.org/wiki/Mont%C3%ADculo%20(inform%C3%A1tica)?oldid=80124142 *Colaboradores:* Crescent Moon, Ascánder, Porao, Klemen Kocjancic, Rembiapo pohyiete (bot), RobotQuistnix, Yrbot, YurikBot, Javi007, CEM-bot, Alexisabarca, Ggenellina, Thijs!bot, Rafael Gálvez, JAnDbot, Gsrdzl, TXiKiBoT, Netito777, Jvlivs, Biasoli, VolkovBot, BotMultichill, SieBot, Loveless, Djblack!, BOTarate, Nubecosmica, Brayan Jaimes, Elkim2, Jduarte, Poco a poco, Bgangioni, MelancholieBot, Luckas-bot, Nallimbot, Jotterbot, Stefano4jc, ArthurBot, Xqbot, Arroy, Luisbona, KamikazeBot, ZanPATXImbos, EmausBot, Grillitus, WikitanvirBot, Johnbot, Addbot, Belkis borja y Anónimos: 19
- **Montículo de Fibonacci** *Fuente:* http://es.wikipedia.org/wiki/Mont%C3%ADculo%20de%20Fibonacci?oldid=68519297 *Colaboradores:* Magister Mathematicae, BOTijo, GermanX, Javi007, Jarke, CEM-bot, Rosarinagazo, JAnDbot, Ignacioerrico, Almendro, Raystorm, Muro Bot, Loveless, Djblack!, Doks, LucienBOT, Diegusjaimes, Luckas-bot, Amirobot, Arroy, Luisbona, Carlos Molina Fisico, EmausBot, Addbot y Anónimos: 4
- **Grafo** *Fuente:* http://es.wikipedia.org/wiki/Grafo?oldid=80226923 *Colaboradores:* Pino, Oblongo, Moriel, Juan M Taboada, Ascánder, Tano4595, La Mantis, Digigalos, AlfonsoERomero, Dromero, Yrbot, BOTijo, Armin76, KnightRider, Txo, Er Komandante, Juan Marquez, Ilan.ag1, Kn, BOTpolicia, CEM-bot, Laura Fiorucci, AndresDominguez, Baiji, Mcetina, Fsd141, Thijs!bot, Roberto Fiadone, RoyFocker, IrwinSantos, Ángel Luis Alfaro, Jtoselli, Botones, Mpeinadopa, JAnDbot, Wybot, TXiKiBoT, Ignacioerrico, Cinevoro, Acosta89, Matdrodes, Muro Bot, Manuc82, Meldor, SieBot, Farisori, LordT, Rrupo, UA31, AVBOT, Elliniká, MastiBot, Adelpine, Diegusjaimes, Arjuno3, Luckas-bot, Jotterbot, DirlBot, Jkbw, Rilog, Paladium, AlemanI2.0, PatruBOT, KamikazeBot, TjBot, Ripchip Bot, CentroBabbage, Gulliberto, GrouchoBot, CocuBot, MerlIwBot, UAwiki, Addihockey10 (automated), Bxs3, RosenJax, Makecat-bot, YFdyh-bot, Keyla Herrera Ruiz, Addbot, Zhirose y Anónimos: 51
- **Teoría de grafos** *Fuente:* http://es.wikipedia.org/wiki/Teor%C3%ADa%20de%20grafos?oldid=81405011 *Colaboradores:* Pino, Rumpelstiltskin, Rosarino, Ascánder, Julian Colina, Tano4595, PeiT, Taxman, Rondador, La Mantis, AlfonsoERomero, Emijrp, Rmolina, RobotQuistnix, Alhen, BOT-Superzerocool, Boku wa kage, YurikBot, Equi, Otermin, Tomatejc, Mencey, Nihilo, Tessier, Ilan.ag1, BOTpolicia, Adama~eswiki, Ludoviko, Rdaneel, Hawking, CEM-bot, Ferminmx, JMCC1, Alexav8, Baiji, Roberpl, Mcetina, Ingenioso Hidalgo, Thijs!bot, Ingridchan, Pablo Olmos, Tortillovsky, Davidrodriguez, IrwinSantos, Seba.29.8, Zifra, Sapiensjpa, Botones, Gusgus, Mpeinadopa, JAnDbot, Nando.sm, Inmortra, Aalvarez12, Gullo, Humberto, Pabloallo, Esteban fcr, Chabbot, Pólux, Rovnet, Mundokeko, VolkovBot, XinuXano, Elmermosher, Matdrodes, AlleborgoBot, Muro Bot, Feministo, Numbo3, Gerakibot, SieBot, Loveless, Drinibot, Macarse, Raul.lara, Xiscobernal, PipepBot, Schwallex, Gato ocioso, Farisori, Eduardosalg, Neodop, Botellín, Leonpolanco, LordT, Juan Mayordomo, Darkicebot, Aikasse, BodhisattvaBot, Raulshc, AVBOT, LucienBOT, MastiBot, Diegusjaimes, A.garridob, Luckas-bot, Ricardo Castelo, El Quinche, Kilom691, Vic Fede, Habilbakhat, Marioxcc, Medium69, Diogeneselcinico42, SuperBraulio13, Xqbot, Jkbw, Angenio2, Botarel, Andrestand, Paladium, Linux65, RedBot, Wsu-dm-jb, Galileicanarias, Dinamik-bot, TjBot, Ripchip Bot, Dark Bane, Jorge c2010, Foundling, EleferenBot, Axvolution, EmausBot, Savh, Khaos258, Grillitus, ChuispastonBot, Alejozsz, AvocatoBot, MetroBot, Invadibot, SoleFabrizio, Leitoxx, Addbot, DaveGomez, JacobRodrigues, Gonzalo.villarreal, Julian Araoz, Lectorina y Anónimos: 168
- **Grafo dirigido** *Fuente:* http://es.wikipedia.org/wiki/Grafo%20dirigido?oldid=77494308 *Colaboradores:* LogC, VolkovBot, Farisori, Juan Mayordomo, Jotterbot, Jerowiki, MerlIwBot, KLBot2, Melina.testori y Anónimos: 4
- **Matriz de adyacencia** *Fuente:* http://es.wikipedia.org/wiki/Matriz%20de%20adyacencia?oldid=80336431 *Colaboradores:* Jespa, GermanX, Davius, Thijs!bot, Fernandopcg, Escarbot, Uruk, VolkovBot, Matdrodes, Muro Bot, Srbanana, Periergeia, Dnu72, Farisori, PixelBot, Alexbot, Adelpine, Luckas-bot, Nallimbot, Habilbakhat, Yonidebot, ArthurBot, Panderine!, TobeBot, Savh, ZéroBot, ChuispastonBot, WikitanvirBot, Thelamproject, MerlIwBot, KLBot2 y Anónimos: 13
- **Algoritmo de Floyd-Warshall** *Fuente:* http://es.wikipedia.org/wiki/Algoritmo%20de%20Floyd-Warshall?oldid=82228246 *Colaboradores:* Dodo, Ejmeza, Tano4595, Rondador, Rembiapo pohyiete (bot), Aliman5040, RobotQuistnix, Yrbot, BOTijo, Oscarif, GermanX, M4r10c354r, CEM-bot, Julian Mendez, Ingenioso Hidalgo, MSBOT, JAnDbot, Netito777, VolkovBot, Muro Bot, SieBot, Victorespejo, PaintBot, Bigsus-bot, BOTarate, Belb, Kikobot, Farisori, Alecs.bot, LordT, Juan Mayordomo, UA31, AVBOT, MastiBot, Ockie, Luckas-bot, ArthurBot, Jkbw, Jipsy, Cem-auxBOT, EmausBot, JackieBot, WikitanvirBot, Dariog88, KLBot2, Invadibot, Elvisor y Anónimos: 35
- **Problema del camino más corto** *Fuente:* http://es.wikipedia.org/wiki/Problema%20del%20camino%20m%C3%A1s%20corto?oldid=82296398 *Colaboradores:* Tony Rotondas, GermanX, CEM-bot, CommonsDelinker, Norman rengifo, Antón Francho, Farisori, Poco a poco, Juan Mayordomo, Luckas-bot, Xqbot, Jipsy, MAfotBOT, RedBot, AnselmiJuan, WikitanvirBot, KLBot2, Gustavoiglesias y Anónimos: 4
- **Búsqueda en anchura** *Fuente:* http://es.wikipedia.org/wiki/B%C3%BAsqueda%20en%20anchura?oldid=78711562 *Colaboradores:* Sabbut, Robbot, Alberto Salguero, Dodo, Rondador, Almorca, AlfonsoERomero, JMPerez, Taichi, Rembiapo pohyiete (bot), Orgullobot~eswiki, RobotQuistnix, Chobot, Yrbot, BOT-Superzerocool, YurikBot, Wiki-Bot, GermanX, KnightRider, Regnaron~eswiki, Chlewbot, BOTpolicia, Davidrodriguez, RoyFocker, JAnDbot, TXiKiBoT, VolkovBot, Pmontaldo, Muro Bot, Macarse, Farisori, VanBot, MastiBot, DumZiBoT, Fuelusumar, Luckas-bot, Xqbot, Jkbw, Rubinbot, MondalorBot, ErikvanB, Dinamik-bot, EmausBot, ChessBOT, YFdyh-bot, Addbot, Fquinto bcn y Anónimos: 27
- **Búsqueda en profundidad** *Fuente:* http://es.wikipedia.org/wiki/B%C3%BAsqueda%20en%20profundidad?oldid=77849870 *Colaboradores:* Sabbut, Almorca, Caos, Taragui, Edrabc~eswiki, RobotQuistnix, Chobot, Yrbot, BOT-Superzerocool, GermanX, CEM-bot, Pinar~eswiki, Mcetina, Davidrodriguez, JAnDbot, Pmontaldo, Muro Bot, SieBot, Gswarlus, Farisori, VanBot, SilvonenBot, AVBOT, DumZiBoT, Fuelusumar, Luckas-bot, Xqbot, Jkbw, Botarel, RedBot, ErikvanB, AngelTC, EmausBot, ZéroBot, ChessBOT, ChuispastonBot, Pore1991, Addbot y Anónimos: 19
- **Vuelta atrás** *Fuente:* http://es.wikipedia.org/wiki/Vuelta%20atr%C3%A1s?oldid=80490990 *Colaboradores:* Dianai, Txuspe, AlfonsoERomero, JMPerez, Dem, Dromero, Yrbot, FlaBot, BOTijo, YurikBot, Manolo456, Eskimbot, Lasneyx, CEM-bot, Jgomo3, Betty1984, Alva~eswiki, AlbertMA, Pablete im, Moonrak, Pablo Olmos, JAnDbot, Rei-bot, VolkovBot, Muro Bot, SieBot, Juan Mayordomo, Varyvol, Diegusjaimes, DumZiBoT, MelancholieBot, Arjuno3, Viajeero, Diegotorquemada, ArthurBot, Obersachsebot, Xqbot, D'ohBot, Ganímedes, TjBot, Ykanada~eswiki, SebastianRC, EdoBot, MerlIwBot, KLBot2, AvocatoBot, Rubenlagus, Vetranio, Helmy oved, Makecat-bot, Hbb9, CalongeManu y Anónimos: 26
- **Ordenación topológica** *Fuente:* http://es.wikipedia.org/wiki/Ordenaci%C3%B3n%20topol%C3%B3gica?oldid=67074904 *Colaboradores:* Jynus, Jecanre, CEM-bot, Davius, Davidrodriguez, WikiCholi, Machao, King of Hearts, Muro Bot, Gswarlus, Luckas-bot, Allforrous, ChuispastonBot, Elvisor, Addbot y Anónimos: 4

- **Algoritmo de Dijkstra** *Fuente:* http://es.wikipedia.org/wiki/Algoritmo%20de%20Dijkstra?oldid=80252714 *Colaboradores:* Riviera, Dodo, Ejmeza, Ascánder, Sms, Tano4595, Tin nqn, Birckin, Dianai, Jacobo Tarrio, Jecanre, Rondador, Renabot, Cosmonauta~eswiki, Rembiapo pohyiete (bot), Orgullobot~eswiki, RobotQuistnix, Chobot, Yrbot, BOTijo, YurikBot, GermanX, KnightRider, YoaR, Juan Antonio Cordero, Maldoror, FrozenFlame, Jarke, Fer31416, BOTpolicia, CEM-bot, Retama, Eli22, Baiji, Roberpl, Antur, Joemmanuel, Mcetina, Thijs!bot, Diosa, Davidrodriguez, JoaquinFerrero, Soulbot, Miguelo on the road, Rafa3040, Muro de Aguas, CommonsDelinker, TXiKiBoT, Netito777, Rei-bot, Verseek, Chabbot, Pólux, Dusan, VolkovBot, Urdangaray, Matdrodes, Shooke, Lucien leGrey, AlleborgoBot, Muro Bot, SieBot, Loveless, Djblack!, CASF, Macarse, Manwë, Juanmihd, Tirithel, Ligimeno, Farisori, Eduardosalg, Alecs.bot, Botito777, SilvonenBot, AVBOT, HanPritcher, Akumy, Diegusjaimes, DumZiBoT, Latamyk, InflaBOT, Luckas-bot, Yonidebot, ArthurBot, Jkbw, Rubinbot, Wikimelf, Leandro.ditommaso, Timm~eswiki, Bmiso, Hspitia, Jipsy, D'ohBot, E404, MondalorBot, KennyMcC, EmausBot, Khaos258, ChuispastonBot, Dariog88, KLBot2 y Anónimos: 154
- **Algoritmo voraz** *Fuente:* http://es.wikipedia.org/wiki/Algoritmo%20voraz?oldid=80855958 *Colaboradores:* Sanbec, Riviera, Tano4595, Porao, San650, AlfonsoERomero, JMPerez, Ricki.rg, RobotQuistnix, LuchoX, Dromero, Yrbot, YurikBot, KnightRider, Gothmog, C-3POrao, Kn, CEM-bot, TheOm3ga, Davius, Gafotas, Manz, Thijs!bot, Esf3ra, AntonioLG, Jurgens~eswiki, JAnDbot, TXiKiBoT, Chabbot, VolkovBot, AlleborgoBot, Muro Bot, Edmenb, Estigma, PaintBot, Danieldmoreno, Arlekean, Tirithel, LordT, BotSottile, AVBOT, Karj, SuperBraulio13, Jkbw, ZéroBot, MerlIwBot, KLBot2, Nabil py, Makecat-bot y Anónimos: 30
- **Problema del viajante** *Fuente:* http://es.wikipedia.org/wiki/Problema%20del%20viajante?oldid=80626323 *Colaboradores:* Riviera, Crescent Moon, RGLago, Jarfil, Xenoforme, Marctc, Deleatur, Yrithinnd, Unf, Yrbot, Beto29, Miguelio, Jesuja, Banfield, Tamorlan, CEM-bot, Alexav8, Gafotas, PabloCastellano, Pablohe, Zifra, JAnDbot, TXiKiBoT, Capiscuas, Dhidalgo, Cinevoro, XinuXano, Muro Bot, Anual, Arlekean, BOTarate, Crexcent Moon, DragonBot, Farisori, Alecs.bot, Raulshc, Sbelza, DumZiBoT, MelancholieBot, Jorge.maturana, Letuño, Yonidebot, Hirnschlacht, ArthurBot, Xqbot, Realwizard~eswiki, Hprmedina, Niko.villano, AnselmiJuan, Ripchip Bot, EmausBot, Africanus, Grillitus, Ylermicl, KLBot2, MetroBot, Invadibot, JacobRodrigues, DafneGarciaDeArmas y Anónimos: 23

## 3.2 Images

- **Archivo:10-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/8/86/10-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:11-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/9/9b/11-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:3-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/b/be/3-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:4-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/2d/4-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:5-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/e/e9/5-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:6-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/c8/6-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:6n-graf.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/5/5b/6n-graf.svg *Licencia:* Public domain *Colaboradores:* Image:6n-graf.png simlar input data *Artista original:* User:AzaToth
- **Archivo:6n-graph2.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/28/6n-graph2.svg *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:7-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/cb/7-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:7_bridges.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/9/91/7_bridges.svg *Licencia:* CC-BY-SA-3.0 *Colaboradores:* ? *Artista original:* ?
- **Archivo:8-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/2c/8-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:9-simplex_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/b/bb/9-simplex_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:ABBEJEM.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/6/69/ABBEJEM.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Joanga
- **Archivo:ABBHOJA3.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/a/ac/ABBHOJA3.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Joanga
- **Archivo:ABBHOJA4.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/b/ba/ABBHOJA4.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Joanga
- **Archivo:ABBHOJA5.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/1/1b/ABBHOJA5.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Joanga
- **Archivo:Aco_TSP.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/2a/Aco_TSP.svg *Licencia:* CC-BY-SA-3.0 *Colaboradores:* ? *Artista original:* ?
- **Archivo:AntColony.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/8/8c/AntColony.gif *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Saurabh.harsh
- **Archivo:Arbolbelimnodom.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/7/79/Arbolbelimnodom.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Hichamaliouat
- **Archivo:Arbolbins.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/0/0c/Arbolbins.png *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* MyName (hicham aliouat)

- **Archivo:BinaryTreeRotations.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/4/43/BinaryTreeRotations.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Josell7
- **Archivo:Binary_search_tree.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/d/da/Binary_search_tree.svg *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Binary_tree_(oriented_digraph).png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/36/Binary_tree_%28oriented_digraph%29.png *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Branch&bound_low.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/b/bb/Branch%26bound_low.jpg *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Branchbound.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/3c/Branchbound.gif *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Saurabh.harsh
- **Archivo:Breadth-first-tree.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/33/Breadth-first-tree.svg *Licencia:* CC BY 3.0 *Colaboradores:* Trabajo propio *Artista original:* Alexander Drichel
- **Archivo:Bruteforce.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/2b/Bruteforce.gif *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Saurabh.harsh
- **Archivo:Caminosmascortos.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/e/ea/Caminosmascortos.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Jipsy
- **Archivo:Commons-emblem-question_book_orange.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/1/1f/Commons-emblem-question_book_orange.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* <a href='//commons.wikimedia.org/wiki/File:Commons-emblem-issue.svg' class='image'><img alt='Commons-emblem-issue.svg' src='//upload.wikimedia.org/wikipedia/commons/thumb/b/bc/Commons-emblem-issue.svg/25px-Commons-emblem-issue.svg.png' width='25' height='25' srcset='//upload.wikimedia.org/wikipedia/commons/thumb/b/bc/Commons-emblem-issue.svg/38px-Commons-emblem-issue.svg.png 1.5x, //upload.wikimedia.org/wikipedia/commons/thumb/b/bc/Commons-emblem-issue.svg/50px-Commons-emblem-issue.svg.png 2x' data-file-width='48' data-file-height='48' /></a> + <a href='//commons.wikimedia.org/wiki/File:Question_book.svg' class='image'><img alt='Question book.svg' src='//upload.wikimedia.org/wikipedia/commons/thumb/9/97/Question_book.svg/25px-Question_book.svg.png' width='25' height='20' srcset='//upload.wikimedia.org/wikipedia/commons/thumb/9/97/Question_book.svg/38px-Question_book.svg.png 1.5x, //upload.wikimedia.org/wikipedia/commons/thumb/9/97/Question_book.svg/50px-Question_book.svg.png 2x' data-file-width='252' data-file-height='199' /></a> *Artista original:* GNOME icon artists, Jorge 2701
- **Archivo:Commons-logo.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/4/4a/Commons-logo.svg *Licencia:* Public domain *Colaboradores:* This version created by Pumbaa, using a proper partial circle and SVG geometry features. (Former versions used to be slightly warped.) *Artista original:* SVG version was created by User:Grunt and cleaned up by 3247, based on the earlier PNG version, created by Reidab.
- **Archivo:Complete_graph_K1.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/a/ad/Complete_graph_K1.svg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* David Benbennick wrote this file.
- **Archivo:Complete_graph_K2.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/9/96/Complete_graph_K2.svg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* David Benbennick wrote this file.
- **Archivo:Complete_graph_K3.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/5/5a/Complete_graph_K3.svg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* David Benbennick wrote this file.
- **Archivo:Complete_graph_K7.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/9/9e/Complete_graph_K7.svg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* David Benbennick
- **Archivo:Connexe_et_pas_connexe.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/6/65/Connexe_et_pas_connexe.svg *Licencia:* CC BY-SA 4.0 *Colaboradores:* Trabajo propio *Artista original:* Kilom691
- **Archivo:Dart_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/9/96/Dart_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:Depth-first-tree.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/1/1f/Depth-first-tree.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Alexander Drichel
- **Archivo:Dijkstra_Animation.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/5/57/Dijkstra_Animation.gif *Licencia:* Public domain *Colaboradores:* Work by uploader. *Artista original:* Ibmua
- **Archivo:Directed.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/a/a2/Directed.svg *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Directed_acyclic_graph.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/0/08/Directed_acyclic_graph.png *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Dodecahedral_graph.neato.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/0/05/Dodecahedral_graph.neato.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:Fibonacci_heap-decreasekey.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/0/09/Fibonacci_heap-decreasekey.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* ? *Artista original:* ?
- **Archivo:Fibonacci_heap.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/4/45/Fibonacci_heap.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* ? *Artista original:* ?
- **Archivo:Fibonacci_heap_extractmin1.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/5/56/Fibonacci_heap_extractmin1.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* ? *Artista original:* ?
- **Archivo:Fibonacci_heap_extractmin2.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/9/95/Fibonacci_heap_extractmin2.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* ? *Artista original:* ?
- **Archivo:Fish_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/e/ee/Fish_graph.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Koko90
- **Archivo:Grafo_cot.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/6/6a/Grafo_cot.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Davidroriguez

- **Archivo:Grafo_ejemplo_3_árbol.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/cc/Grafo_ejemplo_3_%C3%A1rbol.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Transferido desde es.wikipedia a Commons. *Artista original:* Romero Schmidtke de Wikipedia en español
- **Archivo:Grafo_ejemplo_5_conecsi.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/5/50/Grafo_ejemplo_5_conecsi.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Transferido desde es.wikipedia a Commons. *Artista original:* Romero Schmidtke de Wikipedia en español
- **Archivo:Grafo_ejemplo_5_países.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/5/5e/Grafo_ejemplo_5_pa%C3%ADses.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Transferido desde es.wikipedia a Commons. *Artista original:* Romero Schmidtke de Wikipedia en español
- **Archivo:Grafo_ejemplo_6.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/0/04/Grafo_ejemplo_6.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Transferido desde es.wikipedia a Commons. *Artista original:* Romero Schmidtke de Wikipedia en español
- **Archivo:Grafo_ejemplo_7.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/f/f7/Grafo_ejemplo_7.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Transferido desde es.wikipedia a Commons. *Artista original:* Romero Schmidtke de Wikipedia en español
- **Archivo:Grafo_sot.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/31/Grafo_sot.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Davidrodriguez
- **Archivo:Grafodirigido.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/0/07/Grafodirigido.jpg *Licencia:* CC0 *Colaboradores:* Trabajo propio *Artista original:* Thelamproject
- **Archivo:Greedy_algorithm_36_cents.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/d/da/Greedy_algorithm_36_cents.svg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Nandhp
- **Archivo:Hamiltonian_path.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/6/60/Hamiltonian_path.svg *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Trabajo propio *Artista original:* Christoph Sommer
- **Archivo:Infix-dia.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/a/a2/Infix-dia.svg *Licencia:* Public domain *Colaboradores:*
- Infix-dia.png *Artista original:* Infix-dia.png: blahedo
- **Archivo:Insertar.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/c0/Insertar.svg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Gorivero
- **Archivo:Insertarelemmonticulo.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/34/Insertarelemmonticulo.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Elkim2
- **Archivo:Kaari_suunnattu_graafiteoria.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/32/Kaari_suunnattu_graafiteoria.png *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Kaari_suuntaamaton_graafiteoria.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/6/67/Kaari_suuntaamaton_graafiteoria.png *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Konigsberg_bridges.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/5/5d/Konigsberg_bridges.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Public domain (PD), based on the image
  - <a href='//commons.wikimedia.org/wiki/File:Image-Koenigsberg,_Map_by_Merian-Erben_1652.jpg' class='image'><img alt='Image-Koenigsberg, Map by Merian-Erben 1652.jpg' src='//upload.wikimedia.org/wikipedia/commons/thumb/1/15/Image-Koenigsberg%2C_Map_by_Merian-Erben_1652.jpg/120px-Image-Koenigsberg%2C_Map_by_Merian-Erben_1652.jpg' width='120' height='84' srcset='//upload.wikimedia.org/wikipedia/commons/thumb/1/15/Image-Koenigsberg%2C_Map_by_Merian-Erben_1652.jpg/180px-Image-Koenigsberg%2C_Map_by_Merian-Erben_1652.jpg 1.5x, //upload.wikimedia.org/wikipedia/commons/thumb/1/15/Image-Koenigsberg%2C_Map_by_Merian-Erben_1652.jpg/240px-Image-Koenigsberg%2C_Map_by_Merian-Erben_1652.jpg 2x' data-file-width='628' data-file-height='437' /></a>

  *Artista original:* Bogdan Giuşcă
- **Archivo:Lista_nodos.JPG** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/26/Lista_nodos.JPG *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Monticulo.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/a/af/Monticulo.jpg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* Jduarte
- **Archivo:N-ary_to_binary.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/cd/N-ary_to_binary.svg *Licencia:* Public domain *Colaboradores:* Trabajo propio *Artista original:* CyHawk
- **Archivo:Nearestneighbor.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/23/Nearestneighbor.gif *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* Saurabh.harsh
- **Archivo:Nuvola_apps_edu_mathematics-p.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/c2/Nuvola_apps_edu_mathematics-p.svg *Licencia:* GPL *Colaboradores:* Derivative of Image:Nuvola apps edu mathematics.png created by self *Artista original:* David Vignoni (original icon); Flamurai (SVG convertion)
- **Archivo:Portal.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/c9/Portal.svg *Licencia:* CC BY 2.5 *Colaboradores:*
  - Portal.svg

  *Artista original:* Portal.svg: Pepetps
- **Archivo:Postfix-dia.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/a/a6/Postfix-dia.svg *Licencia:* Public domain *Colaboradores:*
- Postfix-dia.png *Artista original:* Postfix-dia.png: blahedo
- **Archivo:Prefix-dia.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/0/09/Prefix-dia.png *Licencia:* Public domain *Colaboradores:* made by me *Artista original:* blahedo

- **Archivo:Spanish_Language_Wiki.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/2a/Spanish_Language_Wiki.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Derived from Wiki puzzle.svg by user:Kimbar *Artista original:* James.mcd.nz
- **Archivo:Tree_Rebalancing.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/c/c4/Tree_Rebalancing.gif *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Transferred from en.wikipedia; transferred to Commons by User:Common Good using CommonsHelper. *Artista original:* Original uploader was Mtanti at en.wikipedia
- **Archivo:Tree_Rotations.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/1/15/Tree_Rotations.gif *Licencia:* CC-BY-SA-3.0 *Colaboradores:* Transferido desde en.wikipedia a Commons. *Artista original:* Mtanti de Wikipedia en inglés
- **Archivo:Tree_graph.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/24/Tree_graph.svg *Licencia:* Public domain *Colaboradores:* ? *Artista original:* ?
- **Archivo:Tree_rotation.png** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/2/23/Tree_rotation.png *Licencia:* CC-BY-SA-3.0 *Colaboradores:* EN-Wikipedia *Artista original:* User:Ramasamy
- **Archivo:Tree_rotation_animation_250x250.gif** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/31/Tree_rotation_animation_250x250.gif *Licencia:* CC BY-SA 4.0 *Colaboradores:* Trabajo propio *Artista original:* Tar-Elessar
- **Archivo:Weighted_K4.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/3/30/Weighted_K4.svg *Licencia:* CC BY-SA 2.5 *Colaboradores:* self-made using xfig *Artista original:* Sdo
- **Archivo:Wikibooks-logo.svg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/f/fa/Wikibooks-logo.svg *Licencia:* CC BY-SA 3.0 *Colaboradores:* Trabajo propio *Artista original:* User:Bastique, User:Ramac et al.
- **Archivo:William_Rowan_Hamilton_painting.jpg** *Fuente:* http://upload.wikimedia.org/wikipedia/commons/1/15/William_Rowan_Hamilton_painting.jpg *Licencia:* Public domain *Colaboradores:* http://www.askaboutireland.ie/search.xml?query=William+Rowan+Hamilton&radio_filter=images&type=and *Artista original:* Desconocido

## 3.3 Content license

- Creative Commons Attribution-Share Alike 3.0

View publication stats